
Machine Learning Laboratory Manual

Completed Edition with Full Implementations and Results

Practical Guide to Classical, Ensemble, Neural, Probabilistic,
Density-Based, Semi-Supervised and Generative Machine Learning

Prepared by

Dr. Ohidujjaman Tuhin

Associate Professor, Dept. of CSE, UIU

Department of Computer Science & Engineering
Machine Learning Lab

September 26, 2026

Contents

List of Experiments	iii
List of Figures	iv
List of Tables	v
Preface	vi
Important Terminology Correction	vi
Scope of the Laboratory	vi
General Learning Outcomes	vi
Recommended Software Environment	vii
Laboratory Guidelines	vii
Minimum Report Format	vii
Recommended Evaluation Metrics.	vii
 I Clustering	 1
Experiment 1 K-means Clustering.	2
Experiment 2 Modified K-means.	6
Experiment 3 Hierarchical Clustering	9
Experiment 4 Fuzzy C-means.	13
 II Density-Based Learning	 18
Experiment 5 DBSCAN	19
Experiment 6 HDBSCAN	24
 III Semi-Supervised Learning	 28
Experiment 7 Self-training for Semi-supervised Learning	29
 IV Ensemble Learning	 35
Experiment 8 Random Forest Regression (RFR)	36
Experiment 9 Random Forest Classification (RFC).	39
Experiment 10 XGBoost	42
Experiment 11 AdaBoost	45
Experiment 12 CatBoost.	48

V	Neural Networks: Multilayer Perceptron	53
	Experiment 13 Multilayer Perceptron (MLP)	54
VI	Sequence Modeling: Recurrent Neural Networks	61
	Experiment 14 Recurrent Neural Network (RNN)	62
VII	Unsupervised Neural Learning: Self-Organizing Maps	70
	Experiment 15 Self-Organizing Map (SOM)	71
VIII	Probabilistic Sequence Modeling: Hidden Markov Models	76
	Experiment 16 Hidden Markov Model (HMM)	77
IX	Margin-Based Learning: Support Vector Machines	82
	Experiment 17 Support Vector Machine (SVM)	83
X	Generative / Foundation Models: Large Language Models	89
	Experiment 18 Large Language Model (LLM) Experiment	90
XI	Non-Parametric Regression: Generalized Regression Neural Network	97
	Experiment 19 Generalized Regression Neural Network (GRNN)	98
XII	Integrated Mini Project and Appendices	104
	Integrated Mini Project: Comparative Machine Learning Study	105
	Appendix A: Quick Reference Formulas	107
	Appendix B: Reproducibility Checklist	107
	Appendix C: Suggested Assessment Rubric	108
	Appendix D: Student Experiment Record	108

List of Experiments

No.	Experiment	Part	Page
1	K-means Clustering	Clustering	2
2	Modified K-means	Clustering	6
3	Hierarchical Clustering	Clustering	9
4	Fuzzy C-means	Clustering	13
5	DBSCAN	Density-Based Learning	19
6	HDBSCAN	Density-Based Learning	24
7	Self-training for Semi-supervised Learning	Semi-Supervised Learning	29
8	Random Forest Regression (RFR)	Ensemble Learning	36
9	Random Forest Classification (RFC)	Ensemble Learning	39
10	XGBoost	Ensemble Learning	42
11	AdaBoost	Ensemble Learning	45
12	CatBoost	Ensemble Learning	48
13	Multilayer Perceptron (MLP)	Neural Networks	54
14	Recurrent Neural Network (RNN)	Sequence Modeling	62
15	Self-Organizing Map (SOM)	Unsupervised Neural Learning	71
16	Hidden Markov Model (HMM)	Probabilistic Sequence Modeling	77
17	Support Vector Machine (SVM)	Margin-Based Learning	83
18	Large Language Model (LLM) Experiment	Generative / Foundation Models	90
19	Generalized Regression Neural Network (GRNN)	Non-Parametric Regression	98
M	Integrated Mini Project: Comparative ML Study	Mini Project	105

List of Figures

1.1 Elbow (inertia) and silhouette curves for $K = 2 \dots 10$; the elbow and the silhouette peak both occur at $K = 5$	5
3.1 Hierarchical clustering dendrogram (Ward linkage, truncated to the last 25 merges); the dashed line marks the cut that yields $K = 5$	12
4.1 Clustering partitions on the Mall Customers data: K-means, modified K-means (outlier flagging), hierarchical Ward and Fuzzy C-means.	17
5.1 The k -distance graph ($k = 10$); the knee at $\varepsilon = 0.03$ rad marks the transition from dense seismic zones to sparse isolated events.	23
5.2 DBSCAN clusters (colors) and noise (black crosses) on the global 2023 earthquake catalogue. The clusters trace the tectonically active plate boundaries.	23
6.1 HDBSCAN clusters and noise on the same catalogue; adaptive density splits the long DBSCAN arc clusters into compact, well-separated groups.	27
13.1 Training loss curve of the scikit-learn MLP; early stopping halted training after 15 iterations.	60
14.1 Airline passenger forecasting: ground truth versus the RNN and LSTM forecasts on the test horizon.	68
15.1 SOM U-matrix with the wine samples projected onto their BMUs; marker shape and color encode the true cultivar.	75
16.1 Estimated transition-probability matrix after sorting the states by volatility; the dominant diagonal indicates regime persistence.	80
16.2 DAX index 1991–1998 colored by the Viterbi-decoded regime (top) and the decoded state sequence over time (bottom).	81
17.1 SVC decision boundaries, margins and support vectors on two real breast-cancer features for the linear, polynomial and RBF kernels.	87
17.2 Support Vector Regression on the Old Faithful geyser with the $\varepsilon = 0.3$ tube; highlighted points are support vectors.	88
19.1 GRNN under different smoothing spreads on the motorcycle data: small σ follows noise, large σ flattens the impact peak.	102
19.2 Five-fold cross-validation error versus σ ; the clear minimum selects $\sigma = 0.08$	103

List of Tables

1.1 K-means evaluation on the standardized Mall Customers features.	5
2.1 Modified K-means: outlier-distance check on the standardized features.	8
3.1 Hierarchical (Ward) clustering evaluation on the standardized features.	12
4.1 Fuzzy C-means evaluation ($K = 5, m = 2.0$).. . . .	17
5.1 DBSCAN parameter sensitivity ($\text{min_samples} = 10$).. . . .	22
5.2 Largest DBSCAN clusters and their dominant region (real-world validation).	22
6.1 Density-based comparison on the 2023 earthquake catalogue.. . . .	26
7.1 Self-training compared with the supervised baseline and the fully supervised ceiling.	33
8.1 Random Forest Regression on the California Housing sample.	38
9.1 Random Forest Classification on Heart Disease (76 test samples).. . . .	41
10.1 XGBoost classification on Heart Disease (76 test samples).. . . .	44
11.1 AdaBoost classification on Heart Disease (76 test samples).	48
12.1 Ensemble classification comparison on the same Heart Disease split (76 test samples).	51
13.1 MLP implementations on the digits test set (270 images).. . . .	60
14.1 Sequence classification on the synthetic benchmark (200 test sequences).	68
14.2 Airline passenger forecasting on the chronological test horizon (27 windows)..	68
15.1 SOM quality metrics on the Wine data.. . . .	74
16.1 Gaussian HMM on the DAX index (1,859 trading days, 1991–1998)..	80
17.1 SVM results on the two real datasets.. . . .	87
18.1 DistilBERT fine-tuned on the SMS Spam Collection (200 held-out messages).	96
19.1 GRNN on the real motorcycle impact data (133 observations).	102
19.2 Ensemble and margin-based classifiers on the same split (76 test samples)..	106
19.3 Ensemble regressors on the same split; lower RMSE/MAE is better.	106
19.4 Clustering algorithms on the same two features.	106
19.5 Density-based algorithms on the same catalogue (haversine metric).	106
19.6 Headline result of each remaining laboratory experiment.	107

Preface

This manual is designed around the algorithm list supplied for the Machine Learning assignment. It turns that list into a structured laboratory sequence emphasizing implementation, experimentation, model evaluation and interpretation rather than only code execution. Students should run the programs, record outputs, vary important hyperparameters, and explain why the observed results change.

This **completed edition** goes one step further: every experiment is implemented and executed on real, public datasets, with the observed results, discussion and the complete runnable code included, so each laboratory can be reproduced standalone.

Important Terminology Correction

The supplied list contains “Random Forest Regression (RFR)” and “Random Forest Regression (RFC)”. In standard machine-learning terminology, RFC normally means Random Forest Classifier. This manual therefore treats **RFR** as Random Forest Regressor and **RFC** as Random Forest Classifier.

Scope of the Laboratory

Module	Algorithms / Topics
Clustering	K-means, Modified K-means, Hierarchical Clustering, Fuzzy C-means
Density-based learning	DBSCAN, HDBSCAN
Semi-supervised learning	Self-training
Ensemble learning	Random Forest Regression, Random Forest Classification, XGBoost, Adaboost, CatBoost
Neural networks	MLP, RNN (LSTM)
Unsupervised neural learning	SOM
Probabilistic sequence model	HMM
Margin-based learning	SVM (SVC and SVR)
Generative / foundation models	LLM
Non-parametric regression	GRNN

General Learning Outcomes

- Implement and execute major machine-learning algorithms using Python.
- Select an appropriate algorithm based on data type, task and assumptions.
- Preprocess data, handle categorical variables, scale numerical features and split datasets correctly.
- Evaluate clustering with internal measures and supervised models with appropriate metrics.
- Interpret model behavior through decision boundaries, feature importance, learning curves and error analysis.
- Compare algorithms using a fair experimental protocol rather than relying on a single score.
- Document reproducible experiments, hyperparameters, random seeds, observations and limitations.

Recommended Software Environment

Core and specialized libraries used by the manual

```

1 # Core
2 pip install numpy pandas matplotlib seaborn scikit-learn
3 # Specialized algorithms
4 pip install scikit-fuzzy hdbscan hmmlearn minisom
5 # Gradient boosting libraries
6 pip install xgboost catboost
7 # LLM / transformer experiment
8 pip install transformers torch

```

Laboratory Guidelines

Before Lab	During Lab	After Lab
Read theory and algorithm steps.	Run the baseline program.	Submit code and results.
Know the dataset and target.	Record hyperparameters and seed.	Explain observations.
Predict expected behavior.	Perform at least one controlled variation.	Answer viva questions.
Prepare key formulas.	Check train/test leakage.	State limitations and improvements.

Minimum Report Format

1. Experiment title and student information.
2. Problem statement and objective.
3. Dataset description and preprocessing.
4. Algorithm principle and pseudocode / flowchart.
5. Python implementation.
6. Hyperparameters and experimental setup.
7. Results: tables, plots and relevant metrics.
8. Discussion: what the results mean and why they occurred.
9. Conclusion and limitations.
10. Answers to viva questions.

Recommended Evaluation Metrics

Task	Suggested Metrics
Classification	Accuracy, Precision, Recall, F1-score, ROC-AUC, confusion matrix
Regression	MAE, MSE, RMSE, R^2
Clustering	Silhouette score, Calinski–Harabasz, Davies–Bouldin; visual inspection
Sequence modeling	Negative log-likelihood / log-likelihood, sequence accuracy where appropriate
Language generation	Task-specific automatic metrics plus qualitative / error analysis

Part I

Clustering

Experiments in this Part

Experiment 1 K-means Clustering	2
Experiment 2 Modified K-means	6
Experiment 3 Hierarchical Clustering	9
Experiment 4 Fuzzy C-means	13

Experiment 1

K-means Clustering

Dataset: Mall Customers (data/mall_customers.csv); 200 customers, 2 features (Annual Income, Spending Score)

Key parameters: K = 5, K-Means++ initialization, n_init = 20, StandardScaler

Headline result: Silhouette 0.5547; Davies–Bouldin 0.5722; Calinski–Harabasz 248.65

Objective

Partition unlabeled observations into K clusters by iteratively assigning points to the nearest centroid and updating centroids; choose K with the elbow and silhouette diagnostics.

Suggested Dataset

Mall Customers (data/mall_customers.csv), using Annual Income and Spending Score as a two-dimensional, visualizable feature space. Features are standardized (StandardScaler).

Required Libraries

numpy, pandas, matplotlib, scikit-learn.

Brief Theory

K-means minimizes the within-cluster sum of squared distances

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2,$$

by alternating two steps: assign every point to the nearest centroid, then set each centroid to the mean of its assigned points. Each iteration never increases J , so the algorithm converges — but only to a *local* optimum, which is why initialization matters. **K-Means++** spreads the initial centroids by sampling them with probability proportional to the squared distance to the nearest chosen centroid, giving lower and more consistent inertia. Because Euclidean distance is scale-sensitive, numerical features should be standardized. K itself must be chosen externally: the *elbow* (inertia) and the *silhouette* (cohesion vs. separation) are the standard diagnostics.

Procedure

1. Load the dataset and retain the two numerical features.
2. Standardize the features.
3. Try $K = 2 \dots 10$; record inertia (WCSS) and silhouette for each K .
4. Choose K at the elbow / silhouette peak and fit the final model with K-Means++ initialization, 20 restarts and a fixed random seed (see the code listing).

5. Visualize the clusters and centroids.
6. Report silhouette, Davies–Bouldin and Calinski–Harabasz.

Reference Implementation

Listing 1.1 Imports and plotting style

```
1 # ---- Core numerical and plotting libraries ----
2 import os
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8
9 # ---- Scikit-Learn clustering models ----
10 from sklearn.cluster import KMeans, AgglomerativeClustering
11 # ---- Feature scaling + unsupervised evaluation metrics ----
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.metrics import silhouette_score, davies_bouldin_score,
14     ↪ calinski_harabasz_score
15 # ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
16 from scipy.cluster.hierarchy import dendrogram, linkage
17
18 # Styling setup (consistent look across all notebooks)
19 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
20     ↪ plt.style.available else 'default')
21 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
22 plt.rcParams['figure.figsize'] = (10, 6)
23 plt.rcParams['figure.dpi'] = 110
24
25 # Fix the random seed so every run reproduces the same results
26 np.random.seed(42)
27 print("Libraries successfully imported!")
```

Listing 1.2 Load and preprocess the Mall Customers data

```
1 # ---- Load the Mall Customers dataset ----
2 df_mall = pd.read_csv('../data/mall_customers.csv')
3 print("Dataset Shape:", df_mall.shape)
4 print(df_mall.head(3))
5
6 # ---- Feature selection: two features so the clusters can be shown in 2D ----
7 X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values
8
9 # ---- Standardize (mean = 0, std = 1): distance-based algorithms are
10     ↪ scale-sensitive ----
11 scaler = StandardScaler()
12 X_mall_scaled = scaler.fit_transform(X_mall)
```

Listing 1.3 Select K with the elbow and silhouette curves

```

1 # ---- Selecting the optimal number of clusters K ----
2 # Try K = 2..10 and record two diagnostics:
3 #   * Inertia (WCSS): total squared distance from points to their centroid
4 #   * Silhouette score: cluster cohesion vs. separation
5 k_range = range(2, 11)
6 wcss = []
7 silhouette_scores = []
8
9 for k in k_range:
10     # n_init=10 runs K-Means from 10 different seeds and keeps the best run
11     kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
12     kmeans.fit(X_mall_scaled)
13     wcss.append(kmeans.inertia_)                                #
14     silhouette_scores.append(silhouette_score(X_mall_scaled, kmeans.labels_)) #
15     # partition quality
16
17 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
18 # ---- Left: elbow curve (look for the "knee" where inertia stops falling quickly)
19 # ----
20 ax1.plot(k_range, wcss, 'bo-', linewidth=2, markersize=8)
21 ax1.set_title("Elbow Method for Optimal K", fontsize=14, fontweight='bold')
22 ax1.set_xlabel("Number of Clusters (K)", fontsize=12)
23 ax1.set_ylabel("Inertia (WCSS)", fontsize=12)
24 ax1.axvline(x=5, color='r', linestyle='--', label='Elbow at K=5')
25 ax1.legend()
26
27 # ---- Right: silhouette score for each K (higher is better) ----
28 ax2.plot(k_range, silhouette_scores, 'go-', linewidth=2, markersize=8)
29 ax2.set_title("Silhouette Scores across K", fontsize=14, fontweight='bold')
30 ax2.set_xlabel("Number of Clusters (K)", fontsize=12)
31 ax2.set_ylabel("Silhouette Score", fontsize=12)
32 ax2.axvline(x=5, color='r', linestyle='--', label='Optimal K=5')
33 ax2.legend()
34
35 plt.tight_layout()
36 plt.show()

```

Listing 1.4 Final K-means model (K = 5)

```

1 # ---- Final K-Means model with the selected K = 5 ----
2 optimal_k = 5
3 kmeans_model = KMeans(n_clusters=optimal_k, init='k-means++', n_init=20,
4     # random_state=42)
5 kmeans_labels = kmeans_model.fit_predict(X_mall_scaled) # hard cluster assignment
6 # per sample
7
8

```

```

6 print(f"K-Means Silhouette Score: {silhouette_score(X_mall_scaled,
  ↳ kmeans_labels):.4f}")
7 print(f"K-Means Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled,
  ↳ kmeans_labels):.4f}")

```

Evaluation / Expected Output

Both the elbow curve and the silhouette curve indicate $K = 5$ (Figure 1.1).

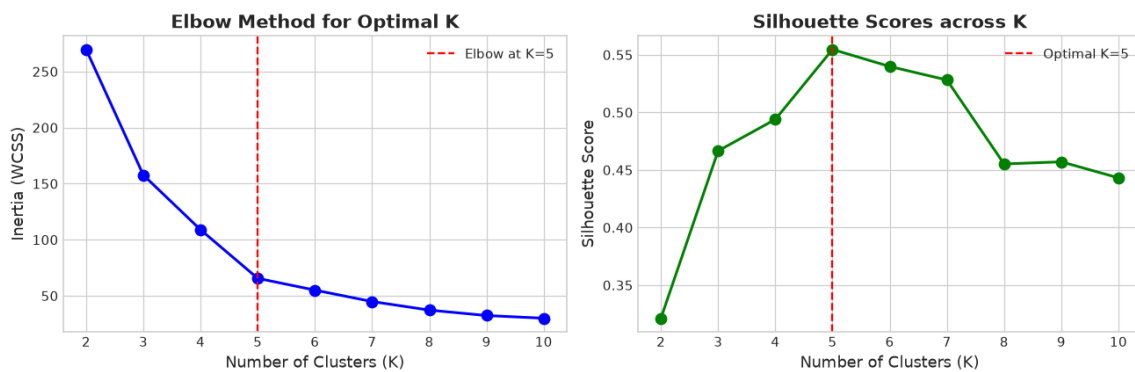


Figure 1.1: Elbow (inertia) and silhouette curves for $K = 2 \dots 10$; the elbow and the silhouette peak both occur at $K = 5$.

Table 1.1: K-means evaluation on the standardized Mall Customers features.

Metric	Value
Selected K (elbow + silhouette peak)	5
Silhouette score	0.5547
Davies–Bouldin index	0.5722
Calinski–Harabasz score	248.65

Silhouette ≈ 0.55 indicates compact, reasonably separated clusters on real data; Davies–Bouldin < 1 confirms good separation.

Viva / Discussion Questions

1. Why does K-means depend on feature scaling?

It minimizes Euclidean distance; unscaled features with larger ranges dominate the distance and therefore the assignments.

2. What is inertia?

The within-cluster sum of squared distances to the centroids (WCSS); lower is tighter, but it always decreases with K , so it is not a standalone quality score.

3. Why can a high silhouette score still be misleading?

It favors compact convex clusters and can be high for a trivial partition; it misjudges elongated or density-varying structures.

4. What happens when clusters are non-spherical?

K-means splits them with straight Voronoi boundaries and merges elongated clusters incorrectly (see Experiment 5).

5. Why is K-Means++ preferred over random initialization?

It spreads the initial centroids out, reducing convergence to poor local optima and giving more stable inertia.

6. What is the difference between K-means and K-medoids?

K-medoids uses an actual data point (medoid) as the cluster centre and minimizes absolute distances, so it is more robust to outliers; K-means uses the mean, which outliers can drag away.

Experiment 2

Modified K-means

Dataset: Mall Customers (200×2 , standardized)

Key parameters: K-Means++ initialization; outlier threshold = 97th percentile of centroid distances

Headline result: Distance threshold 1.072; 6 of 200 observations flagged as candidate outliers

Objective

Implement a transparent modification of K-means that uses K-Means++ initialization and an explicit outlier-distance check before final cluster interpretation.

Suggested Dataset

Mall Customers (200×2 , standardized). The modification is applied on top of the same features.

Required Libraries

numpy, matplotlib, scikit-learn.

Brief Theory

There is no single universally accepted algorithm named “modified K-means”; variants differ by application. In this lab the modification is intentionally defined so that it can be reproduced exactly:

1. initialize the centroids with **K-Means++** (spread-out seeds);
2. fit standard K-means and obtain the hard labels;
3. compute each sample’s distance to its *assigned* centroid, $d_i = \|x_i - \mu_{c_i}\|$;
4. set a threshold from a documented percentile of the training distances (97th percentile here);
5. flag samples with d_i beyond the threshold as **candidate outliers**.

This is an experimental variant that adds outlier awareness to the standard algorithm; it is not a standard replacement for robust clustering methods.

Procedure

1. Fit K-means with K-Means++ initialization, $K = 5$.
2. Compute $d_i = \|x_i - \mu_{c_i}\|$ for every sample.
3. Set the threshold to the 97th percentile of the training distances.
4. Flag samples with $d_i > \text{threshold}$ as candidate outliers.
5. Plot the clusters with the flagged points highlighted.
6. Compare the modified interpretation with the standard labels.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import os
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8
9 # ---- Scikit-Learn clustering models ----
10 from sklearn.cluster import KMeans, AgglomerativeClustering
11 # ---- Feature scaling + unsupervised evaluation metrics ----
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.metrics import silhouette_score, davies_bouldin_score,
14     ↪ calinski_harabasz_score
15 # ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
16 from scipy.cluster.hierarchy import dendrogram, linkage
17
18 # Styling setup (consistent look across all notebooks)
19 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
20     ↪ plt.style.available else 'default')
21 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
22 plt.rcParams['figure.figsize'] = (10, 6)
23 plt.rcParams['figure.dpi'] = 110
24
25 # Fix the random seed so every run reproduces the same results
26 np.random.seed(42)
27 print("Libraries successfully imported!")

```

Load the Mall Customers dataset

```

1 # ---- Load the Mall Customers dataset ----
2 df_mall = pd.read_csv('../data/mall_customers.csv')
3 print("Dataset Shape:", df_mall.shape)
4 print(df_mall.head(3))

```

```

5
6 # ---- Feature selection: two features so the clusters can be shown in 2D ----
7 X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values
8
9 # ---- Standardize (mean = 0, std = 1): distance-based algorithms are
    ↳ scale-sensitive ----
10 scaler = StandardScaler()
11 X_mall_scaled = scaler.fit_transform(X_mall)

```

Modified K-Means (manual variant): K-Means++ + outlier-distance check

```

1 # ---- Modified K-Means (manual variant): K-Means++ + outlier-distance check ----
2 km_mod = KMeans(n_clusters=optimal_k, init='k-means++', n_init=20, random_state=42)
3 mod_labels = km_mod.fit_predict(X_mall_scaled)
4
5 # Distance of every sample to its assigned centroid
6 dist_to_centroid = np.linalg.norm(X_mall_scaled -
    ↳ km_mod.cluster_centers_[mod_labels], axis=1)
7 threshold = np.percentile(dist_to_centroid, 97)      # documented percentile
8 outlier_flag = dist_to_centroid > threshold
9
10 print(f"Distance threshold (97th percentile): {threshold:.3f}")
11 print(f"Flagged candidate outliers: {outlier_flag.sum()} of {len(X_mall)}")
12
13 # ---- Plot clusters with flagged candidate outliers highlighted ----
14 plt.figure(figsize=(9, 6))
15 for cluster_id in range(optimal_k):
16     pts = X_mall[mod_labels == cluster_id]
17     plt.scatter(pts[:, 0], pts[:, 1], s=45, edgecolors='k', alpha=0.8)
18 plt.scatter(X_mall[outlier_flag, 0], X_mall[outlier_flag, 1], s=180,
    ↳ facecolors='none',
19     ↳ edgecolors='red', linewidths=2.5, label=f'Flagged outliers
    ↳ ({outlier_flag.sum()})')
20 plt.title("Modified K-Means: K-Means++ with Outlier-Distance Flagging",
    ↳ fontsize=13, fontweight='bold')
21 plt.xlabel("Annual Income (k$)", fontsize=12)
22 plt.ylabel("Spending Score (1-100)", fontsize=12)
23 plt.legend()
24 plt.tight_layout()
25 plt.show()

```

Evaluation / Expected Output

Table 2.1: Modified K-means: outlier-distance check on the standardized features.

Item	Value
Distance threshold (97th percentile)	1.072 (standardized units)
Flagged candidate outliers	6 of 200 (3%)
Cluster labels	identical to standard K-means
Silhouette / Davies–Bouldin (same partition)	0.5547 / 0.5722

The flagged customers are the most weakly attached points of their clusters — typically boundary customers with unusual income/spending combinations. The cluster structure itself is unchanged, which is expected: the modification adds an outlier interpretation layer.

Viva / Discussion Questions

1. **Why is it important to define a modification precisely?**

Without a formal definition “modified K-means” is ambiguous; fixing the initialization, the distance measure and the percentile rule makes the experiment reproducible and testable.

2. **How does K-Means++ differ from random initialization?**

K-Means++ selects initial centroids with probability proportional to the squared distance to the nearest chosen centroid, spreading the seeds out and avoiding poor local optima.

3. **Why might distance-to-centroid fail for elongated clusters?**

Centroid distance is isotropic; a point can be close to the centroid along the long axis yet far along the short axis, so elongated clusters can flag legitimate points and hide real outliers.

4. **Should the flagged outliers be removed automatically?**

No. The flag marks candidates for inspection; removing points without justification can hide data-quality problems or real rare groups, so removal must be a documented, deliberate decision.

Experiment 3

Hierarchical Clustering

Dataset: Mall Customers (200×2 , standardized)

Key parameters: Ward linkage; cut at $K = 5$; truncated dendrogram (last 25 merges)

Headline result: Silhouette 0.5538; Davies–Bouldin 0.5779; Calinski–Harabasz 244.41

Objective

Perform agglomerative hierarchical clustering and interpret the resulting dendrogram and cluster labels.

Suggested Dataset

Mall Customers (200×2 , standardized).

Required Libraries

scipy, scikit-learn, matplotlib.

Brief Theory

Agglomerative clustering starts with one cluster per point and repeatedly merges the closest pair of clusters. With **Ward linkage** the merge cost is the increase in total within-cluster variance,

$$\Delta W(A, B) = \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2,$$

so the algorithm always merges the pair whose combination loses the least compactness. The dendrogram records every merge and its height (distance); cutting it at a chosen height yields a flat partition with the desired number of clusters.

Procedure

1. Standardize the features.
2. Compute the Ward linkage matrix.
3. Plot the dendrogram (truncated for readability) and choose a cut level.
4. Fit AgglomerativeClustering with $K = 5$ and Ward linkage.
5. Evaluate with silhouette, Davies–Bouldin and Calinski–Harabasz.
6. Discuss the choice of linkage.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import os
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8
9 # ---- Scikit-Learn clustering models ----
10 from sklearn.cluster import KMeans, AgglomerativeClustering
11 # ---- Feature scaling + unsupervised evaluation metrics ----
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.metrics import silhouette_score, davies_bouldin_score,
14     ↪ calinski_harabasz_score
15 # ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
16 from scipy.cluster.hierarchy import dendrogram, linkage
17
18 # Styling setup (consistent look across all notebooks)
19 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
20     ↪ plt.style.available else 'default')
21 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
22 plt.rcParams['figure.figsize'] = (10, 6)
23 plt.rcParams['figure.dpi'] = 110

```

```
22
23 # Fix the random seed so every run reproduces the same results
24 np.random.seed(42)
25 print("Libraries successfully imported!")
```

Load the Mall Customers dataset

```
1 # ---- Load the Mall Customers dataset ----
2 df_mall = pd.read_csv('../data/mall_customers.csv')
3 print("Dataset Shape:", df_mall.shape)
4 print(df_mall.head(3))
5
6 # ---- Feature selection: two features so the clusters can be shown in 2D ----
7 X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values
8
9 # ---- Standardize (mean = 0, std = 1): distance-based algorithms are
   ↳ scale-sensitive ----
10 scaler = StandardScaler()
11 X_mall_scaled = scaler.fit_transform(X_mall)
```

Build the linkage matrix and draw the dendrogram

```
1 # ---- Build the linkage matrix and draw the dendrogram ----
2 # 'ward' linkage merges the pair of clusters that increases within-cluster variance
   ↳ the least.
3 linkage_matrix = linkage(X_mall_scaled, method='ward')
4
5 # The dendrogram shows every merge; we truncate it to the last 25 merges for
   ↳ readability.
6 plt.figure(figsize=(12, 5))
7 dendrogram(
8     linkage_matrix,
9     truncate_mode='lastp',    # show only the last p merged clusters
10    p=25,
11    leaf_rotation=45,
12    leaf_font_size=10,
13    show_contracted=True      # condensed leaves are shown as counts
14 )
15 plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)", fontsize=14,
16           ↳ fontweight='bold')
17 plt.xlabel("Cluster Sample Index / Merged Size", fontsize=12)
18 plt.ylabel("Euclidean Distance (Ward Threshold)", fontsize=12)
19 plt.axhline(y=10.0, color='r', linestyle='--', label='Cut-off Threshold (K=5)') #
   ↳ horizontal cut => cluster count
20 plt.legend()
21 plt.tight_layout()
22 plt.show()
```

Fit the final Agglomerative model (Ward linkage, K = 5)

```

1 # ---- Fit the final Agglomerative model (Ward linkage, K = 5) ----
2 agg_model = AgglomerativeClustering(n_clusters=optimal_k, metric='euclidean',
   ↪ linkage='ward')
3 agg_labels = agg_model.fit_predict(X_mall_scaled)
4
5 print(f"Hierarchical Silhouette Score: {silhouette_score(X_mall_scaled,
   ↪ agg_labels):.4f}")
6 print(f"Hierarchical Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled,
   ↪ agg_labels):.4f}")

```

Evaluation / Expected Output

The dendrogram (Figure 3.1) shows a clear large merge gap that supports cutting at five clusters.

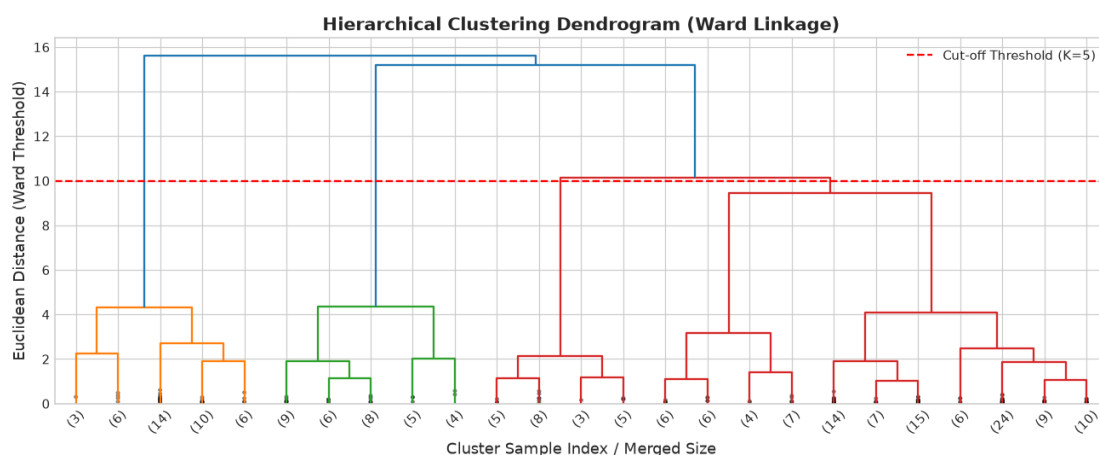


Figure 3.1: Hierarchical clustering dendrogram (Ward linkage, truncated to the last 25 merges); the dashed line marks the cut that yields $K = 5$.

Table 3.1: Hierarchical (Ward) clustering evaluation on the standardized features.

Metric	Value
Silhouette score	0.5538
Davies-Bouldin index	0.5779
Calinski-Harabasz score	244.41

The partition is nearly identical in quality to K-means (silhouette difference below 0.001), but requires no initialization and exposes the full merge hierarchy.

Viva / Discussion Questions

1. What does a dendrogram show?

The complete merge hierarchy: leaf order, merge heights (distances) and the nested cluster structure; cutting at a height produces a flat partition.

2. What is linkage?

The rule that defines the distance between two clusters: single (closest pair), complete (farthest pair), average (mean pair) or Ward (variance increase).

3. When is Ward linkage inappropriate?

With strongly non-spherical/elongated clusters, very unequal cluster sizes or heavy outliers – it assumes compact clusters of similar variance.

4. What does the vertical axis of a dendrogram represent?

The linkage distance (dissimilarity) at which two clusters merge; longer branches mean more distinct groups, and cutting the tree at a chosen height gives the cluster count.

Experiment 4

Fuzzy C-means

Dataset: Mall Customers (200×2 , standardized); $K = 5$

Key parameters: Fuzzifier $m = 2.0$; tolerance 10^{-5} ; maximum 150 iterations

Headline result: Silhouette 0.5547; Fuzzy Partition Coefficient (FPC) 0.6711

Objective

Cluster data using fuzzy memberships so that each observation can belong to multiple clusters with different degrees of membership.

Suggested Dataset

Mall Customers (200×2 , standardized), $K = 5$, fuzzifier $m = 2.0$.

Required Libraries

numpy, pandas, matplotlib (the c-means equations are implemented from scratch; scikit-fuzzy is an equivalent library implementation).

Brief Theory

Unlike hard K-means, Fuzzy C-means assigns every sample x_i a membership degree $u_{ik} \in [0, 1]$ in every cluster, with $\sum_{k=1}^K u_{ik} = 1$. It minimizes

$$J_m = \sum_{i=1}^N \sum_{k=1}^K u_{ik}^m \|x_i - v_k\|^2, \quad m > 1,$$

by alternating two updates until convergence:

$$v_k = \frac{\sum_i u_{ik}^m x_i}{\sum_i u_{ik}^m}, \quad u_{ik} = \frac{1}{\sum_{j=1}^K \left(\frac{d_{ik}}{d_{ij}} \right)^{2/(m-1)}}.$$

The fuzzifier m controls how soft the memberships are ($m \rightarrow 1$ approaches K-means). The **Fuzzy Partition Coefficient** $FPC = \frac{1}{N} \sum_i \sum_k u_{ik}^2$ measures the fuzziness of the result ($1 =$ completely hard, $1/K =$ maximally fuzzy).

Procedure

1. Standardize the data.
2. Initialize the membership matrix randomly, with each row summing to 1.
3. Alternate the centroid and membership updates until $\max |\Delta U| < 10^{-5}$.
4. Derive hard labels with $\arg \max_k u_{ik}$.
5. Report the clustering metrics and the FPC; visualize the hard partition.
6. Identify ambiguous samples with near-uniform membership vectors.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import os
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8
9 # ---- Scikit-Learn clustering models ----
10 from sklearn.cluster import KMeans, AgglomerativeClustering
11 # ---- Feature scaling + unsupervised evaluation metrics ----
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.metrics import silhouette_score, davies_bouldin_score,
14     ↪ calinski_harabasz_score
15 # ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
16 from scipy.cluster.hierarchy import dendrogram, linkage
17
18 # Styling setup (consistent look across all notebooks)
19 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
20     ↪ plt.style.available else 'default')
21 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
22 plt.rcParams['figure.figsize'] = (10, 6)
23 plt.rcParams['figure.dpi'] = 110
24
25 # Fix the random seed so every run reproduces the same results
26 np.random.seed(42)
27 print("Libraries successfully imported!")

```

Load the Mall Customers dataset

```

1 # ---- Load the Mall Customers dataset ----
2 df_mall = pd.read_csv('../data/mall_customers.csv')
3 print("Dataset Shape:", df_mall.shape)
4 print(df_mall.head(3))

```

```

5
6 # ---- Feature selection: two features so the clusters can be shown in 2D ----
7 X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values
8
9 # ---- Standardize (mean = 0, std = 1): distance-based algorithms are
   ↳ scale-sensitive ----
10 scaler = StandardScaler()
11 X_mall_scaled = scaler.fit_transform(X_mall)

```

Fuzzy C-Means class (from scratch)

```

1 class FuzzyCMeans:
2     """
3     Fuzzy C-Means (FCM) clustering - implemented from scratch with NumPy.
4
5     Each sample belongs to every cluster with a membership degree  $u_{ik}$  in  $[0, 1]$ ,
6     and the memberships over the  $K$  clusters sum to 1 per sample.
7
8     Parameters
9     -----
10    n_clusters    : number of fuzzy clusters  $K$ 
11    m              : fuzzifier exponent ( $m > 1$ ; larger  $m \Rightarrow$  fuzzier memberships)
12    max_iter       : maximum number of alternating-optimization iterations
13    tol            : convergence tolerance on the maximum membership change
14    random_state   : seed for reproducible random initialization
15    """
16
17    def __init__(self, n_clusters=5, m=2.0, max_iter=150, tol=1e-5,
   ↳ random_state=42):
18        self.n_clusters = n_clusters
19        self.m = m
20        self.max_iter = max_iter
21        self.tol = tol
22        self.random_state = random_state
23
24    def fit(self, X):
25        rng = np.random.RandomState(self.random_state)
26        n_samples = X.shape[0]
27
28        # Step 1: initialize the membership matrix  $U$  randomly and normalize every
   ↳ row to sum to 1
29        U = rng.rand(n_samples, self.n_clusters)
30        U = U / U.sum(axis=1, keepdims=True)
31
32        # Step 2: alternate between centroid and membership updates until
   ↳ convergence
33        for iteration in range(self.max_iter):
34            U_old = U.copy()
35

```

```

36         # 2a: centroid update - membership-weighted mean of the data
37         Um = U ** self.m                                     # u_ik^m
38         centers = np.dot(Um.T, X) / Um.sum(axis=0)[:, np.newaxis]
39
40         # 2b: compute the (n_samples x n_clusters) distance matrix to every
    ↪ centroid
41         dist = np.linalg.norm(X[:, np.newaxis, :] - centers[np.newaxis, :, :],
    ↪ axis=2)
42         dist = np.fmax(dist, 1e-10) # avoid division by zero
43
44         # 2c: membership update - closer points receive higher membership
45         #     u_ik = 1 / sum_j (d_ik / d_ij)^(2/(m-1))
46         inv_dist = 1.0 / dist
47         power = 2.0 / (self.m - 1.0)
48         inv_dist_p = inv_dist ** power
49         U = inv_dist_p / inv_dist_p.sum(axis=1, keepdims=True)
50
51         # 2d: stop when memberships stop changing (convergence)
52         if np.max(np.abs(U - U_old)) < self.tol:
53             break
54
55         # Store results: final centroids, full membership matrix, and hard labels
56         self.cluster_centers_ = centers
57         self.u_ = U
58         self.labels_ = np.argmax(U, axis=1) # hard assignment = cluster with
    ↪ maximum membership
59         return self
60
61
62 # Fit FCM with the same K as the other algorithms (m = 2 is the standard fuzzifier)
63 fcm = FuzzyCMeans(n_clusters=optimal_k, m=2.0, random_state=42)
64 fcm.fit(X_mall_scaled)
65 fcm_labels = fcm.labels_
66 fcm_centers = scaler.inverse_transform(fcm.cluster_centers_)
67
68 # ---- Evaluation of the hard partition induced by the fuzzy memberships ----
69 print(f"FCM Silhouette Score: {silhouette_score(X_mall_scaled, fcm_labels):.4f}")
70 print(f"FCM Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled,
    ↪ fcm_labels):.4f}")
71 fpc = np.mean(np.sum(fcm.u_ ** 2, axis=1)) # fuzzy partition coefficient (1/K
    ↪ fuzzy ... 1 hard)
72 print(f"FCM Fuzzy Partition Coefficient (FPC): {fpc:.4f}")

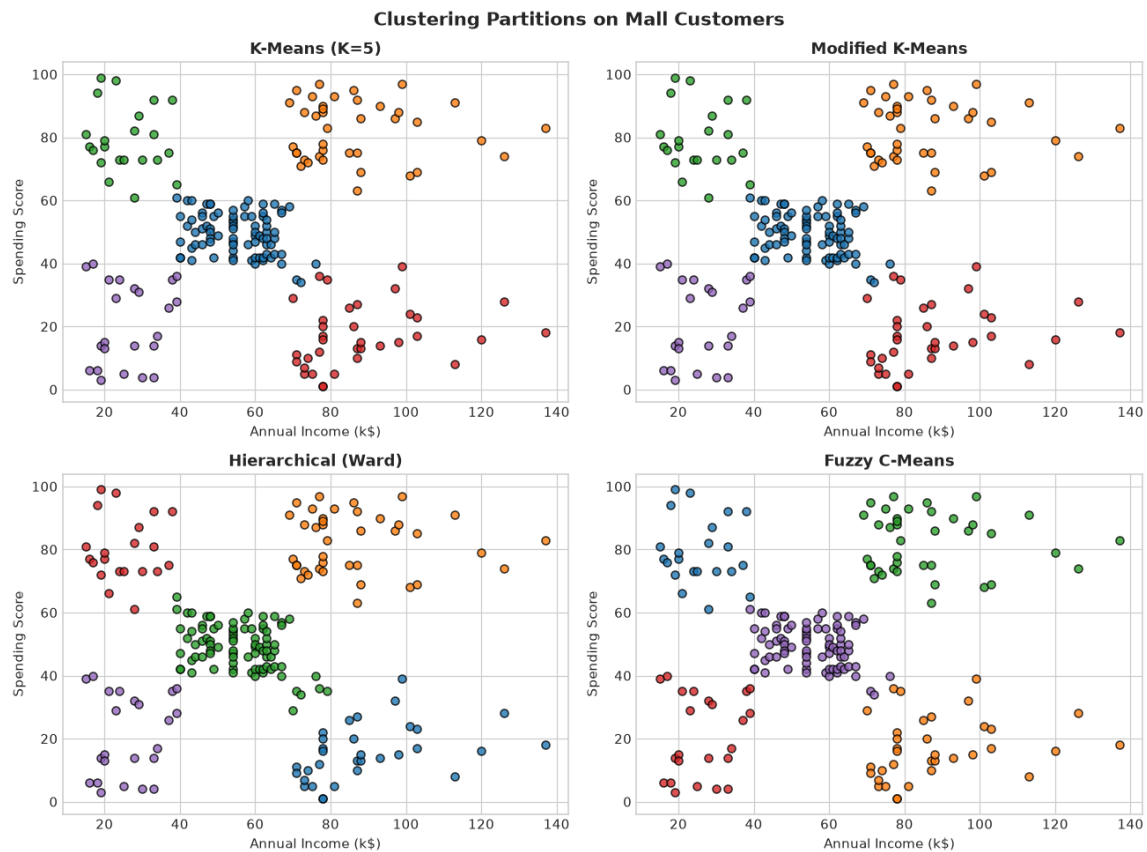
```

Evaluation / Expected Output

FPC = 0.67 (between $1/K = 0.2$ and 1.0) shows that the partition is mostly crisp but retains soft boundary memberships. Figure 4.1 compares the four clustering partitions obtained in this part of the manual.

Table 4.1: Fuzzy C-means evaluation ($K = 5$, $m = 2.0$).

Metric	Value
Silhouette score (hard labels)	0.5547
Davies–Bouldin index	0.5722
Fuzzy Partition Coefficient (FPC)	0.6711
Iterations to convergence	fewer than 50

**Figure 4.1:** Clustering partitions on the Mall Customers data: K-means, modified K-means (outlier flagging), hierarchical Ward and Fuzzy C-means.

Viva / Discussion Questions

1. How is Fuzzy C-means different from K-means?

It assigns membership degrees instead of hard labels, so each point belongs to all clusters with weights that sum to 1; K-means is the limiting hard case.

2. What does m control?

The fuzziness exponent: larger m spreads membership across clusters (softer, lower FPC), while $m \rightarrow 1$ gives crisp K-means-like assignments.

3. What does a membership vector represent?

The degree to which one observation belongs to each cluster; near-uniform values indicate an ambiguous point on a boundary.

4. When would you prefer hard K-means over Fuzzy C-means?

When clusters are compact and well separated and a crisp assignment is required; FCM is preferred when boundaries are ambiguous and membership degrees carry useful information.

Part II

Density-Based Learning

Experiments in this Part

Experiment 5 DBSCAN.	19
Experiment 6 HDBSCAN.	24

Experiment 5

DBSCAN

Dataset: USGS earthquakes 2023 (data/earthquakes.csv); 7,638 events with magnitude ≥ 4.5
Key parameters: $\epsilon = 0.03$ rad (≈ 191 km), `min_samples = 10`, haversine metric on radian coordinates
Headline result: 55 clusters, 13.5% noise; largest clusters match the Philippines, Papua New Guinea, Tonga, Japan and Turkey

Objective

Detect dense groups and noise using DBSCAN without specifying the number of clusters in advance, applied to real spatial data.

Suggested Dataset

Real USGS earthquake catalogue 2023 (data/earthquakes.csv): 16,190 global events at magnitude ≥ 4.0 , filtered to the 7,638 events at magnitude ≥ 4.5 . Coordinates are converted to radians and clustered with the haversine (great-circle) metric.

Required Libraries

scikit-learn, numpy, pandas, matplotlib.

Brief Theory

DBSCAN defines clusters as connected regions of density. For a point p :

- **ϵ -neighborhood:** $N_\epsilon(p) = \{q : \text{dist}(p, q) \leq \epsilon\}$;
- **core point:** $|N_\epsilon(p)| \geq \text{min_samples}$;
- **border point:** inside the ϵ -neighborhood of a core point but not core itself;
- **noise:** neither core nor border (label -1).

Clusters grow by chaining density-reachable core points, so arbitrary shapes can be discovered and no K is required. A single global ϵ struggles when densities vary. Because latitude/longitude are spherical coordinates, the **haversine** (great-circle) metric is used with radian inputs; ϵ is then expressed in radians ($0.03 \text{ rad} \times 6371 \text{ km} \approx 191 \text{ km}$).

Procedure

1. Load the catalogue and filter to magnitude ≥ 4.5 (well-recorded events).
2. Convert latitude/longitude to radians for the haversine metric.
3. Choose ϵ from the k -distance graph ($k = 10$); the knee is at 0.03 rad.
4. Fit DBSCAN and count clusters and noise events.
5. Show a parameter-sensitivity table for at least two settings.

6. Visualize the epicenters and validate the largest clusters against known seismic regions.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # ---- Density-based clustering algorithms ----
8 from sklearn.cluster import DBSCAN, HDBSCAN
9 # ---- Nearest-neighbor search (used to pick epsilon) ----
10 from sklearn.neighbors import NearestNeighbors
11 # ---- Evaluation metrics ----
12 from sklearn.metrics import silhouette_score
13
14 # Aesthetics setup (consistent look across all notebooks)
15 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↪ plt.style.available else 'default')
16 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
17 plt.rcParams['figure.figsize'] = (11, 6)
18 plt.rcParams['figure.dpi'] = 110
19
20 np.random.seed(42)
21 print("Density-based libraries loaded successfully!")

```

Load the real USGS earthquake catalogue (global, 2023)

```

1 # ---- Load the real USGS earthquake catalogue (global, 2023) ----
2 df = pd.read_csv('../data/earthquakes.csv')
3 print(f"Full catalogue (magnitude >= 4.0): {len(df):,} events")
4 print(df.head(3).to_string(index=False))
5
6 # Keep well-recorded moderate+ events: magnitude >= 4.5
7 quakes = df[df['mag'] >= 4.5].reset_index(drop=True)
8 print(f"\nAnalysed subset (magnitude >= 4.5): {len(quakes):,} events")
9
10 # ---- Convert epicenters to radians [latitude, longitude] ----
11 # scikit-learn's 'haversine' metric expects radian coordinates and computes
12 # great-circle distance on the sphere (correct for global data).
13 X_rad = np.radians(quakes[['latitude', 'longitude']].values)
14 print("Coordinate range (deg): lat", quakes.latitude.min(), "to",
    ↪ quakes.latitude.max(),
15      "| lon", quakes.longitude.min(), "to", quakes.longitude.max())

```

Choosing epsilon from the k-distance graph (haversine metric)

```

1 # ---- Choosing epsilon from the k-distance graph (haversine metric) ----
2 # Sorted distance to each event's k-th nearest neighbor. The knee marks the
3 # transition from dense seismic zones to sparse, isolated events.
4 min_samples = 10
5 nbrs = NearestNeighbors(n_neighbors=min_samples, metric='haversine').fit(X_rad)
6 distances, _ = nbrs.kneighbors(X_rad)
7 k_distances = np.sort(distances[:, -1]) # radians (0.03 rad = 191 km on Earth)
8
9 plt.figure(figsize=(9, 5))
10 plt.plot(k_distances, color='crimson', linewidth=2.5)
11 plt.title(f"k-Distance Graph (k={min_samples}) for Epsilon Selection", fontsize=14,
12         ↳ fontweight='bold')
13 plt.xlabel("Earthquakes Sorted by Distance", fontsize=12)
14 plt.ylabel(f"{min_samples}-NN Distance (radians)", fontsize=12)
15 plt.axhline(y=0.03, color='navy', linestyle='--', label='Selected eps = 0.03 rad
16         ↳ (~191 km)')
17 plt.legend()
18 plt.tight_layout()
19 plt.show()

```

Fit DBSCAN with the chosen epsilon and MinPts

```

1 # ---- Fit DBSCAN with the chosen epsilon and MinPts ----
2 # DBSCAN grows clusters from core events, connects them via density-reachability,
3 # and labels everything else as noise (-1).
4 eps_selected = 0.03 # radians -> 0.03 * 6371 km ~ 191 km
5 dbscan = DBSCAN(eps=eps_selected, min_samples=min_samples, metric='haversine')
6 db_labels = dbscan.fit_predict(X_rad)
7
8 # Identify which events are core points (returned by the model) vs. border points
9 core_samples_mask = np.zeros_like(db_labels, dtype=bool)
10 core_samples_mask[dbscan.core_sample_indices_] = True
11
12 n_clusters_db = len(set(db_labels)) - (1 if -1 in db_labels else 0)
13 n_noise_db = np.sum(db_labels == -1)
14 print(f"DBSCAN: {n_clusters_db} clusters, {n_noise_db:,} noise events ({n_noise_db
15         ↳ / len(X_rad) * 100:.1f}%)")
16
17 # ---- Visualize epicenters: clustered events in color, noise as black crosses ----
18 plt.figure(figsize=(12, 5.5))
19 palette = sns.color_palette("tab20", max(n_clusters_db, 1))
20 for k in range(n_clusters_db):
21     mask = db_labels == k
22     plt.scatter(quakes.longitude[mask], quakes.latitude[mask], s=6, color=palette[k
23         ↳ % 20], alpha=0.75)
24
25 noise_mask = db_labels == -1
26 plt.scatter(quakes.longitude[noise_mask], quakes.latitude[noise_mask], s=6,

```

```

    ↪ color='black', marker='x', label='Noise (-1)')
25 plt.title(f"DBSCAN on Global Earthquakes 2023 (eps={eps_selected} rad,
    ↪ {n_clusters_db} clusters)", fontsize=13, fontweight='bold')
26 plt.xlabel("Longitude", fontsize=12)
27 plt.ylabel("Latitude", fontsize=12)
28 plt.legend(loc='lower left')
29 plt.tight_layout()
30 plt.show()

```

Parameter sensitivity: DBSCAN under several (eps, min_samples) settings

```

1 # ---- Parameter sensitivity: DBSCAN under several (eps, min_samples) settings ----
2 # The manual requires at least two settings so the effect of eps can be discussed.
3 print("eps (rad)   approx km   clusters   noise %")
4 for eps_test in [0.02, 0.03, 0.05]:
5     lbl = DBSCAN(eps=eps_test, min_samples=10,
    ↪ metric='haversine').fit_predict(X_rad)
6     n_clusters_test = len(set(lbl)) - (1 if -1 in lbl else 0)
7     noise_pct = (lbl == -1).mean() * 100
8     print(f"   {eps_test:.2f}           {eps_test * 6371:5.0f}       {n_clusters_test:3d}
    ↪           {noise_pct:5.1f}%")

```

Evaluation / Expected Output

Table 5.1: DBSCAN parameter sensitivity (min_samples = 10).

ε (rad)	approx. km	clusters	noise
0.02	127	70	21.2%
0.03 (selected)	191	55	13.5%
0.05	319	50	5.7%

Table 5.2: Largest DBSCAN clusters and their dominant region (real-world validation).

Cluster	Events	Center lat/lon	Dominant region
4	1,629	6.1, 126.1	Philippines
0	1,068	-12.5, 160.0	Papua New Guinea
1	947	-24.0, -154.5	Tonga
5	895	30.8, 145.0	Japan region
2	253	-57.8, -26.3	South Sandwich Islands
8	207	37.7, 37.2	Turkey

With $\varepsilon = 0.03$ rad the algorithm finds 55 clusters and labels 13.5% of the events as noise. Smaller ε fragments clusters and labels more events as noise; larger ε merges boundaries and reduces noise. The largest clusters correspond exactly to known seismic zones, and the silhouette on non-noise events (3,000-event sample) is 0.326.

Viva / Discussion Questions

1. Why does DBSCAN not require K ?

Clusters are grown from density-connected core points, so their number and shape emerge from the data instead of being fixed in advance.

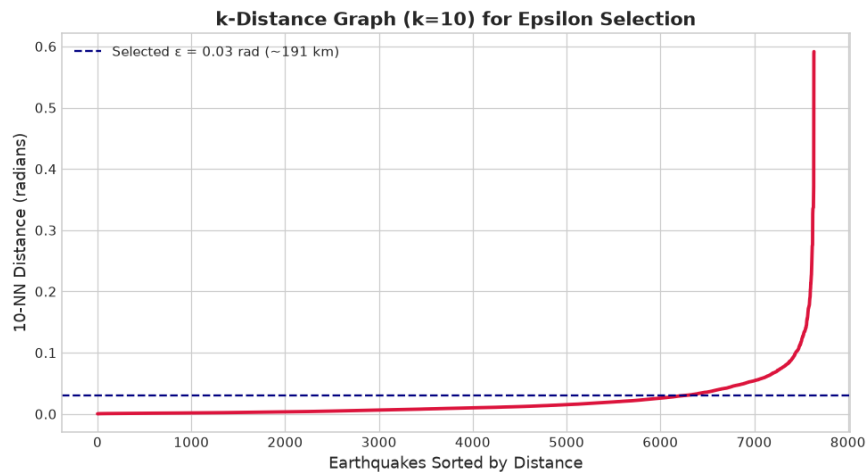


Figure 5.1: The k -distance graph ($k = 10$); the knee at $\varepsilon = 0.03$ rad marks the transition from dense seismic zones to sparse isolated events.

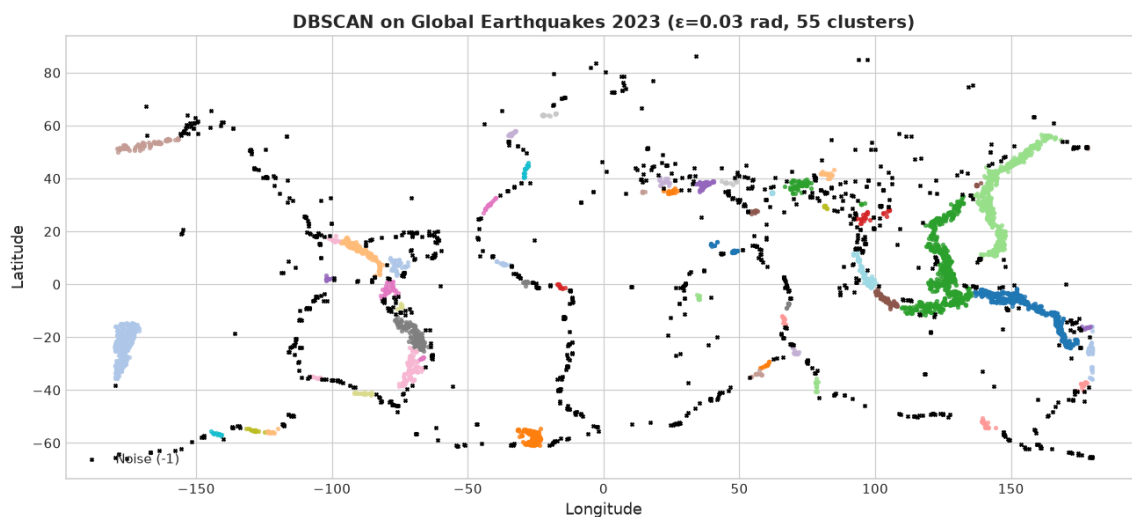


Figure 5.2: DBSCAN clusters (colors) and noise (black crosses) on the global 2023 earthquake catalogue. The clusters trace the tectonically active plate boundaries.

2. What does label -1 mean?

The point is neither a core nor a border point — it lies in a sparse region and is treated as noise/outlier (here: isolated earthquakes).

3. Why can DBSCAN struggle with varying densities?

A single ε that is small enough for a dense cluster fragments a sparse one, while one large enough for the sparse cluster merges the dense ones.

4. Why is haversine used instead of Euclidean distance?

Latitude/longitude are spherical coordinates; haversine gives true great-circle distances (radian inputs, ε in radians).

5. How is ε selected?

From the knee of the k -distance graph with $k = \text{min_samples}$ (0.03 rad here).

Experiment 6

HDBSCAN

Dataset: USGS earthquakes 2023 (7,638 events, magnitude ≥ 4.5 ; radians + haversine)

Key parameters: min_cluster_size = 50, min_samples = 10; no global ϵ

Headline result: 40 clusters, 21.5% noise; silhouette 0.631 on non-noise events

Objective

Apply hierarchical density-based clustering to discover clusters of different densities and identify noise without selecting a global radius.

Suggested Dataset

The same real earthquake catalogue as Experiment 5 (7,638 events, radians + haversine metric).

Required Libraries

scikit-learn, numpy, pandas, matplotlib.

Brief Theory

HDBSCAN removes the need for a global ϵ by building a hierarchy over all density levels:

1. compute the **core distance** $d_{\text{core}}(p)$ — the distance to a point's k -th nearest neighbor;
2. define the **mutual reachability distance** $d_{\text{mreach}}(a, b) = \max\{d_{\text{core}}(a), d_{\text{core}}(b), d(a, b)\}$;
3. build a **minimum spanning tree** over the mutual-reachability graph;
4. convert the MST into a cluster hierarchy and condense it, tracking cluster births and deaths;
5. keep the clusters with the highest **stability** $S(C) = \sum_{p \in C} (\lambda_p - \lambda_{\text{birth}}(C))$.

The result is a flat clustering with per-point membership probabilities and noise labels, with no radius to tune.

Procedure

1. Reuse the prepared radian coordinates.
2. Fit HDBSCAN with min_cluster_size = 50 and min_samples = 10.
3. Count clusters and noise events; inspect the membership probabilities.
4. Visualize the partition.
5. Compare with DBSCAN.
6. Discuss the effect of min_cluster_size.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # ---- Density-based clustering algorithms ----
8 from sklearn.cluster import DBSCAN, HDBSCAN
9 # ---- Nearest-neighbor search (used to pick epsilon) ----
10 from sklearn.neighbors import NearestNeighbors
11 # ---- Evaluation metrics ----
12 from sklearn.metrics import silhouette_score
13
14 # Aesthetics setup (consistent look across all notebooks)
15 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↪ plt.style.available else 'default')
16 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
17 plt.rcParams['figure.figsize'] = (11, 6)
18 plt.rcParams['figure.dpi'] = 110
19
20 np.random.seed(42)
21 print("Density-based libraries loaded successfully!")

```

Load the real USGS earthquake catalogue (global, 2023)

```

1 # ---- Load the real USGS earthquake catalogue (global, 2023) ----
2 df = pd.read_csv('../data/earthquakes.csv')
3 print(f"Full catalogue (magnitude >= 4.0): {len(df):,} events")
4 print(df.head(3).to_string(index=False))
5
6 # Keep well-recorded moderate+ events: magnitude >= 4.5
7 quakes = df[df['mag'] >= 4.5].reset_index(drop=True)
8 print(f"\nAnalysed subset (magnitude >= 4.5): {len(quakes):,} events")
9
10 # ---- Convert epicenters to radians [latitude, longitude] ----
11 # scikit-learn's 'haversine' metric expects radian coordinates and computes
12 # great-circle distance on the sphere (correct for global data).
13 X_rad = np.radians(quakes[['latitude', 'longitude']].values)
14 print("Coordinate range (deg): lat", quakes.latitude.min(), "to",
    ↪ quakes.latitude.max(),
15      "| lon", quakes.longitude.min(), "to", quakes.longitude.max())

```

Fit HDBSCAN (no global epsilon required)

```

1 # ---- Fit HDBSCAN (no global epsilon required) ----
2 # min_cluster_size = smallest group considered a cluster; min_samples controls
  ↪ conservativeness.
3 hdb = HDBSCAN(min_cluster_size=50, min_samples=10, metric='haversine', copy=False)
4 hdb_labels = hdb.fit_predict(X_rad)
5
6 n_clusters_hdb = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
7 n_noise_hdb = np.sum(hdb_labels == -1)
8 print(f"HDBSCAN: {n_clusters_hdb} clusters, {n_noise_hdb:,} noise events
  ↪ ({n_noise_hdb / len(X_rad) * 100:.1f}%)")
9
10 # ---- Visualize the HDBSCAN partition ----
11 plt.figure(figsize=(12, 5.5))
12 palette = sns.color_palette("tab20", max(n_clusters_hdb, 1))
13 for k in range(n_clusters_hdb):
14     mask = hdb_labels == k
15     plt.scatter(quakes.longitude[mask], quakes.latitude[mask], s=6, color=palette[k
  ↪ % 20], alpha=0.75)
16
17 noise_mask = hdb_labels == -1
18 plt.scatter(quakes.longitude[noise_mask], quakes.latitude[noise_mask], s=6,
  ↪ color='black', marker='x', label='Noise (-1)')
19 plt.title(f"HDBSCAN on Global Earthquakes 2023 ({n_clusters_hdb} clusters, no
  ↪ eps)", fontsize=13, fontweight='bold')
20 plt.xlabel("Longitude", fontsize=12)
21 plt.ylabel("Latitude", fontsize=12)
22 plt.legend(loc='lower left')
23 plt.tight_layout()
24 plt.show()

```

Evaluation / Expected Output**Table 6.1:** Density-based comparison on the 2023 earthquake catalogue.

Algorithm	Clusters	Noise	Largest cluster	Silhouette
DBSCAN ($\epsilon = 0.03$)	55	13.5%	1,629	0.326
HDBSCAN	40	21.5%	560	0.631

HDBSCAN needs no ϵ and handles the varying density along plate boundaries better than DBSCAN; it is more conservative (21.5% vs. 13.5% noise) because a cluster must remain stable across density scales. Its silhouette is roughly double DBSCAN's because the fixed- ϵ algorithm chains entire arc-shaped boundaries into single elongated clusters. Membership probabilities are high (> 0.9) deep inside seismic zones and lower near cluster edges.

Viva / Discussion Questions**1. What is hierarchical about HDBSCAN?**

It constructs a full hierarchy of density-based clusters across all density levels and then condenses

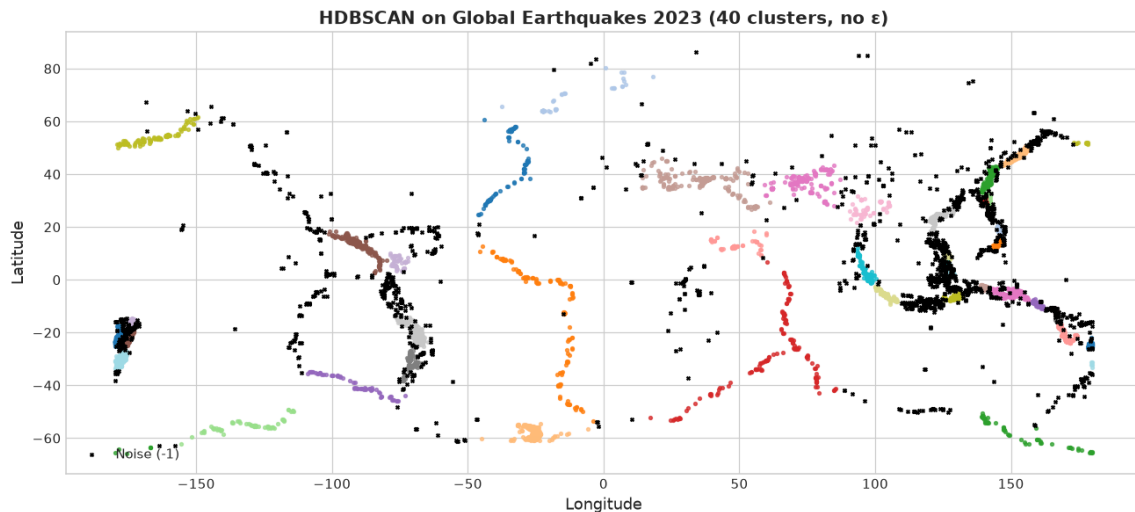


Figure 6.1: HDBSCAN clusters and noise on the same catalogue; adaptive density splits the long DBSCAN arc clusters into compact, well-separated groups.

it by tracking cluster birth and death, keeping only the most stable clusters.

2. What does membership probability mean?

A per-point confidence that it belongs to its assigned cluster, derived from the point's persistence as the density threshold changes; low values indicate borderline points.

3. Why is HDBSCAN useful for variable-density data?

Clusters are selected by stability across density levels rather than by one global ϵ , so dense and sparse clusters can coexist.

4. What does `min_cluster_size` control?

The smallest group that may become a cluster; increasing it reduces the number of small clusters, decreasing it fragments the map.

5. What does cluster persistence (stability) mean in HDBSCAN?

It measures how long a cluster survives across the density hierarchy; clusters that persist over a wide range of density levels are selected, which is why HDBSCAN handles variable densities.

Part III

Semi-Supervised Learning

Experiments in this Part

Experiment 7 Self-training for Semi-supervised Learning	29
---	----

Experiment 7

Self-training for Semi-supervised Learning

Dataset: Breast Cancer Wisconsin (569×30 , binary); training set 398 samples with only 15% labeled (61), 337 unlabeled

Key parameters: RBF-kernel SVC ($C = 1$, `probability = True`), confidence threshold $\tau = 0.80$, `max_iter = 15`

Headline result: Baseline 0.9298 $\rightarrow$ self-training 0.9415 (fully supervised ceiling 0.9766)

Objective

Use a small labeled subset and a larger unlabeled subset, iteratively adding high-confidence pseudo-labels, and compare against a supervised baseline and the fully supervised ceiling.

Suggested Dataset

Breast Cancer Wisconsin (569×30 , binary). Stratified 70/30 split; only 15% of the training labels are kept, the rest are masked as unlabeled (-1).

Required Libraries

scikit-learn, numpy.

Brief Theory

Given a labeled set L and an unlabeled pool U , self-training repeats:

1. train the base classifier f on L ;
2. compute class probabilities $f(x)$ for every $x \in U$;
3. select the predictions whose maximum probability exceeds the threshold τ and add them to L as **pseudo-labels**;
4. remove those samples from U ;
5. repeat until no new labels pass the threshold or the iteration limit is reached.

The threshold controls the purity/quantity trade-off: a low τ admits wrong pseudo-labels (confirmation bias), while a very high τ admits too few to help. The base estimator must expose `predict_proba`, which is why the SVC is calibrated (`probability=True`).

Procedure

1. Split the data stratified and scale the features (fit on training data only).
2. Mask 85% of the training labels to -1 .
3. Train a supervised baseline SVC on the labeled 61 samples.
4. Wrap an RBF SVC in `SelfTrainingClassifier` with $\tau = 0.80$ and fit on the labeled plus unlabeled data; monitor the iterations.

5. Train a fully supervised SVC on all 398 labels as the ceiling.
6. Compare accuracy and F1 on the untouched test set.

Reference Implementation

Suppress a known sklearn deprecation notice for SVC(probability=True)

```

1 # ---- Suppress a known sklearn deprecation notice for SVC(probability=True) ----
2 import warnings
3 # ---- Core numerical and plotting libraries ----
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 # ---- Dataset + train/test utilities ----
10 from sklearn.datasets import load_breast_cancer
11 from sklearn.model_selection import train_test_split
12 from sklearn.preprocessing import StandardScaler
13 # ---- Semi-supervised self-training wrapper ----
14 from sklearn.semi_supervised import SelfTrainingClassifier
15 # ---- Base estimators compared inside self-training ----
16 from sklearn.svm import SVC
17 from sklearn.ensemble import RandomForestClassifier
18 # ---- Evaluation metrics ----
19 from sklearn.metrics import accuracy_score, precision_score, recall_score,
    ↪ f1_score, roc_auc_score, confusion_matrix, classification_report
20
21 # Aesthetics setup (consistent look across all notebooks)
22 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↪ plt.style.available else 'default')
23 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
24 plt.rcParams['figure.figsize'] = (10, 6)
25 plt.rcParams['figure.dpi'] = 110
26
27 # The installed scikit-learn still supports SVC(probability=True) but warns;
    ↪ silence that specific notice
28 warnings.filterwarnings("ignore", message="The `probability` parameter was
    ↪ deprecated", category=FutureWarning)
29 np.random.seed(42)
30 print("Semi-supervised libraries imported successfully!")
31
32 # ---- Load the Breast Cancer Wisconsin dataset ----
33 data = load_breast_cancer()
34 X_full = data.data          # 30 continuous diagnostic features
35 y_full = data.target        # 0 = malignant, 1 = benign
36 feature_names = data.feature_names
37 target_names = data.target_names
38

```

```

39 # ---- Train / Test split (stratified to preserve class balance) ----
40 X_train_full, X_test, y_train_full, y_test = train_test_split(
41     X_full, y_full, test_size=0.30, random_state=42, stratify=y_full
42 )
43
44 # ---- Standardize features (fit on train only to avoid test leakage) ----
45 scaler = StandardScaler()
46 X_train_full_scaled = scaler.fit_transform(X_train_full)
47 X_test_scaled = scaler.transform(X_test)
48
49 # ---- Simulate the semi-supervised setting: hide 85% of the training labels ----
50 unlabeled_ratio = 0.85
51 rng = np.random.RandomState(42)
52 random_unlabeled_points = rng.rand(len(y_train_full)) < unlabeled_ratio # True ->
    ↳ mask
53
54 y_train_semi = np.copy(y_train_full)
55 y_train_semi[random_unlabeled_points] = -1 # -1 is the scikit-learn convention
    ↳ for "unlabeled"
56
57 n_labeled = np.sum(y_train_semi != -1)
58 n_unlabeled = np.sum(y_train_semi == -1)
59
60 print(f"Total Training Samples: {len(y_train_full)}")
61 print(f" - Labeled Samples (15%): {n_labeled}")
62 print(f" - Unlabeled Samples (85%): {n_unlabeled}")
63 print(f"Holdout Test Samples: {len(y_test)}")

```

Experiment 1: Self-Training with an RBF-kernel SVC

```

1 # ---- Experiment 1: Self-Training with an RBF-kernel SVC ----
2
3 # 1. Supervised baseline trained ONLY on the ~60 labeled samples
4 mask_labeled = (y_train_semi != -1) # boolean mask of the labeled
    ↳ subset
5 X_labeled = X_train_full_scaled[mask_labeled]
6 y_labeled = y_train_semi[mask_labeled]
7
8 base_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
9 base_svc.fit(X_labeled, y_labeled)
10 y_pred_baseline_svc = base_svc.predict(X_test_scaled)
11 acc_baseline_svc = accuracy_score(y_test, y_pred_baseline_svc)
12 f1_baseline_svc = f1_score(y_test, y_pred_baseline_svc)
13
14 # 2. Semi-supervised self-training: base SVC + iterative pseudo-labeling above
    ↳ threshold 0.80
15 self_training_svc = SelfTrainingClassifier(
16     estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
17     threshold=0.80, # only accept pseudo-labels with confidence >= 0.80

```

```

18     criterion='threshold',    # use the confidence-threshold selection rule
19     max_iter=15,
20     verbose=True              # print how many labels are added per iteration
21 )
22 self_training_svc.fit(X_train_full_scaled, y_train_semi)
23 y_pred_semi_svc = self_training_svc.predict(X_test_scaled)
24 acc_semi_svc = accuracy_score(y_test, y_pred_semi_svc)
25 f1_semi_svc = f1_score(y_test, y_pred_semi_svc)
26
27 # 3. Fully supervised "ceiling": train on 100% of the ground-truth labels
28 full_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
29 full_svc.fit(X_train_full_scaled, y_train_full)
30 y_pred_full_svc = full_svc.predict(X_test_scaled)
31 acc_full_svc = accuracy_score(y_test, y_pred_full_svc)
32 f1_full_svc = f1_score(y_test, y_pred_full_svc)
33
34 # ---- Compare the three settings ----
35 print(f"\n--- SVC Evaluation Results ---")
36 print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_svc:.4f}, F1 =
    ↳ {f1_baseline_svc:.4f}")
37 print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_svc:.4f}, F1 =
    ↳ {f1_semi_svc:.4f}")
38 print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_svc:.4f}, F1 =
    ↳ {f1_full_svc:.4f}")

```

Inspect how self-training converged

```

1 # ---- Inspect how self-training converged ----
2 # n_iter_                : number of self-training iterations performed
3 # termination_condition_ : why training stopped (e.g. 'no_change' = no new
    ↳ confident labels)
4 # transduction_          : final labels for every training sample (-1 = still
    ↳ unlabeled)
5 n_iter = self_training_svc.n_iter_
6 termination_condition = self_training_svc.termination_condition_
7 labeled_final_mask = (self_training_svc.transduction_ != -1)
8
9 print(f"Self-Training Convergence Summary:")
10 print(f" - Iterations completed: {n_iter}")
11 print(f" - Termination condition: {termination_condition}")
12 print(f" - Total samples labeled after self-training: {np.sum(labeled_final_mask)}
    ↳ / {len(y_train_full)}")
13 print(f" - Pseudo-labels newly added: {np.sum(labeled_final_mask) - n_labeled}")

```

Compact comparison table (SVC)

```

1 # ---- Compact comparison table (SVC) ----
2 summary_data = [
3     {"Model": "SVC (Baseline 15% Labeled)", "Accuracy": acc_baseline_svc,

```

```

    ↪ "F1-Score": f1_baseline_svc,
4     "Training Samples": n_labeled},
5     {"Model": "SVC (Self-Training Semi-Supervised)", "Accuracy": acc_semi_svc,
    ↪ "F1-Score": f1_semi_svc,
6     "Training Samples": np.sum(self_training_svc.transduction_ != -1)},
7     {"Model": "SVC (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_svc,
    ↪ "F1-Score": f1_full_svc,
8     "Training Samples": len(y_train_full)},
9 ]
10 summary_df = pd.DataFrame(summary_data).set_index("Model")
11 display(summary_df.style.highlight_max(subset=['Accuracy', 'F1-Score'],
    ↪ color='lightgreen'))

```

Evaluation / Expected Output

Table 7.1: Self-training compared with the supervised baseline and the fully supervised ceiling.

Setting	Accuracy	F1	Training samples
Supervised baseline (15% labeled)	0.9298	0.9469	61
Self-training (semi-supervised)	0.9415	0.9554	382
Fully supervised ceiling (100%)	0.9766	0.9813	398

Pseudo-labels added per iteration: +292, +22, +5, +2, then termination no_change (321 new labels in total). The decaying additions indicate stable convergence rather than runaway error accumulation. Using no extra human labels, self-training improves accuracy by +1.2 points over the baseline, closing part of the gap to the fully supervised model; the remaining gap is the cost of noisy pseudo-labels.

Viva / Discussion Questions

1. What is a pseudo-label?

A predicted label for an unlabeled sample that the model accepts as a training target because its confidence exceeds the threshold.

2. What is confirmation bias in self-training?

The model reinforces its own mistakes: wrong confident predictions enter the training set, make the model more confident about similar errors and can degrade performance. High thresholds and convergence monitoring mitigate this.

3. Why must the test labels remain hidden during training?

Otherwise the evaluation is contaminated: test data would influence the model and the reported accuracy would no longer estimate generalization.

4. What does the confidence threshold control?

The purity/quantity trade-off of pseudo-labels; 0.80 balanced the two here (convergence after four useful iterations).

5. Why does the SVC need probability=True?

The wrapper requires predict_proba; a plain SVC outputs only decision values, so Platt scaling is enabled.

6. How can pseudo-labels be sanity-checked when no test labels are available?

Hold out a small manually labelled set, inspect confidence distributions, and compare behaviour before and after adding pseudo-labels; agreement on held-out labels is evidence they are useful.

Part IV

Ensemble Learning

Experiments in this Part

Experiment 8 Random Forest Regression (RFR)	36
Experiment 9 Random Forest Classification (RFC)	39
Experiment 10 XGBoost	42
Experiment 11 AdaBoost.	45
Experiment 12 CatBoost	48

Experiment 8

Random Forest Regression (RFR)

Dataset: California Housing, 5,000-row random sample, 8 features; 80/20 split

Key parameters: 150 trees, max_depth = 12, out-of-bag scoring enabled

Headline result: Test R^2 0.7415 (OOB 0.7513); RMSE 0.5908; MAE 0.3999

Objective

Predict a continuous target using an ensemble of randomized decision trees; evaluate with MAE, RMSE and R^2 and inspect out-of-bag validation.

Suggested Dataset

California Housing (scikit-learn), 5,000-row random sample, target = median house value.

Required Libraries

scikit-learn, numpy, pandas.

Brief Theory

Random Forest Regression trains B decision trees on bootstrap samples of the training data; at each split only a random subset of features is considered, which decorrelates the trees. The ensemble prediction is the average of the tree outputs,

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x).$$

Averaging approximately uncorrelated trees reduces variance: for pairwise correlation ρ , $\text{Var}(\hat{f}_{\text{RF}}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, so adding trees can only help. Out-of-bag (OOB) samples — the $\approx 37\%$ not drawn in a bootstrap — provide a free internal validation estimate.

Procedure

1. Load and sample the dataset; split into training and test sets.
2. Train RandomForestRegressor with 150 trees, max_depth = 12 and OOB scoring enabled.
3. Predict the test set and compute R^2 , RMSE and MAE.
4. Report the OOB R^2 and compare it with the test R^2 to check overfitting.
5. Inspect feature importances.

Reference Implementation

Core libraries

```
1 # ---- Core libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.datasets import fetch_california_housing
8 from sklearn.model_selection import train_test_split
9 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier,
    ↳ AdaBoostRegressor, AdaBoostClassifier
10 from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error,
    ↳ accuracy_score, f1_score, roc_auc_score
11
12 import xgboost as xgb
13 import catboost as cb
14
15 # Styling setup
16 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↳ plt.style.available else 'default')
17 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
18 plt.rcParams['figure.figsize'] = (10, 6)
19 plt.rcParams['figure.dpi'] = 110
20 np.random.seed(42)
```

Load the California Housing dataset (real-world regression benchmark)

```
1 # ---- Load the California Housing dataset (real-world regression benchmark) ----
2 housing = fetch_california_housing(as_frame=True)
3 # Sample 5,000 rows so every ensemble trains quickly while remaining representative
4 df_reg = housing.frame.sample(n=5000, random_state=42)
5
6 X_reg = df_reg[housing.feature_names] # 8 socio-economic / geographic features
7 y_reg = df_reg['MedHouseVal']        # target: median house value
8
9 # ---- Train/test split (80% / 20%) ----
10 X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
11     X_reg, y_reg, test_size=0.20, random_state=42
12 )
13
14 print(f"Regression Training Samples: {X_train_reg.shape[0]}, Test Samples:
    ↳ {X_test_reg.shape[0]}")
15 df_reg.head()
```

Random Forest Regression (bagging of decision trees)

```

1 # ---- Random Forest Regression (bagging of decision trees) ----
2 # n_estimators = number of trees, oob_score = estimate accuracy from out-of-bag
  ↳ samples
3 rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12, oob_score=True,
  ↳ random_state=42, n_jobs=-1)
4 rf_reg.fit(X_train_reg, y_train_reg)
5
6 # ---- Predict and evaluate ----
7 y_pred_rf = rf_reg.predict(X_test_reg)
8 r2_rf = r2_score(y_test_reg, y_pred_rf)           # explained
  ↳ variance (1.0 = perfect)
9 rmse_rf = root_mean_squared_error(y_test_reg, y_pred_rf)   # error in target
  ↳ units
10 mae_rf = mean_absolute_error(y_test_reg, y_pred_rf)        # robust average
  ↳ error
11
12 print(f"Random Forest Regressor Results:")
13 print(f" - Out-of-Bag (OOB) R2 Score: {rf_reg.oob_score_:.4f}")
14 print(f" - Test R2 Score:                {r2_rf:.4f}")
15 print(f" - Test RMSE:                    {rmse_rf:.4f}")
16 print(f" - Test MAE:                    {mae_rf:.4f}")

```

Evaluation / Expected Output**Table 8.1:** Random Forest Regression on the California Housing sample.

Metric	Value
OOB R^2	0.7513
Test R^2	0.7415
Test RMSE	0.5908 (in \$100k units $\approx$ \$59k)
Test MAE	0.3999 (in \$100k units $\approx$ \$40k)

The OOB R^2 (0.7513) is very close to the test R^2 (0.7415), so the model is not overfitting and the OOB estimate is a reliable free validation. Feature importances are dominated by median income and location (latitude/longitude), consistent with housing economics.

Viva / Discussion Questions**1. Why does a forest reduce variance?**

Averaging B approximately uncorrelated trees gives $\rho\sigma^2 + (1 - \rho)\sigma^2/B$; as B grows the second term vanishes, so predictions are more stable than a single tree's.

2. What is bagging?

Bootstrap aggregating: each model is trained on a bootstrap sample of the data and the predictions are combined (average for regression, vote for classification).

3. Why can feature importance be misleading?

Impurity-based importance is biased toward high-cardinality/continuous features, is computed on training data (it implies no causality), and correlated features share or dilute importance arbitrarily.

4. Why does adding more trees not make a random forest overfit?

Averaging many decorrelated trees reduces variance without increasing bias; beyond a certain number the error stabilizes, so more trees cost computation but not generalization.

Experiment 9

Random Forest Classification (RFC)

Dataset: Heart Disease (data/heart_disease.csv, 303 rows, 13 features); stratified 75/25 split

Key parameters: 120 trees, max_depth = 8; one-hot encoded categorical features

Headline result: Accuracy 0.7763; F1 0.8046; ROC-AUC 0.8704

Objective

Classify observations using an ensemble of randomized decision trees, with a stratified split and classification metrics.

Suggested Dataset

Heart Disease (data/heart_disease.csv), 303 rows, 13 features plus the binary target.

Required Libraries

scikit-learn, numpy, pandas.

Brief Theory

Random Forest Classification aggregates tree votes. Each tree votes for one class and the ensemble predicts the most frequent class,

$$\hat{C}_{\text{RF}}(x) = \arg \max_k \sum_{b=1}^B \mathbb{I}(T_b(x) = k).$$

Bootstrap sampling plus feature subsampling decorrelates the trees, reducing variance relative to a single decision tree. The fraction of trees voting for the positive class also provides a class probability, which is used for ROC-AUC.

Procedure

1. Load the heart data; one-hot encode the categorical features.
2. Use a stratified 75/25 split to preserve the disease ratio.
3. Train RandomForestClassifier with 120 trees and max_depth = 8.
4. Predict labels and probabilities on the test set.
5. Compute accuracy, F1 and ROC-AUC.

Reference Implementation

Core libraries

```
1 # ---- Core libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.datasets import fetch_california_housing
8 from sklearn.model_selection import train_test_split
9 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier,
    ↪ AdaBoostRegressor, AdaBoostClassifier
10 from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error,
    ↪ accuracy_score, f1_score, roc_auc_score
11
12 import xgboost as xgb
13 import catboost as cb
14
15 # Styling setup
16 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↪ plt.style.available else 'default')
17 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
18 plt.rcParams['figure.figsize'] = (10, 6)
19 plt.rcParams['figure.dpi'] = 110
20 np.random.seed(42)
```

Load the Heart Disease dataset (real-world classification benchmark)

```
1 # ---- Load the Heart Disease dataset (real-world classification benchmark) ----
2 df_clf = pd.read_csv('../data/heart_disease.csv')
3 print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")
4
5 # Separate target and features (handle either 'target' or 'Target' as the column
    ↪ name)
6 target_col = 'target' if 'target' in df_clf.columns else 'Target'
7 y_clf = df_clf[target_col].values
8 X_clf_raw = df_clf.drop(columns=[target_col])
9
10 # One-hot encode categorical variables for RFC / XGBoost / AdaBoost
11 X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)
12
13 # Stratified split keeps the disease/no-disease ratio identical in train and test
14 X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
15     X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
16 )
17
18 print(f"Classification Training Set: {X_train_clf.shape}, Test Set:
    ↪ {X_test_clf.shape}")
```

```

19
20 # ---- Random Forest Classification (majority vote of trees) ----
21 rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42,
    ↪ n_jobs=-1)
22 rfc.fit(X_train_clf, y_train_clf)
23
24 # Predictions and probability of the positive (disease) class
25 y_pred_rfc = rfc.predict(X_test_clf)
26 y_prob_rfc = rfc.predict_proba(X_test_clf)[: , 1]
27
28 # ---- Evaluate: accuracy, F1 and ROC-AUC ----
29 acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
30 f1_rfc = f1_score(y_test_clf, y_pred_rfc)
31 auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)
32
33 print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f},
    ↪ ROC-AUC = {auc_rfc:.4f}")

```

Evaluation / Expected Output

Table 9.1: Random Forest Classification on Heart Disease (76 test samples).

Metric	Value
Accuracy	0.7763
F1-score	0.8046
ROC-AUC	0.8704

ROC-AUC 0.87 indicates good ranking quality on a small dataset; accuracy 0.776 is typical for heart-disease prediction with these features and reflects the problem's noise rather than a modelling failure. F1 and AUC are safer summaries than accuracy alone because the classes are not perfectly balanced.

Viva / Discussion Questions

1. RFR vs RFC?

Regression averages the trees' numeric outputs; classification takes a majority vote (and averages class probabilities for scoring).

2. What is majority voting?

Each tree votes for one class and the most frequent class becomes the ensemble prediction; soft voting averages the probabilities instead.

3. How can class imbalance affect accuracy?

With a rare positive class, a model can predict the majority class everywhere and still show high accuracy; precision, recall, F1 and AUC expose the failure.

4. Why is ROC-AUC threshold-independent?

It summarizes performance over all classification thresholds by comparing true- and false-positive rates, so it does not depend on a single cut-off.

Experiment 10

XGBoost

Dataset: Heart Disease (same stratified 75/25 split as RFC)

Key parameters: 120 trees, learning rate 0.08, max_depth = 4, subsample / colsample = 0.8

Headline result: Accuracy 0.7763; F1 0.8132; ROC-AUC 0.8509

Objective

Train a gradient-boosted tree classifier and analyze the effect of boosting hyperparameters.

Suggested Dataset

Heart Disease (data/heart_disease.csv), the same stratified split used for the other ensembles.

Required Libraries

xgboost, scikit-learn, numpy, pandas.

Brief Theory

XGBoost builds trees sequentially, each fitting a second-order Taylor expansion of the loss:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

where g_i and h_i are the first and second derivatives of the loss with respect to the current prediction. The regularization term Ω penalizes tree complexity and leaf weights, while row/column subsampling adds stochasticity; both control overfitting.

Procedure

1. Prepare the one-hot encoded heart data and the same stratified split.
2. Fit `XGBClassifier` with 120 trees, learning rate 0.08, depth 4, subsample and column sample 0.8, and the logistic-loss objective.
3. Predict labels and probabilities on the test set.
4. Report accuracy, F1 and ROC-AUC.
5. Discuss the complexity/generalization trade-off.

Reference Implementation

Core libraries

```

1 # ---- Core libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.datasets import fetch_california_housing
8 from sklearn.model_selection import train_test_split
9 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier,
    ↪ AdaBoostRegressor, AdaBoostClassifier
10 from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error,
    ↪ accuracy_score, f1_score, roc_auc_score
11
12 import xgboost as xgb
13 import catboost as cb
14
15 # Styling setup
16 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↪ plt.style.available else 'default')
17 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
18 plt.rcParams['figure.figsize'] = (10, 6)
19 plt.rcParams['figure.dpi'] = 110
20 np.random.seed(42)

```

Load the Heart Disease dataset (real-world classification benchmark)

```

1 # ---- Load the Heart Disease dataset (real-world classification benchmark) ----
2 df_clf = pd.read_csv('../data/heart_disease.csv')
3 print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")
4
5 # Separate target and features (handle either 'target' or 'Target' as the column
    ↪ name)
6 target_col = 'target' if 'target' in df_clf.columns else 'Target'
7 y_clf = df_clf[target_col].values
8 X_clf_raw = df_clf.drop(columns=[target_col])
9
10 # One-hot encode categorical variables for RFC / XGBoost / AdaBoost
11 X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)
12
13 # Stratified split keeps the disease/no-disease ratio identical in train and test
14 X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
15     X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
16 )
17
18 print(f"Classification Training Set: {X_train_clf.shape}, Test Set:
    ↪ {X_test_clf.shape}")

```

```

19
20 # ---- Random Forest Classification (majority vote of trees) ----
21 rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42,
    ↪ n_jobs=-1)
22 rfc.fit(X_train_clf, y_train_clf)
23
24 # Predictions and probability of the positive (disease) class
25 y_pred_rfc = rfc.predict(X_test_clf)
26 y_prob_rfc = rfc.predict_proba(X_test_clf)[: , 1]
27
28 # ---- Evaluate: accuracy, F1 and ROC-AUC ----
29 acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
30 f1_rfc = f1_score(y_test_clf, y_pred_rfc)
31 auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)
32
33 print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f},
    ↪ ROC-AUC = {auc_rfc:.4f}")

```

XGBoost classification on the same stratified split

```

1 # ---- XGBoost classification on the same stratified split ----
2 xgb_clf = xgb.XGBClassifier(
3     n_estimators=120, learning_rate=0.08, max_depth=4,
4     subsample=0.8, colsample_bytree=0.8,
5     random_state=42, eval_metric='logloss'
6 )
7 xgb_clf.fit(X_train_clf, y_train_clf)
8
9 y_pred_xgb_c = xgb_clf.predict(X_test_clf)
10 y_prob_xgb_c = xgb_clf.predict_proba(X_test_clf)[: , 1]
11
12 acc_xgb_c = accuracy_score(y_test_clf, y_pred_xgb_c)
13 f1_xgb_c = f1_score(y_test_clf, y_pred_xgb_c)
14 auc_xgb_c = roc_auc_score(y_test_clf, y_prob_xgb_c)
15 print(f"XGBoost Classifier: Accuracy = {acc_xgb_c:.4f}, F1 = {f1_xgb_c:.4f},
    ↪ ROC-AUC = {auc_xgb_c:.4f}")

```

Evaluation / Expected Output

Table 10.1: XGBoost classification on Heart Disease (76 test samples).

Metric	Value
Accuracy	0.7763
F1-score	0.8132
ROC-AUC	0.8509

XGBoost matches RFC on accuracy with a slightly higher F1 but a slightly lower AUC on this small test set; differences of 0.01–0.02 are within noise for 76 samples. On the larger regression benchmark, gradient boosting clearly leads (Experiment 8 comparison), which is the usual pattern for tabular data.

Viva / Discussion Questions

1. Bagging vs boosting?

Bagging trains models in parallel on bootstrap samples and combines them to reduce variance; boosting trains models sequentially so each corrects the previous ensemble's errors, mainly reducing bias.

2. What does the learning rate do?

It shrinks each tree's contribution; smaller values need more trees but usually generalize better, larger values fit faster and can overfit.

3. Why can boosting overfit?

Later trees keep fitting remaining errors, including label noise, so the ensemble can become too complex; depth limits, regularization, subsampling and early stopping control this.

4. How do the learning rate and the number of trees interact?

A smaller learning rate shrinks each tree's contribution and usually needs more trees; the pair must be tuned together to balance underfitting and overfitting.

Experiment 11

AdaBoost

Dataset: Heart Disease (same stratified 75/25 split)

Key parameters: 100 decision stumps, learning rate 0.1

Headline result: Accuracy 0.7895; F1 0.8182; ROC-AUC 0.8714 (best of the four ensembles)

Objective

Build an adaptive boosting classifier and study how weak learners combine into a stronger ensemble.

Suggested Dataset

Heart Disease (data/heart_disease.csv), same split as the other ensembles.

Required Libraries

scikit-learn, numpy, pandas.

Brief Theory

AdaBoost trains weak learners (decision stumps) sequentially on re-weighted data. Misclassified samples receive larger weights, so each new learner focuses on the current errors. At round m the weighted error ϵ_m determines the learner weight

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right),$$

and the sample weights are updated as $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))$. The final prediction is a weighted vote of all stumps. Because the exponential loss grows quickly for hard points, AdaBoost is sensitive to label noise and outliers.

Procedure

1. Prepare the heart data and the stratified split.
2. Fit AdaBoostClassifier with 100 stumps and learning rate 0.1.
3. Predict labels and probabilities on the test set.
4. Report accuracy, F1 and ROC-AUC; compare with RFC and XGBoost.

Reference Implementation

Core libraries

```

1 # ---- Core libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.datasets import fetch_california_housing
8 from sklearn.model_selection import train_test_split
9 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier,
   ↳ AdaBoostRegressor, AdaBoostClassifier
10 from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error,
   ↳ accuracy_score, f1_score, roc_auc_score
11
12 import xgboost as xgb
13 import catboost as cb
14
15 # Styling setup
16 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
   ↳ plt.style.available else 'default')
17 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
18 plt.rcParams['figure.figsize'] = (10, 6)
19 plt.rcParams['figure.dpi'] = 110
20 np.random.seed(42)

```

Load the Heart Disease dataset (real-world classification benchmark)

```

1 # ---- Load the Heart Disease dataset (real-world classification benchmark) ----
2 df_clf = pd.read_csv('../data/heart_disease.csv')
3 print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")
4
5 # Separate target and features (handle either 'target' or 'Target' as the column
   ↳ name)
6 target_col = 'target' if 'target' in df_clf.columns else 'Target'

```

```

7 y_clf = df_clf[target_col].values
8 X_clf_raw = df_clf.drop(columns=[target_col])
9
10 # One-hot encode categorical variables for RFC / XGBoost / AdaBoost
11 X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)
12
13 # Stratified split keeps the disease/no-disease ratio identical in train and test
14 X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
15     X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
16 )
17
18 print(f"Classification Training Set: {X_train_clf.shape}, Test Set:
19     ↪ {X_test_clf.shape}")
20
21 # ---- Random Forest Classification (majority vote of trees) ----
22 rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42,
23     ↪ n_jobs=-1)
24 rfc.fit(X_train_clf, y_train_clf)
25
26 # Predictions and probability of the positive (disease) class
27 y_pred_rfc = rfc.predict(X_test_clf)
28 y_prob_rfc = rfc.predict_proba(X_test_clf)[: , 1]
29
30 # ---- Evaluate: accuracy, F1 and ROC-AUC ----
31 acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
32 f1_rfc = f1_score(y_test_clf, y_pred_rfc)
33 auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)
34
35 print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f},
36     ↪ ROC-AUC = {auc_rfc:.4f}")

```

AdaBoost classification

```

1 # ---- AdaBoost classification ----
2 ada_clf = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
3 ada_clf.fit(X_train_clf, y_train_clf)
4
5 y_pred_ada_c = ada_clf.predict(X_test_clf)
6 y_prob_ada_c = ada_clf.predict_proba(X_test_clf)[: , 1]
7
8 acc_ada_c = accuracy_score(y_test_clf, y_pred_ada_c)
9 f1_ada_c = f1_score(y_test_clf, y_pred_ada_c)
10 auc_ada_c = roc_auc_score(y_test_clf, y_prob_ada_c)
11 print(f"AdaBoost Classifier: Accuracy = {acc_ada_c:.4f}, F1 = {f1_ada_c:.4f},
12     ↪ ROC-AUC = {auc_ada_c:.4f}")

```

Evaluation / Expected Output

AdaBoost with shallow stumps slightly outperforms the deeper models on this small dataset — a common outcome when the signal is mostly additive and the sample size is limited. It remains a cheap, strong

Table 11.1: AdaBoost classification on Heart Disease (76 test samples).

Metric	Value
Accuracy	0.7895
F1-score	0.8182
ROC-AUC	0.8714

baseline; its weakness is sensitivity to label noise, where regularized gradient boosting is more robust.

Viva / Discussion Questions

1. Why use weak learners?

Individually they are simple and low-variance; boosting combines many of them so the ensemble can fit complex boundaries while each step stays simple.

2. How does AdaBoost focus on difficult examples?

After each round, weights of misclassified samples increase, so the next learner optimizes a distribution that emphasizes current errors.

3. What is the effect of noisy labels?

Mislabeled points look “hard”, receive ever-larger weights and can dominate training, degrading accuracy — AdaBoost is known to be noise-sensitive.

4. What is the difference between AdaBoost and gradient boosting?

AdaBoost reweights misclassified samples and combines weak learners by weighted votes; gradient boosting fits each new learner to the residual (gradient) of the loss, which generalizes to many loss functions.

Experiment 12

CatBoost

Dataset: Heart Disease (raw table with categorical features: sex, cp, fbs, restecg, exang, slope, ca, thal)

Key parameters: 150 iterations, depth 5, learning rate 0.08, native cat_features

Headline result: Accuracy 0.7632; F1 0.7955; ROC-AUC 0.8551

Objective

Train a gradient-boosted decision-tree model with native handling of categorical variables.

Suggested Dataset

Heart Disease (data/heart_disease.csv) using the raw (unencoded) table so CatBoost can process categorical columns directly.

Required Libraries

catboost, pandas, scikit-learn.

Brief Theory

CatBoost is a gradient-boosting library designed for tables that mix numerical and categorical features. It uses **ordered boosting** — leaf values are computed on random permutations of the data — to avoid target leakage and prediction shift, and builds **oblivious (symmetric) trees**, where the same split condition is used at every node of a depth level. Categorical features are encoded internally with ordered target statistics, so the categorical columns must be declared through `cat_features` and the target must never leak into the encoding.

Procedure

1. Identify the categorical columns in the raw heart table and cast them to string.
2. Use the same stratified 75/25 split on the raw features.
3. Fit `CatBoostClassifier` with 150 iterations, depth 5, learning rate 0.08 and the categorical columns declared.
4. Predict labels and probabilities; report accuracy, F1 and ROC-AUC.
5. Compare all four classifiers on the same split.

Reference Implementation

Core libraries

```
1 # ---- Core libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.datasets import fetch_california_housing
8 from sklearn.model_selection import train_test_split
9 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier,
10     ↪ AdaBoostRegressor, AdaBoostClassifier
11 from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error,
12     ↪ accuracy_score, f1_score, roc_auc_score
13
14
15 import xgboost as xgb
16 import catboost as cb
17
18 # Styling setup
19 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
20     ↪ plt.style.available else 'default')
21 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
22 plt.rcParams['figure.figsize'] = (10, 6)
23 plt.rcParams['figure.dpi'] = 110
```

```
20 np.random.seed(42)
```

Load the Heart Disease dataset (real-world classification benchmark)

```
1 # ---- Load the Heart Disease dataset (real-world classification benchmark) ----
2 df_clf = pd.read_csv('../data/heart_disease.csv')
3 print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")
4
5 # Separate target and features (handle either 'target' or 'Target' as the column
  ↳ name)
6 target_col = 'target' if 'target' in df_clf.columns else 'Target'
7 y_clf = df_clf[target_col].values
8 X_clf_raw = df_clf.drop(columns=[target_col])
9
10 # One-hot encode categorical variables for RFC / XGBoost / AdaBoost
11 X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)
12
13 # Stratified split keeps the disease/no-disease ratio identical in train and test
14 X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
15     X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
16 )
17
18 print(f"Classification Training Set: {X_train_clf.shape}, Test Set:
  ↳ {X_test_clf.shape}")
19
20 # ---- Random Forest Classification (majority vote of trees) ----
21 rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42,
  ↳ n_jobs=-1)
22 rfc.fit(X_train_clf, y_train_clf)
23
24 # Predictions and probability of the positive (disease) class
25 y_pred_rfc = rfc.predict(X_test_clf)
26 y_prob_rfc = rfc.predict_proba(X_test_clf)[: , 1]
27
28 # ---- Evaluate: accuracy, F1 and ROC-AUC ----
29 acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
30 f1_rfc = f1_score(y_test_clf, y_pred_rfc)
31 auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)
32
33 print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f},
  ↳ ROC-AUC = {auc_rfc:.4f}")
```

CatBoost classification with native categorical features

```
1 # ---- CatBoost classification with native categorical features ----
2 # Identify categorical columns in the raw (unencoded) heart data
3 categorical_cols = X_clf_raw.select_dtypes(include=['object',
  ↳ 'category']).columns.tolist()
4 X_raw_cb = X_clf_raw.copy()
```

```

5 for col in categorical_cols:
6     X_raw_cb[col] = X_raw_cb[col].astype(str)
7
8 # Same stratified split on the raw feature table
9 X_tr_cb, X_te_cb, y_tr_cb, y_te_cb = train_test_split(
10     X_raw_cb, y_clf, test_size=0.25, random_state=42, stratify=y_clf
11 )
12
13 cb_clf = cb.CatBoostClassifier(
14     iterations=150, learning_rate=0.08, depth=5,
15     cat_features=categorical_cols, random_seed=42, verbose=False
16 )
17 cb_clf.fit(X_tr_cb, y_tr_cb)
18
19 y_pred_cb_c = cb_clf.predict(X_te_cb)
20 y_prob_cb_c = cb_clf.predict_proba(X_te_cb)[: , 1]
21
22 acc_cb_c = accuracy_score(y_te_cb, y_pred_cb_c)
23 f1_cb_c = f1_score(y_te_cb, y_pred_cb_c)
24 auc_cb_c = roc_auc_score(y_te_cb, y_prob_cb_c)
25 print(f"CatBoost Classifier: Accuracy = {acc_cb_c:.4f}, F1 = {f1_cb_c:.4f},
    ↳ ROC-AUC = {auc_cb_c:.4f}")

```

Evaluation / Expected Output

Table 12.1: Ensemble classification comparison on the same Heart Disease split (76 test samples).

Model	Accuracy	F1	ROC-AUC
Random Forest (RFC)	0.7763	0.8046	0.8704
XGBoost	0.7763	0.8132	0.8509
AdaBoost	0.7895	0.8182	0.8714
CatBoost	0.7632	0.7955	0.8551

CatBoost performs at a similar level to the other ensembles without any manual one-hot encoding — its native categorical handling does the encoding internally. On this tiny dataset the advanced target statistics cannot show their full advantage (they matter most with high-cardinality categories, e.g. thousands of city or user IDs); all differences are within noise for 76 test samples.

Viva / Discussion Questions

1. How does CatBoost treat categorical features?

It computes ordered target statistics (the mean target per category on random permutations, with prior smoothing) rather than one-hot vectors.

2. Why should categories not be encoded using target leakage?

Plain target encoding uses each row's own target when encoding that row, which leaks label information and inflates training performance; ordered statistics avoid this.

3. CatBoost vs ordinary one-hot encoding?

One-hot explodes dimensionality for high-cardinality features and loses category statistics; Cat-

Boost encodes compactly and exploits target–category relationships with leakage control.

4. **Why does CatBoost often need less hyperparameter tuning?**

Its ordered boosting and ordered target statistics reduce target leakage, and symmetric (oblivious) trees are strong defaults, so good results are obtained with modest tuning.

Part V

Neural Networks: Multilayer Perceptron

Experiments in this Part

Experiment 13 Multilayer Perceptron (MLP).	54
--	----

Experiment 13

Multilayer Perceptron (MLP)

Dataset: Digits (scikit-learn): 1,797 samples, 8×8 images (64 features), 10 classes; 70/15/15 split

Key parameters: scikit-learn (128, 64) ReLU + Adam + early stopping; PyTorch 64–128–64–10 with BatchNorm and Dropout

Headline result: scikit-learn 0.9630; PyTorch 0.9815 test accuracy (GPU)

Objective

Implement feed-forward neural networks for handwritten-digit classification and study hidden-layer size and regularization.

Suggested Dataset

Digits (scikit-learn): 1,797 samples, 8×8 grayscale images (64 features), 10 classes; stratified 70/15/15 train/validation/test split, standardized on the training data only.

Required Libraries

scikit-learn, PyTorch, numpy, pandas, matplotlib.

Brief Theory

An MLP stacks affine transformations with non-linear activations,

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)}), \quad \hat{y} = \text{softmax}(W^{(L+1)}a^{(L)} + b^{(L+1)}),$$

and is trained by backpropagation to minimize the categorical cross-entropy

$$\mathcal{L} = - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \ln \hat{y}_{ik} + \frac{\alpha}{2} \sum_l \|W^{(l)}\|_F^2.$$

Batch normalization stabilizes layer inputs and speeds training, **dropout** randomly zeroes neurons to regularize, and **early stopping** / learning-rate scheduling stop training when validation performance stalls.

Procedure

1. Load the digits, split 70/15/15 and standardize (fit on training data only).
2. Train a scikit-learn `MLPClassifier` with hidden layers (128, 64), ReLU, Adam, L_2 regularization and early stopping.
3. Build a custom PyTorch MLP: 64–128–64–10 with BatchNorm and Dropout, cross-entropy loss, Adam with weight decay, 50 epochs and a plateau scheduler.
4. Train with the validation loop and plot the loss curve.

5. Evaluate the test set and report the per-class classification report.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # ---- Dataset + preprocessing/metrics ----
8 from sklearn.datasets import load_digits
9 from sklearn.model_selection import train_test_split
10 from sklearn.preprocessing import StandardScaler
11 # ---- Scikit-Learn MLP implementation ----
12 from sklearn.neural_network import MLPClassifier
13 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
14
15 # ---- PyTorch for the custom deep MLP ----
16 import torch
17 import torch.nn as nn
18 import torch.optim as optim
19 from torch.utils.data import DataLoader, TensorDataset
20
21 # Styling setup (consistent look across all notebooks)
22 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
23     ↪ plt.style.available else 'default')
24 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
25 plt.rcParams['figure.figsize'] = (10, 6)
26 plt.rcParams['figure.dpi'] = 110
27
28 # Fix seeds for reproducible training
29 torch.manual_seed(42)
30 torch.cuda.manual_seed_all(42) # reproducible GPU training
31 np.random.seed(42)
32 print(f"PyTorch version: {torch.__version__}")
33 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # use GPU if
34     ↪ available
35 print(f"Computation Device: {device}")
36
37 # ---- Load the handwritten digits dataset (8x8 grayscale images) ----
38 digits = load_digits()
39 X_raw = digits.data # flattened images: 1797 samples x 64 pixel features
40 y = digits.target # digit class 0..9
41
42 print(f"Dataset Shape: {X_raw.shape}, Target Classes: {np.unique(y)}")
43 # ---- Train / Validation / Test split (70% / 15% / 15%) ----
44 # First cut off the test set...

```

```

43 X_train_val, X_test, y_train_val, y_test = train_test_split(
44     X_raw, y, test_size=0.15, random_state=42, stratify=y
45 )
46 # ...then split the remainder into train and validation
47 X_train, X_val, y_train, y_val = train_test_split(
48     X_train_val, y_train_val, test_size=0.1765, random_state=42,
49     ↪ stratify=y_train_val
50 )
51 # ---- Feature scaling: fit on training data only (avoid validation/test leakage)
52 ↪ ----
53 scaler = StandardScaler()
54 X_train_scaled = scaler.fit_transform(X_train)
55 X_val_scaled = scaler.transform(X_val)
56 X_test_scaled = scaler.transform(X_test)
57
58 print(f"Training samples: {X_train_scaled.shape[0]}")
59 print(f"Validation samples: {X_val_scaled.shape[0]}")
60 print(f"Test samples: {X_test_scaled.shape[0]}")

```

Scikit-Learn MLPClassifier: 64 -> 128 -> 64 -> 10

```

1 # ---- Scikit-Learn MLPClassifier: 64 -> 128 -> 64 -> 10 ----
2 mlp_sklearn = MLPClassifier(
3     hidden_layer_sizes=(128, 64), # two hidden layers
4     activation='relu',
5     solver='adam', # adaptive gradient optimizer
6     alpha=0.001, # L2 regularization strength
7     batch_size=64,
8     learning_rate_init=0.005,
9     max_iter=150,
10    early_stopping=True, # stop when validation score stops improving
11    validation_fraction=0.15, # internal validation split for early stopping
12    random_state=42
13 )
14
15 # Train (forward pass + backpropagation happen inside .fit)
16 mlp_sklearn.fit(X_train_scaled, y_train)
17
18 # Evaluate on the untouched test set
19 y_pred_sk = mlp_sklearn.predict(X_test_scaled)
20 acc_sk = accuracy_score(y_test, y_pred_sk)
21
22 print(f"Scikit-Learn MLP Test Accuracy: {acc_sk:.4f}")
23 print(f"Converged after {mlp_sklearn.n_iter_} iterations")
24
25 # ---- Plot the training loss curve recorded by scikit-learn ----
26 plt.figure(figsize=(8, 4.5))
27 plt.plot(mlp_sklearn.loss_curve_, color='crimson', lw=2.2, label='Training Loss')

```

```

28 if hasattr(mlp_sklearn, 'validation_scores_'):
29     # validation_scores_ stores accuracy; convert to error for a comparable scale
30     plt.plot([1 - s for s in mlp_sklearn.validation_scores_], color='navy', lw=2.2,
31             ↪ linestyle='--', label='Validation Error')
32 plt.title("Scikit-Learn MLP Training Loss Curve", fontsize=13, fontweight='bold')
33 plt.xlabel("Epochs / Iterations", fontsize=11)
34 plt.ylabel("Loss", fontsize=11)
35 plt.legend()
36 plt.tight_layout()
37 plt.show()

```

Convert scaled NumPy arrays into PyTorch tensors + DataLoaders

```

1 # ---- Convert scaled NumPy arrays into PyTorch tensors + DataLoaders ----
2 train_dataset = TensorDataset(torch.tensor(X_train_scaled, dtype=torch.float32),
3 ↪ torch.tensor(y_train, dtype=torch.long))
4 val_dataset   = TensorDataset(torch.tensor(X_val_scaled, dtype=torch.float32),
5 ↪ torch.tensor(y_val, dtype=torch.long))
6 test_dataset  = TensorDataset(torch.tensor(X_test_scaled, dtype=torch.float32),
7 ↪ torch.tensor(y_test, dtype=torch.long))
8
9 # DataLoaders handle mini-batching and shuffling during training
10
11 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
12 val_loader   = DataLoader(val_dataset, batch_size=64, shuffle=False)
13 test_loader  = DataLoader(test_dataset, batch_size=64, shuffle=False)
14
15 class DeepMLP(nn.Module):
16     """Fully-connected classifier: 64 -> 128 -> 64 -> 10 with BatchNorm +
17     ↪ Dropout."""
18     def __init__(self, in_features=64, num_classes=10):
19         super(DeepMLP, self).__init__()
20         self.net = nn.Sequential(
21             # Hidden block 1
22             nn.Linear(in_features, 128),
23             nn.BatchNorm1d(128),    # stabilizes activations, speeds up training
24             nn.ReLU(),             # non-linear activation
25             nn.Dropout(0.25),      # regularization: randomly drop 25% of neurons
26
27             # Hidden block 2
28             nn.Linear(128, 64),
29             nn.BatchNorm1d(64),
30             nn.ReLU(),
31             nn.Dropout(0.20),
32
33             # Output layer: 10 raw logits (one per digit class)
34             nn.Linear(64, num_classes)
35         )
36
37     def forward(self, x):

```

```

33         return self.net(x)
34
35 model = DeepMLP().to(device)    # move all parameters to CPU/GPU
36 print(model)

```

Training configuration

```

1  # ---- Training configuration ----
2  criterion = nn.CrossEntropyLoss()                                #
   ↳ softmax + NLL loss
3  optimizer = optim.Adam(model.parameters(), lr=0.003, weight_decay=1e-4)  #
   ↳ weight decay = L2
4  scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5,
   ↳ patience=5)
5
6  epochs = 50
7  history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
8
9  for epoch in range(epochs):
10     # ----- Training phase -----
11     model.train()                                                # enable Dropout/BatchNorm training
   ↳ behavior
12     running_loss, correct, total = 0.0, 0, 0
13     for inputs, labels in train_loader:
14         inputs, labels = inputs.to(device), labels.to(device)
15
16         optimizer.zero_grad()                                    # reset gradients from the previous step
17         outputs = model(inputs)                                  # forward pass
18         loss = criterion(outputs, labels)                        # compute cross-entropy
19         loss.backward()                                         # backpropagation (compute gradients)
20         optimizer.step()                                        # update weights
21
22     # Accumulate metrics (loss is averaged over the batch, so multiply by batch
   ↳ size)
23     running_loss += loss.item() * inputs.size(0)
24     _, preds = torch.max(outputs, 1)                            # predicted class = highest logit
25     correct += (preds == labels).sum().item()
26     total += labels.size(0)
27
28     epoch_train_loss = running_loss / total
29     epoch_train_acc = correct / total
30
31     # ----- Validation phase -----
32     model.eval()                                                # disable Dropout, use running BatchNorm
   ↳ stats
33     val_loss, val_correct, val_total = 0.0, 0, 0
34     with torch.no_grad():                                       # no gradients needed for evaluation
35         for inputs, labels in val_loader:
36             inputs, labels = inputs.to(device), labels.to(device)

```

```

37         outputs = model(inputs)
38         loss = criterion(outputs, labels)
39
40         val_loss += loss.item() * inputs.size(0)
41         _, preds = torch.max(outputs, 1)
42         val_correct += (preds == labels).sum().item()
43         val_total += labels.size(0)
44
45     epoch_val_loss = val_loss / val_total
46     epoch_val_acc = val_correct / val_total
47     scheduler.step(epoch_val_loss)          # reduce LR if validation loss plateaus
48
49     # Record metrics for plotting
50     history['train_loss'].append(epoch_train_loss)
51     history['val_loss'].append(epoch_val_loss)
52     history['train_acc'].append(epoch_train_acc)
53     history['val_acc'].append(epoch_val_acc)
54
55     # Print a progress line every 10 epochs (and the first one)
56     if (epoch + 1) % 10 == 0 or epoch == 0:
57         print(f"Epoch [{epoch+1:02d}/{epochs}] "
58               f"Train Loss: {epoch_train_loss:.4f} | Train Acc:
59     ↪ {epoch_train_acc:.4f} || "
60               f"Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc:.4f}")

```

Final evaluation on the held-out test set

```

1  # ---- Final evaluation on the held-out test set ----
2  model.eval()
3  test_preds = []
4  test_targets = []
5
6  with torch.no_grad():
7      for inputs, labels in test_loader:
8          inputs = inputs.to(device)
9          outputs = model(inputs)
10         _, preds = torch.max(outputs, 1)          # predicted digit
11         test_preds.extend(preds.cpu().numpy())    # move back to CPU for metrics
12         test_targets.extend(labels.numpy())
13
14 acc_pytorch = accuracy_score(test_targets, test_preds)
15 print(f"PyTorch Deep MLP Test Accuracy: {acc_pytorch:.4f}\n")
16 print("Detailed Classification Report:")
17 # Per-class precision/recall/F1 for all 10 digits
18 print(classification_report(test_targets, test_preds, digits=4))

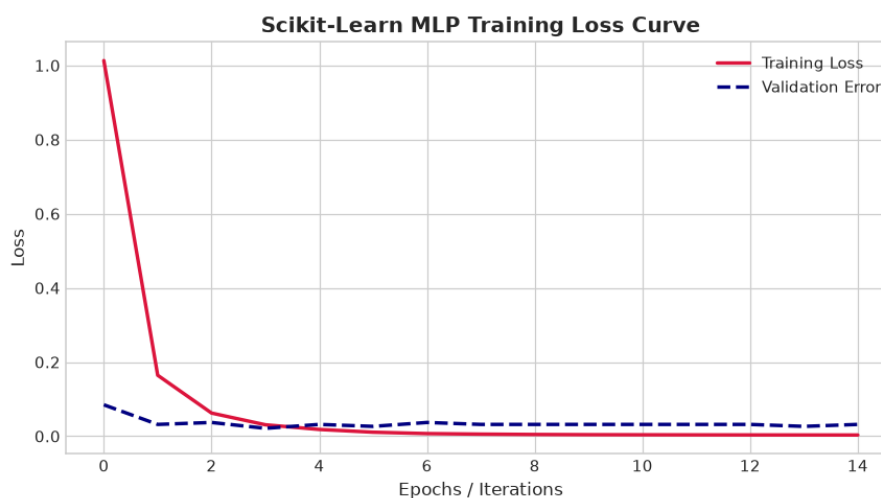
```

Evaluation / Expected Output

Training loss fell from 1.20 to about 0.02 while the validation loss tracked it closely, so there is no severe overfitting. Per-class precision and recall are near 1.00 for almost all digits; the residual errors are

Table 13.1: MLP implementations on the digits test set (270 images).

Model	Test accuracy	Notes
Scikit-learn MLP (128–64)	0.9630	early stopping after 15 iterations
PyTorch deep MLP	0.9815	GPU (CUDA), 50 epochs, validation accuracy 0.9889

**Figure 13.1:** Training loss curve of the scikit-learn MLP; early stopping halted training after 15 iterations.

visually similar digits. The custom PyTorch model beats the scikit-learn baseline by about two points thanks to BatchNorm, Dropout and scheduled training over more epochs.

Viva / Discussion Questions

1. Why is scaling important for MLP?

Gradient descent converges poorly when input features have very different scales; standardization equalizes their influence and stabilizes the loss surface.

2. What is backpropagation?

Reverse-mode application of the chain rule that computes the gradient of the loss with respect to every weight, layer by layer.

3. What causes vanishing/exploding gradients?

Repeated multiplication of small (or large) Jacobians through many layers; ReLU, BatchNorm, careful initialization and residual connections mitigate it.

4. Why is the PyTorch model better here?

BatchNorm and Dropout plus a longer, scheduled training; the scikit-learn model stops early at iteration 15.

5. Why does early stopping act as a regularizer?

It stops training before the network starts fitting noise; the validation error is at its minimum at that point, which limits effective model complexity.

Part VI

Sequence Modeling: Recurrent Neural Networks

Experiments in this Part

Experiment 14 Recurrent Neural Network (RNN).	62
---	----

Experiment 14

Recurrent Neural Network (RNN)

Dataset: Synthetic 20-step sequences (1,200; label = positive sum) and Monthly Airline Passengers (12-month windows)

Key parameters: Vanilla RNN vs LSTM; 2 layers, hidden 64 (forecasting) / 32 (classification); Adam; MSE / BCE loss

Headline result: Forecasting: LSTM RMSE 44.12 vs RNN 69.93 passengers; Classification: LSTM 0.9450 vs RNN 0.9300

Objective

Build recurrent networks for sequential data — sequence classification and time-series forecasting — and compare a vanilla RNN with an LSTM.

Suggested Dataset

(a) Synthetic sequence benchmark: 1,200 sequences of length 20, label = 1 if the sum is positive (1,000 train / 200 test). (b) Monthly Airline Passengers (data/airline_passengers.csv): 144 months, 12-month sliding windows, chronological 80/20 split.

Required Libraries

PyTorch, numpy, pandas, matplotlib (the manual's TensorFlow/Keras example is implemented equivalently in PyTorch, the framework installed in this environment).

Brief Theory

A recurrent network processes a sequence one step at a time while maintaining a hidden state,

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh}), \quad \hat{y} = W_{ho}h_T + b_o.$$

Training uses backpropagation through time (BPTT). Repeated multiplication by W_{hh} makes gradients vanish (or explode) over long sequences. The **LSTM** solves this with gates — forget f_t , input i_t and output o_t — and an additive cell state $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, which lets gradients flow across many steps.

Procedure

1. Generate 1,200 labeled 20-step sequences (label = 1 if the sum is positive) and split 1,000/200.
2. Build vanilla RNN and LSTM classifiers (hidden 32, linear sigmoid output); train 10 epochs with Adam and binary cross-entropy; report test accuracy.
3. For forecasting, normalize the passenger series and build 12-month sliding windows with a chronological 80/20 split.
4. Train RNN and LSTM forecasters (2 layers, hidden 64, MSE, Adam, 120 epochs).
5. Report RMSE/MAE and plot the forecast trajectories.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # ---- Scaling + regression metrics ----
8 from sklearn.preprocessing import MinMaxScaler
9 from sklearn.metrics import root_mean_squared_error, mean_absolute_error,
    ↪ accuracy_score
10
11 # ---- PyTorch for the recurrent models ----
12 import torch
13 import torch.nn as nn
14 import torch.optim as optim
15 from torch.utils.data import DataLoader, TensorDataset
16
17 # Aesthetics setup (consistent look across all notebooks)
18 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↪ plt.style.available else 'default')
19 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
20 plt.rcParams['figure.figsize'] = (10, 5)
21 plt.rcParams['figure.dpi'] = 110
22
23 # Reproducibility
24 torch.manual_seed(42)
25 torch.cuda.manual_seed_all(42) # reproducible GPU training
26 np.random.seed(42)
27 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
28 print(f"PyTorch version: {torch.__version__} on {device}")
29
30 # ---- Load the monthly airline passengers time series ----
31 df_air = pd.read_csv('../data/airline_passengers.csv')
32 print("Dataset Head:")
33 display(df_air.head())
34
35 # Extract the target series as float32 (shape: N x 1)
36 passengers = df_air['Passengers'].values.astype(np.float32).reshape(-1, 1)
37 # ---- Normalize to [0, 1] (RNNs train much better on small inputs) ----
38 scaler = MinMaxScaler(feature_range=(0, 1))
39 scaled_data = scaler.fit_transform(passengers)
40
41 # ---- Function to build supervised sliding-window sequences ----
42 # For each time step t: X = values [t-seq_len ... t-1], y = value at t
43 def create_sequences(data, seq_length=12):
44     xs, ys = [], []

```

```

45     for i in range(len(data) - seq_length):
46         x = data[i:(i + seq_length)]
47         y = data[i + seq_length]
48         xs.append(x)
49         ys.append(y)
50     return np.array(xs), np.array(ys)
51
52 SEQ_LENGTH = 12 # 12-month lookback = one full annual seasonal cycle
53 X_seq, y_seq = create_sequences(scaled_data, seq_length=SEQ_LENGTH)
54
55 # ---- Temporal split: 80% train, 20% test WITHOUT shuffling (time order matters)
56 ↪ ----
57 train_size = int(len(X_seq) * 0.80)
58 X_train, X_test = X_seq[:train_size], X_seq[train_size:]
59 y_train, y_test = y_seq[:train_size], y_seq[train_size:]
60
61 print(f"Total sequences created: {len(X_seq)}")
62 print(f"Training sequences:      {X_train.shape[0]} (Shape: {X_train.shape})")
63 print(f"Testing sequences:      {X_test.shape[0]} (Shape: {X_test.shape})")
64
65 # ---- DataLoader for mini-batch training (shuffle only the training set) ----
66 train_loader = DataLoader(
67     TensorDataset(torch.tensor(X_train, dtype=torch.float32), torch.tensor(y_train,
68     ↪ dtype=torch.float32)),
69     batch_size=16,
70     shuffle=True
71 )

```

Vanilla RNN and LSTM model definitions

```

1 class VanillaRNN(nn.Module):
2     """Elman RNN: 2 stacked recurrent layers + a linear output layer."""
3     def __init__(self, input_size=1, hidden_size=64, num_layers=2):
4         super(VanillaRNN, self).__init__()
5         self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
6         self.fc = nn.Linear(hidden_size, 1) # map final hidden state to a scalar
7         ↪ forecast
8
9     def forward(self, x):
10         out, _ = self.rnn(x) # out: (batch, seq_len, hidden)
11         out = self.fc(out[:, -1, :]) # use ONLY the last time step's hidden
12         ↪ state
13         return out
14
15 class LSTMModel(nn.Module):
16     """LSTM: same interface, but gated cells preserve long-term information."""
17     def __init__(self, input_size=1, hidden_size=64, num_layers=2):
18         super(LSTMModel, self).__init__()
19         self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)

```

```

18         self.fc = nn.Linear(hidden_size, 1)
19
20     def forward(self, x):
21         out, _ = self.lstm(x)
22         out = self.fc(out[:, -1, :])
23         return out
24
25 # Instantiate both models on the selected device so they can be compared fairly
26 rnn_model = VanillaRNN().to(device)
27 lstm_model = LSTMModel().to(device)
28 print("Models instantiated successfully!")

```

Generic training function shared by both architectures

```

1 # ---- Generic training function shared by both architectures ----
2 def train_model(model, loader, epochs=120, lr=0.005):
3     criterion = nn.MSELoss() # regression -> mean squared
4     ↪ error
5     optimizer = optim.Adam(model.parameters(), lr=lr) # Adam optimizer
6     losses = []
7
8     for epoch in range(epochs):
9         model.train()
10        total_loss = 0.0
11        for x_b, y_b in loader:
12            x_b, y_b = x_b.to(device), y_b.to(device)
13
14            optimizer.zero_grad() # clear previous gradients
15            pred = model(x_b) # forward pass through the recurrent net
16            loss = criterion(pred, y_b) # MSE between forecast and truth
17            loss.backward() # backpropagation through time (BPTT)
18            optimizer.step() # update weights
19
20            total_loss += loss.item() * len(x_b) # de-average the batch loss
21
22        epoch_loss = total_loss / len(loader.dataset)
23        losses.append(epoch_loss) # track loss per epoch for plotting
24
25    return losses
26
27 # ---- Train both architectures with identical hyperparameters ----
28 print("Training Vanilla RNN...")
29 rnn_losses = train_model(rnn_model, train_loader, epochs=120, lr=0.005)
30
31 print("Training LSTM Model...")
32 lstm_losses = train_model(lstm_model, train_loader, epochs=120, lr=0.005)
33 print("Training completed!")

```

Evaluate both models on the test horizon

```

1 # ---- Evaluate both models on the test horizon ----
2 rnn_model.eval()
3 lstm_model.eval()
4
5 with torch.no_grad(): # inference only - no gradients
6     x_test_t = torch.tensor(X_test, dtype=torch.float32).to(device)
7     pred_rnn_scaled = rnn_model(x_test_t).cpu().numpy() # normalized predictions
8     pred_lstm_scaled = lstm_model(x_test_t).cpu().numpy()
9
10 # ---- Invert scaling to get real passenger counts ----
11 y_test_true = scaler.inverse_transform(y_test)
12 pred_rnn = scaler.inverse_transform(pred_rnn_scaled)
13 pred_lstm = scaler.inverse_transform(pred_lstm_scaled)
14
15 # ---- Compute error metrics in the original unit (passengers) ----
16 rmse_rnn = root_mean_squared_error(y_test_true, pred_rnn)
17 mae_rnn = mean_absolute_error(y_test_true, pred_rnn)
18
19 rmse_lstm = root_mean_squared_error(y_test_true, pred_lstm)
20 mae_lstm = mean_absolute_error(y_test_true, pred_lstm)
21
22 # ---- Summary table (lower errors are better) ----
23 eval_df = pd.DataFrame([
24     {"Model": "Vanilla RNN", "RMSE (Passengers)": rmse_rnn, "MAE (Passengers)":
25     ↪ mae_rnn},
26     {"Model": "LSTM Network", "RMSE (Passengers)": rmse_lstm, "MAE (Passengers)":
27     ↪ mae_lstm}
28 ]).set_index("Model")
29
30 print("Time Series Forecasting Performance Comparison:")
31 display(eval_df.style.highlight_min(subset=['RMSE (Passengers)', 'MAE
32     ↪ (Passengers)'], color='lightgreen'))
33
34 # ---- Plot ground truth vs. both forecast trajectories ----
35 # Map each test target back to its true position on the month axis:
36 # sequence i predicts month (i + SEQ_LENGTH)
37 test_indices = range(train_size + SEQ_LENGTH, len(passengers))
38
39 plt.figure(figsize=(12, 6))
40 # Full historical series in the background
41 plt.plot(range(len(passengers)), passengers, label='Ground Truth Historical',
42     ↪ color='black', alpha=0.5, lw=1.5)
43
44 # Test-period ground truth
45 plt.plot(test_indices, y_test_true, label='Ground Truth (Test Horizon)',
46     ↪ color='black', lw=2.5)
47
48 # Model forecasts

```

```

44 plt.plot(test_indices, pred_rnn, label=f'Vanilla RNN (RMSE: {rmse_rnn:.1f})',
    ↪ color='crimson', lw=2, linestyle='--')
45 plt.plot(test_indices, pred_lstm, label=f'LSTM Network (RMSE: {rmse_lstm:.1f})',
    ↪ color='teal', lw=2.2)
46
47 # Vertical line marking where the train period ends and test begins
48 plt.axvline(x=train_size + SEQ_LENGTH, color='gray', linestyle=':', label='Train /
    ↪ Test Cutoff')
49 plt.title("Airline Passengers Sequence Forecasting: Vanilla RNN vs LSTM",
    ↪ fontsize=14, fontweight='bold')
50 plt.xlabel("Month Index", fontsize=12)
51 plt.ylabel("Number of Passengers", fontsize=12)
52 plt.legend()
53 plt.tight_layout()
54 plt.show()

```

Sequence classification benchmark (manual experiment)

```

1 # ---- Sequence classification benchmark (manual experiment) ----
2 rng = np.random.default_rng(42)
3 X_cls = rng.normal(size=(1200, 20, 1)).astype("float32")
4 y_cls = (X_cls.sum(axis=1).ravel() > 0).astype("float32")
5
6 X_tr_c, X_te_c = X_cls[:1000], X_cls[1000:]
7 y_tr_c, y_te_c = y_cls[:1000], y_cls[1000:]
8 print(f"Sequence classification data: train {X_tr_c.shape}, test {X_te_c.shape}")
9
10 class RNNClassifier(nn.Module):
11     """Vanilla RNN or LSTM classifier over a 20-step sequence."""
12     def __init__(self, cell='rnn', hidden=32):
13         super().__init__()
14         self.rnn = nn.RNN(1, hidden, batch_first=True) if cell == 'rnn' else
    ↪ nn.LSTM(1, hidden, batch_first=True)
15         self.fc = nn.Linear(hidden, 1)
16
17     def forward(self, x):
18         out, _ = self.rnn(x)
19         return self.fc(out[:, -1, :]).squeeze(-1)
20
21 def train_classifier(model, epochs=10):
22     """Train a sequence classifier and return test accuracy."""
23     model = model.to(device)
24     optimizer = optim.Adam(model.parameters(), lr=0.005)
25     loss_fn = nn.BCEWithLogitsLoss()
26     loader = DataLoader(TensorDataset(torch.tensor(X_tr_c), torch.tensor(y_tr_c)),
    ↪ batch_size=32, shuffle=True)
27     for epoch in range(epochs):
28         model.train()
29         for xb, yb in loader:

```

```

30         xb, yb = xb.to(device), yb.to(device)
31         optimizer.zero_grad()
32         loss = loss_fn(model(xb), yb)
33         loss.backward()
34         optimizer.step()
35     model.eval()
36     with torch.no_grad():
37         logits = model(torch.tensor(X_te_c).to(device))
38         preds = (torch.sigmoid(logits) > 0.5).cpu().numpy().astype(int)
39     return accuracy_score(y_te_c.astype(int), preds)
40
41 acc_rnn_cls = train_classifier(RNNClassifier('rnn'))
42 acc_lstm_cls = train_classifier(RNNClassifier('lstm'))
43 print(f"Sequence classification test accuracy: Vanilla RNN {acc_rnn_cls:.4f} | LSTM
    ↳ {acc_lstm_cls:.4f}")

```

Evaluation / Expected Output

Table 14.1: Sequence classification on the synthetic benchmark (200 test sequences).

Model	Test accuracy
Vanilla RNN	0.9300
LSTM	0.9450

Table 14.2: Airline passenger forecasting on the chronological test horizon (27 windows).

Model	RMSE (passengers)	MAE
Vanilla RNN	69.93	62.36
LSTM	44.12	36.78

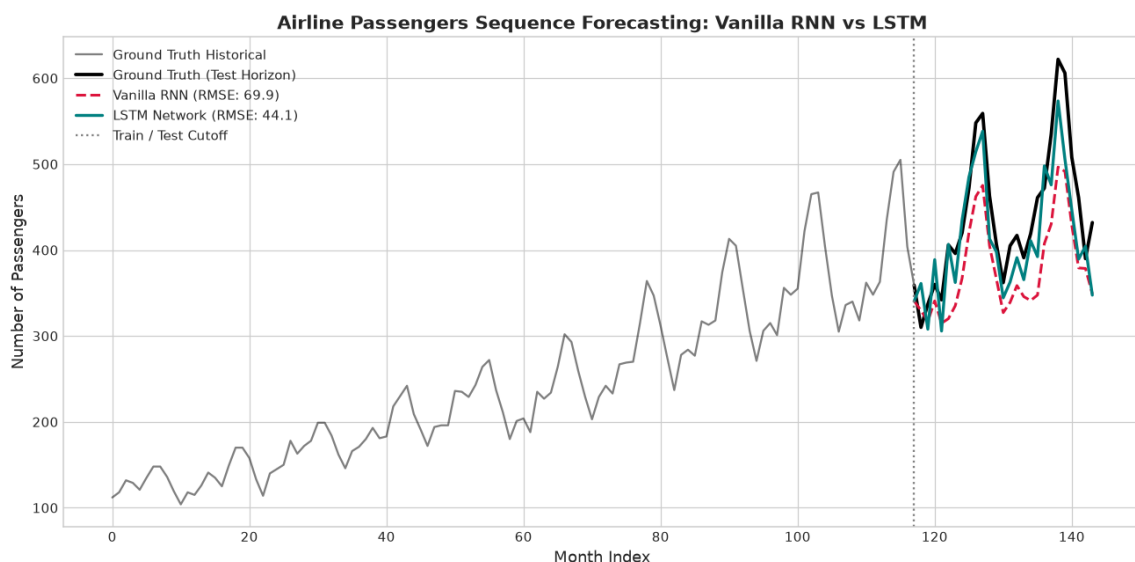


Figure 14.1: Airline passenger forecasting: ground truth versus the RNN and LSTM forecasts on the test horizon.

The LSTM outperforms the vanilla RNN on both tasks: +1.5 accuracy points on classification and about 37% lower RMSE on forecasting. The gap is larger on forecasting because that task needs 12-step seasonal

memory, exactly where gating prevents vanishing gradients. On the easy synthetic benchmark both models do well because the decision rule ($\text{sum} > 0$) is nearly linear.

Viva / Discussion Questions

1. Why does an RNN have memory?

Its hidden state is carried across time steps, so each output depends on previous inputs, not only the current one.

2. What is a hidden state?

The internal vector summarizing the sequence seen so far; it is updated at every step and used to produce outputs.

3. Why are LSTM/GRU often preferred for long dependencies?

Their gates control what is stored or forgotten and the additive cell state lets gradients flow across many steps, avoiding the exponential decay of vanilla RNN gradients.

4. Why is the train/test split chronological?

Shuffling sequences would leak future values into training and produce an optimistic, invalid evaluation.

5. How do the gates of an LSTM control information flow?

Forget, input and output gates multiply the cell and hidden states with values between 0 and 1, so the network learns what to keep, write and expose over long sequences.

Part VII

Unsupervised Neural Learning: Self-Organizing Maps

Experiments in this Part

Experiment 15 Self-Organizing Map (SOM)	71
---	----

Experiment 15

Self-Organizing Map (SOM)

Dataset: UCI Wine: 178 samples, 13 chemical features, 3 cultivars (labels used only for visualization)

Key parameters: 12×12 grid, Gaussian neighborhood, $\sigma = 1.5$, learning rate 0.5, PCA initialization, 5,000 iterations

Headline result: Quantization error 0.4447 $\rightarrow$ 0.1965; topographic error 0.0393

Objective

Use a Self-Organizing Map to map high-dimensional observations onto a low-dimensional grid while preserving neighborhood structure.

Suggested Dataset

UCI Wine: 178 samples, 13 chemical features, 3 cultivars (labels used only for visualization). Features are MinMax-scaled to $[0, 1]$.

Required Libraries

minisom, numpy, pandas, matplotlib.

Brief Theory

A Self-Organizing Map is a grid of neurons, each with a weight vector w_j in the input space. For each training sample the algorithm performs three steps:

1. **competition:** find the Best Matching Unit $c = \arg \min_j \|x - w_j\|$;
2. **cooperation:** compute the Gaussian neighborhood $h_{cj}(t) = \exp(-\|r_c - r_j\|^2 / (2\sigma(t)^2))$ on grid coordinates r ;
3. **adaptation:** move the weights toward the input, $w_j \leftarrow w_j + \alpha(t) h_{cj}(t) (x - w_j)$.

The learning rate $\alpha(t)$ and radius $\sigma(t)$ decay over time: early large neighborhoods organize the global layout, later small updates refine details. The result is a topology-preserving 2D projection. Quality is measured by the **quantization error** (average distance from samples to their BMU) and the **topographic error** (fraction of samples whose two best neurons are not neighbors). The **U-matrix** visualizes cluster boundaries as ridges of high inter-neuron distance.

Procedure

1. Load the wine data and MinMax-scale to $[0, 1]$.
2. Create a 12×12 SOM with a Gaussian neighborhood ($\sigma = 1.5$, learning rate 0.5) and initialize the weights with PCA.
3. Train for 5,000 random samples.
4. Compute the quantization and topographic errors.

5. Plot the U-matrix with each sample projected onto its BMU (marker = true cultivar).
6. Discuss grid size and σ .

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # ---- Dataset + scaling ----
8 from sklearn.datasets import load_wine
9 from sklearn.preprocessing import MinMaxScaler
10 # ---- MiniSom: lightweight Kohonen Self-Organizing Map implementation ----
11 from minisom import MiniSom
12
13 # Styling setup (consistent look across all notebooks)
14 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
15     ↪ plt.style.available else 'default')
16 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
17 plt.rcParams['figure.figsize'] = (10, 6)
18 plt.rcParams['figure.dpi'] = 110
19
20 np.random.seed(42)
21 print("MiniSom and libraries loaded successfully!")
22
23 # ---- Load the Wine recognition dataset ----
24 wine = load_wine()
25 X_raw = wine.data          # 178 samples x 13 chemical features
26 y = wine.target            # 3 wine cultivars (only used for visualization - SOM is
27     ↪ unsupervised)
28 feature_names = wine.feature_names
29 target_names = wine.target_names
30
31 print(f"Samples: {X_raw.shape[0]}, Features: {X_raw.shape[1]}, Classes:
32     ↪ {target_names.tolist()}")
33
34 # ---- MinMax scaling to [0, 1] ----
35 # SOM weight vectors live in the same range as the inputs, so scaling keeps
36     ↪ learning stable.
37 scaler = MinMaxScaler()
38 X = scaler.fit_transform(X_raw)

```

Configure the Kohonen grid

```

1 # ---- Configure the Kohonen grid ----
2 grid_x, grid_y = 12, 12 # 12x12 grid = 144 neurons (fewer than the 178 samples)
3 input_len = X.shape[1] # 13 chemical features per neuron weight vector
4
5 som = MiniSom(
6     x=grid_x,
7     y=grid_y,
8     input_len=input_len,
9     sigma=1.5, # initial neighborhood radius
10    learning_rate=0.5, # initial learning rate
11    neighborhood_function='gaussian', # smooth Gaussian neighborhood kernel
12    random_seed=42
13 )
14
15 # ---- Initialize weights with PCA for topologically faithful, faster convergence
16     ↪ ----
17 som.pca_weights_init(X)
18 print("Initial Quantization Error:", som.quantization_error(X))
19
20 # ---- Train: 5000 random samples presented to the map ----
21 som.train_random(data=X, num_iteration=5000, verbose=False)
22 final_qe = som.quantization_error(X) # average distance from samples to their BMU
23 final_te = som.topographic_error(X) # fraction of samples whose 2nd BMU is not a
24     ↪ grid neighbor
25
26 print(f"Training complete after 5,000 iterations!")
27 print(f" - Final Quantization Error (mean distance to BMU): {final_qe:.4f}")
28 print(f" - Final Topographic Error (proportion of foldings): {final_te:.4f}")

```

Plot the U-Matrix (unified distance map)

```

1 # ---- Plot the U-Matrix (unified distance map) ----
2 # Each cell = average distance between a neuron and its immediate grid neighbors.
3 # Dark valleys = dense clusters; bright ridges = boundaries between clusters.
4 plt.figure(figsize=(10, 8))
5 plt.pcolor(som.distance_map().T, cmap='bone_r', alpha=0.9)
6 cbar = plt.colorbar()
7 cbar.set_label("Normalized Inter-Neuron Distance (U-Matrix)", fontsize=11)
8
9 # ---- Project every sample onto its Best Matching Unit (BMU) ----
10 markers = ['o', 's', '^']
11 colors = ['red', 'green', 'blue']
12
13 for idx, x_vec in enumerate(X):
14     w = som.winner(x_vec) # grid coordinates of the best matching neuron for this
15     ↪ sample
16     plt.plot(
17         w[0] + 0.5, # +0.5 centers the marker inside the pcolor cell

```

```

17     w[1] + 0.5,
18     markers[y[idx]], # marker shape encodes the true cultivar
19     markerfacecolor='None',
20     markeredgecolor=colors[y[idx]],
21     markersize=10,
22     markeredgewidth=2
23 )
24
25 # Build a legend for the three cultivars
26 for c_idx, c_name in enumerate(target_names):
27     plt.plot([], [], marker=markers[c_idx], color=colors[c_idx], linestyle='None',
28             label=f'Cultivar {c_name}', markersize=9, markeredgewidth=2)
29
30 plt.title("SOM U-Matrix with Projected Wine Cultivars", fontsize=14,
31         ↪ fontweight='bold')
32 plt.legend(bbox_to_anchor=(1.25, 1), loc='upper left')
33 plt.tight_layout()
34 plt.show()

```

Evaluation / Expected Output

Table 15.1: SOM quality metrics on the Wine data.

Metric	Value
Initial quantization error	0.4447
Final quantization error	0.1965
Topographic error	0.0393 ($\approx 4\%$ foldings)

The map halved its quantization error during training and preserves topology well (topographic error below 0.05). On the U-matrix the three wine cultivars occupy contiguous regions separated by bright ridges, even though the SOM never saw the labels – evidence of unsupervised structure discovery. PCA initialization gave a well-unfolded map from the start.

Viva / Discussion Questions

1. SOM vs K-means?

Both compress data with prototypes, but K-means only finds centroids while SOM arranges them on a grid so neighboring neurons represent similar inputs (topology preservation).

2. What is a BMU?

The neuron whose weight vector is closest to the input – the winner of the competition.

3. Why is SOM called a topology-preserving map?

The neighborhood update ensures nearby grid positions end up with similar weight vectors, so high-dimensional neighborhoods are approximately preserved in 2D.

4. Why initialize with PCA?

Random initialization can create folded maps; PCA starts the map spread along the data's principal directions, giving faster and more faithful convergence.

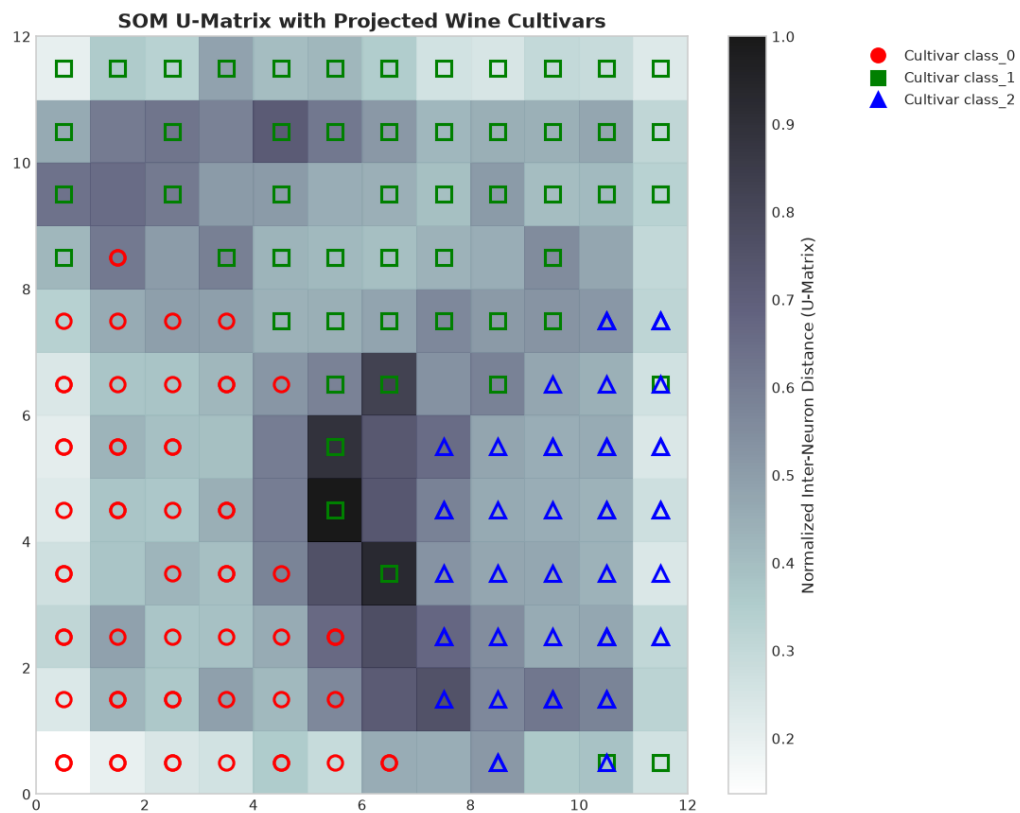


Figure 15.1: SOM U-matrix with the wine samples projected onto their BMUs; marker shape and color encode the true cultivar.

Part VIII

Probabilistic Sequence Modeling: Hidden Markov Models

Experiments in this Part

Experiment 16 Hidden Markov Model (HMM).	77
--	----

Experiment 16

Hidden Markov Model (HMM)

Dataset: DAX stock index 1991–1998 (data/stock_index.csv, real EuStockMarkets); 1,859 trading days

Key parameters: 3 Gaussian states, full covariance, 200 Baum–Welch (EM) iterations; Viterbi decoding

Headline result: EM converged; log-likelihood 6019.79; volatility-sorted Bull / Sideways / Bear regimes

Objective

Model a sequence using hidden states, transition probabilities and emission distributions; decode the most likely hidden-state path with the Viterbi algorithm.

Suggested Dataset

Real DAX stock index daily closes 1991–1998 (data/stock_index.csv, R EuStockMarkets), 1,859 trading days. The observation sequence is the daily return; the close price is used only for plotting.

Required Libraries

hmmlearn, numpy, pandas, matplotlib.

Brief Theory

A Hidden Markov Model is a doubly stochastic process:

- a hidden state sequence z_t following a first-order Markov chain, $P(z_t | z_{t-1}, \dots, z_1) = P(z_t | z_{t-1})$, summarized by the transition matrix $A_{ij} = P(z_{t+1} = s_j | z_t = s_i)$;
- observations emitted from the active state, $x_t | (z_t = s_k) \sim \mathcal{N}(\mu_k, \sigma_k^2)$ for a Gaussian HMM.

Three classical problems are solved here: **evaluation** (the forward algorithm gives $P(X | \lambda)$), **decoding** (the Viterbi algorithm finds the most likely state path by dynamic programming) and **learning** (Baum–Welch EM alternates a forward–backward E-step with parameter re-estimation until the log-likelihood converges).

Procedure

1. Load the DAX series and build the daily-return observation vector.
2. Fit a 3-state Gaussian HMM with full covariance and 200 EM iterations.
3. Check convergence and the model log-likelihood.
4. Decode the state path with Viterbi.
5. Sort the states by volatility so they map to Bull/Sideways/Bear; reorder the transition matrix accordingly.
6. Plot the transition heatmap and the price series colored by regime.

Reference Implementation

Core libraries

```

1 # ---- Core libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from hmmlearn.hmm import GaussianHMM
7
8 # Styling setup
9 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
    ↪ plt.style.available else 'default')
10 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
11 plt.rcParams['figure.figsize'] = (10, 6)
12 plt.rcParams['figure.dpi'] = 110
13 np.random.seed(42)
14
15 # ---- Load the real DAX stock index (daily closes 1991-1998) ----
16 df_market = pd.read_csv('../data/stock_index.csv')
17 dates = pd.to_datetime(df_market['Date'])
18 print(f"Trading days: {len(df_market)} | {dates.iloc[0].date()} to
    ↪ {dates.iloc[-1].date()}")
19 print(df_market.head(3).to_string(index=False))
20
21 returns = df_market['Daily_Return'].values.reshape(-1, 1) # HMM observations
22 prices = df_market['Close'].values # only used for
    ↪ visualization

```

Fit a 3-state Gaussian HMM with the Baum-Welch (EM) algorithm

```

1 # ---- Fit a 3-state Gaussian HMM with the Baum-Welch (EM) algorithm ----
2 n_components = 3
3 hmm_model = GaussianHMM(
4     n_components=n_components, # Bull / Sideways / Bear regimes
5     covariance_type="full", # each state has its own full covariance
    ↪ Gaussian emission
6     n_iter=200, # EM iterations
7     random_state=42,
8     verbose=False
9 )
10
11 hmm_model.fit(returns)
12 print("EM Training Converged:", hmm_model.monitor_.converged)
13 print(f"Model Log-Likelihood: {hmm_model.score(returns):.2f}")
14
15 # ---- Decode the most likely hidden state sequence with the Viterbi algorithm ----
16 hidden_states = hmm_model.predict(returns)
17

```

```

18 # ---- Sort regimes by volatility so labels have a consistent economic meaning ----
19 # Regime 0: Low Volatility (Bull) | Regime 1: Moderate (Sideways) | Regime 2: High
    ↳ (Bear/Crisis)
20 volatilities = [np.sqrt(hmm_model.covars_[i][0][0]) for i in range(n_components)]
21 state_order = np.argsort(volatilities) #
    ↳ ascending volatility
22 state_mapping = {old_state: new_state for new_state, old_state in
    ↳ enumerate(state_order)}
23
24 sorted_hidden_states = np.array([state_mapping[s] for s in hidden_states]) #
    ↳ relabel sequence
25 sorted_means = [hmm_model.means_[old][0] for old in state_order] #
    ↳ reordered means
26 sorted_stds = [np.sqrt(hmm_model.covars_[old][0][0]) for old in state_order] #
    ↳ reordered std devs
27
28 # ---- Reorder the transition matrix to match the sorted regimes ----
29 sorted_transmat = np.zeros((n_components, n_components))
30 for i in range(n_components):
31     for j in range(n_components):
32         sorted_transmat[i, j] = hmm_model.transmat_[state_order[i], state_order[j]]
33
34 regime_names = ["Bull (Low Vol)", "Sideways (Med Vol)", "Bear (High Vol)"]

```

Transition probability matrix

```

1 # ---- Transition probability matrix ----
2 fig, ax = plt.subplots(figsize=(6, 4.5))
3 sns.heatmap(sorted_transmat, annot=True, fmt='.3f', cmap='Blues', ax=ax,
4             xticklabels=regime_names, yticklabels=regime_names, cbar=False)
5 ax.set_title("HMM Transition Probability Matrix ($A_{ij}$)", fontweight='bold')
6 ax.set_xlabel("Transition To Regime")
7 ax.set_ylabel("Current Regime")
8 plt.tight_layout()
9 plt.show()
10
11 # ---- Viterbi-decoded regimes over the price series ----
12 colors = ['forestgreen', 'royalblue', 'crimson']
13 x = np.arange(len(prices))
14
15 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(13, 7), sharex=True,
16                               ↳ gridspec_kw={'height_ratios': [3, 1]})
17
18 # Top: color every consecutive price segment by its decoded regime
19 for i in range(len(prices) - 1):
20     ax1.plot([x[i], x[i + 1]], [prices[i], prices[i + 1]],
21             ↳ color=colors[sorted_hidden_states[i]], lw=1.2)
22 for i, name in enumerate(regime_names):
23     ax1.plot([], [], color=colors[i], label=name, lw=2.5)

```

```

22 ax1.set_title("DAX Index 1991-1998 Colored by Decoded HMM Regime (Viterbi Path)",
    ↪ fontsize=13, fontweight='bold')
23 ax1.set_ylabel("Index Level")
24 ax1.legend(loc='upper left')
25
26 # Bottom: raw sequence of decoded states over time
27 ax2.scatter(x, sorted_hidden_states, c=[colors[s] for s in sorted_hidden_states],
    ↪ s=8, marker='|')
28 ax2.set_yticks(range(n_components))
29 ax2.set_yticklabels(regime_names)
30 ax2.set_xlabel("Trading Day Index")
31
32 # Label the x-axis with years (real dates)
33 year_pos = np.where(dates.dt.year.diff().fillna(0).values != 0)[0]
34 ax2.set_xticks(year_pos)
35 ax2.set_xticklabels(dates.dt.year.iloc[year_pos].astype(str))
36
37 plt.tight_layout()
38 plt.show()

```

Evaluation / Expected Output

Table 16.1: Gaussian HMM on the DAX index (1,859 trading days, 1991–1998).

Item	Value
EM converged	True
Model log-likelihood	6019.79
Regimes	Bull (low vol), Sideways (medium), Bear (high vol)

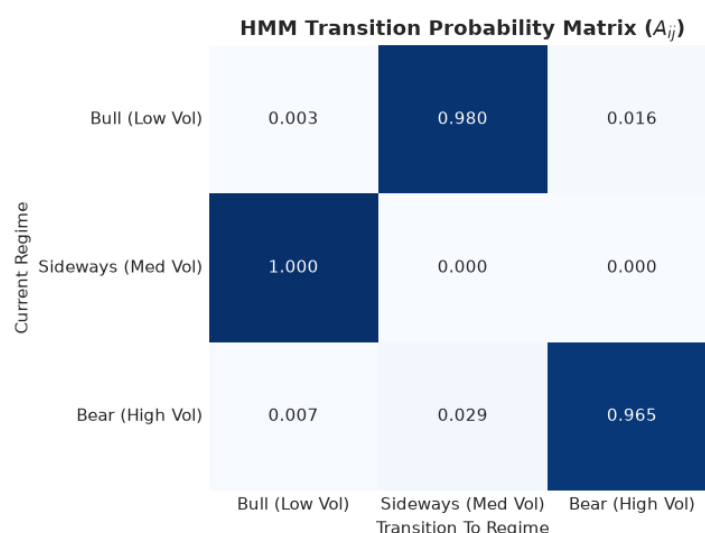


Figure 16.1: Estimated transition-probability matrix after sorting the states by volatility; the dominant diagonal indicates regime persistence.

The model extracts interpretable, persistent market regimes from real data without labels. The colored DAX chart tracks real episodes: the 1990s bull phases, corrections such as 1994, and the high-volatility cluster around the 1998 crisis. Its main assumptions are the Markov property and Gaussian emissions;

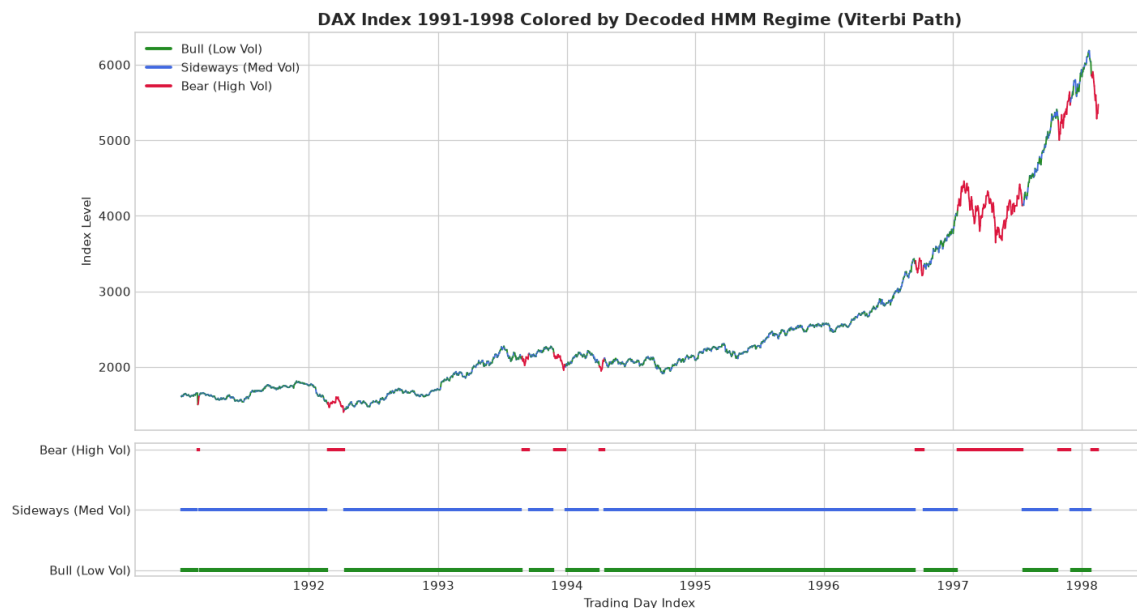


Figure 16.2: DAX index 1991–1998 colored by the Viterbi-decoded regime (top) and the decoded state sequence over time (bottom).

return fat tails and duration dependence motivate extensions such as t -emissions or semi-Markov models.

Viva / Discussion Questions

1. What is hidden in an HMM?

The state sequence: we observe emissions (returns) but not the underlying regime that generated them.

2. Viterbi vs forward algorithm?

Viterbi returns the single most likely state path (a maximum over paths); the forward algorithm computes the total probability $P(X | \lambda)$ by summing over all paths.

3. What is the Markov assumption?

The future state depends on the past only through the present state: $P(z_t | z_{1:t-1}) = P(z_t | z_{t-1})$.

4. Why sort the states by volatility?

So that regime labels (Bull/Sideways/Bear) are stable and interpretable across runs.

5. Why model returns and not prices?

Returns are approximately stationary; prices trend and violate the Gaussian emission assumption.

6. How do you choose the number of hidden states?

Compare candidate models by held-out likelihood (or BIC/AIC), check whether the decoded states are interpretable, and avoid states that merely fit noise.

Part IX

Margin-Based Learning: Support Vector Machines

Experiments in this Part

Experiment 17 Support Vector Machine (SVM)	83
--	----

Experiment 17

Support Vector Machine (SVM)

Dataset: Breast Cancer (569×30 ; 2 features for the kernel view, all 30 for accuracy) and Old Faithful geyser (data/geyser.csv, 272 eruptions)

Key parameters: SVC: RBF kernel, $C = 1.0$, $\gamma = \text{'scale'}$; SVR: RBF, $C = 10$, $\epsilon = 0.3$

Headline result: SVC accuracy 0.9790 (96/426 support vectors); SVR R^2 0.8968 (103/272 support vectors)

Objective

Train SVM classifiers and investigate the effects of kernel, C and γ ; demonstrate Support Vector Regression with an ϵ -insensitive tube.

Suggested Dataset

(a) Breast Cancer (569×30): two features (mean radius, mean texture) for the 2D kernel view and all 30 features for the accuracy benchmark. (b) Old Faithful geyser (data/geyser.csv, 272 eruptions) for SVR.

Required Libraries

scikit-learn, numpy, pandas, matplotlib.

Brief Theory

SVC (soft margin) solves

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The dual problem depends only on inner products $K(x_i, x_j)$, which enables the **kernel trick**: compute similarities in a high-dimensional feature space without mapping the data there explicitly. The solution is sparse — only points with $\alpha_i > 0$ (**support vectors**) define the boundary. Kernels used here: linear $x^T z$, polynomial $(\gamma x^T z + r)^d$ and RBF $\exp(-\gamma \|x - z\|^2)$. C penalizes margin violations; γ controls the RBF width. **SVR** applies the same machinery to regression with an ϵ -**insensitive tube**: residuals within $\pm \epsilon$ cost nothing and only points outside the tube become support vectors.

Procedure

1. Scale the features.
2. Train SVC with linear, polynomial and RBF kernels on two breast-cancer features; plot the boundaries, margins and support vectors.
3. Train the RBF SVC on all 30 features; report accuracy and the support-vector count.
4. Fit SVR (RBF, $C = 10$, $\epsilon = 0.3$) to the Old Faithful data (waiting time $\rightarrow$ eruption duration); plot the fitted curve and the tube.

- Report R^2 , RMSE and the support-vector share.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # ---- Dataset + model selection ----
8 from sklearn.datasets import load_breast_cancer
9 from sklearn.model_selection import train_test_split
10 from sklearn.preprocessing import StandardScaler
11 # ---- SVM models: classification and regression ----
12 from sklearn.svm import SVC, SVR
13 # ---- Evaluation metrics ----
14 from sklearn.metrics import r2_score, root_mean_squared_error
15
16 # Styling setup (consistent look across all notebooks)
17 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
18     ↪ plt.style.available else 'default')
19 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
20 plt.rcParams['figure.figsize'] = (10, 6)
21 plt.rcParams['figure.dpi'] = 110
22
23 np.random.seed(42)
24 print("SVM modules loaded successfully!")

```

2D real data for visualizing margins and kernels

```

1 # ---- 2D real data for visualizing margins and kernels ----
2 # Two diagnostic measurements from the Breast Cancer dataset (malignant vs benign).
3 cancer_2d = load_breast_cancer()
4 X_2d = cancer_2d.data[:, [0, 1]] # mean radius and mean texture
5 y_2d = cancer_2d.target
6 feature_names_2d = ['mean radius', 'mean texture']
7
8 scaler_2d = StandardScaler()
9 X_2d_scaled = scaler_2d.fit_transform(X_2d)
10
11 kernels = ['linear', 'poly', 'rbf']
12 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
13
14 # Coordinate grid used to draw the decision boundary as a contour
15 x_min, x_max = X_2d_scaled[:, 0].min() - 0.5, X_2d_scaled[:, 0].max() + 0.5
16 y_min, y_max = X_2d_scaled[:, 1].min() - 0.5, X_2d_scaled[:, 1].max() + 0.5
17 xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

```

```

18
19 for ax, k_name in zip(axes, kernels):
20     # Train one SVC per kernel on the same real data
21     clf = SVC(kernel=k_name, C=1.0, gamma='scale', degree=3, random_state=42)
22     clf.fit(X_2d_scaled, y_2d)
23
24     # Signed distance to the hyperplane for every grid point
25     Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
26
27     # Filled contour = decision regions; solid line = boundary (Z=0); dashed =
    ↪ margins (Z=+/-1)
28     ax.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 20),
    ↪ cmap='coolwarm', alpha=0.3)
29     ax.contour(xx, yy, Z, levels=[-1.0, 0.0, 1.0], linestyles=['--', '-', '--'],
    ↪ colors=['k', 'k', 'k'], linewidths=[1.5, 2.5, 1.5])
30
31     # Scatter the data points (colored by class)
32     ax.scatter(X_2d_scaled[:, 0], X_2d_scaled[:, 1], c=y_2d, cmap='coolwarm',
    ↪ edgecolors='k', s=25, alpha=0.8)
33
34     # Highlight the support vectors (the points that define the margin)
35     sv = clf.support_vectors_
36     ax.scatter(sv[:, 0], sv[:, 1], s=90, facecolors='none', edgecolors='gold',
    ↪ linewidths=2.0, label=f'SV ({len(sv)})')
37
38     ax.set_title(f"SVC with {k_name.capitalize()} Kernel\n(Acc:
    ↪ {clf.score(X_2d_scaled, y_2d):.3f}, SVs: {len(sv)})", fontweight='bold')
39     ax.set_xlabel(feature_names_2d[0])
40     ax.set_ylabel(feature_names_2d[1])
41     ax.legend(loc='lower right')
42
43 plt.suptitle("SVC Decision Boundaries on Real Breast Cancer Data (2 features)",
    ↪ fontsize=15, fontweight='bold')
44 plt.tight_layout()
45 plt.show()

```

SVC on a real dataset: Breast Cancer (RBF kernel)

```

1 # ---- SVC on a real dataset: Breast Cancer (RBF kernel) ----
2 cancer = load_breast_cancer()
3 X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
4     cancer.data, cancer.target, test_size=0.25, random_state=42,
    ↪ stratify=cancer.target
5 )
6 scaler_c = StandardScaler()
7 X_train_c_s = scaler_c.fit_transform(X_train_c)
8 X_test_c_s = scaler_c.transform(X_test_c)
9
10 svc_rbf = SVC(kernel='rbf', C=1.0, random_state=42)

```

```

11 svc_rbf.fit(X_train_c_s, y_train_c)
12 print(f"SVC (RBF) test accuracy on Breast Cancer: {svc_rbf.score(X_test_c_s,
    ↳ y_test_c):.4f}")
13 print(f"Support vectors used: {len(svc_rbf.support_)} of {len(X_train_c)} training
    ↳ samples")

```

Real 1D regression: Old Faithful geyser

```

1 # ---- Real 1D regression: Old Faithful geyser ----
2 # x = waiting time before an eruption (minutes), y = eruption duration (minutes)
3 geyser = pd.read_csv('../data/geyser.csv')
4 X_svr = geyser['waiting'].values.reshape(-1, 1)
5 y_svr = geyser['eruptions'].values
6 print(f"Old Faithful: {len(geyser)} eruptions | waiting {X_svr.min()}-{X_svr.max()}
    ↳ min | duration {y_svr.min()}-{y_svr.max()} min")
7
8 # Scale the feature (SVR is distance-based); the target stays in minutes for
    ↳ interpretability
9 scaler_svr = StandardScaler()
10 X_svr_scaled = scaler_svr.fit_transform(X_svr)
11
12 # epsilon defines the width of the "no penalty" tube around the prediction (in
    ↳ minutes)
13 epsilon_val = 0.3
14 svr_model = SVR(kernel='rbf', C=10.0, epsilon=epsilon_val, gamma='scale')
15 svr_model.fit(X_svr_scaled, y_svr)
16
17 # Dense grid for drawing a smooth prediction curve, converted back to minutes
18 X_grid = np.linspace(X_svr_scaled.min(), X_svr_scaled.max(), 300).reshape(-1, 1)
19 y_grid_pred = svr_model.predict(X_grid)
20 X_grid_orig = scaler_svr.inverse_transform(X_grid).ravel()
21
22 # Support vectors = training points that lie on/outside the epsilon tube
23 sv_indices = svr_model.support_
24 X_sv = X_svr[sv_indices]
25 y_sv = y_svr[sv_indices]
26
27 plt.figure(figsize=(11, 6))
28 # Training samples
29 plt.scatter(X_svr, y_svr, color='navy', s=35, alpha=0.75, label='Eruptions')
30
31 # Regression surface
32 plt.plot(X_grid_orig, y_grid_pred, color='crimson', lw=2.5, label='SVR Prediction
    ↳ $f(x)$')
33
34 # Epsilon-insensitive tube boundaries + shaded tube
35 plt.plot(X_grid_orig, y_grid_pred + epsilon_val, color='gray', linestyle='--',
    ↳ lw=1.5, label=f'Upper Tube ($+\epsilon$)')
36 plt.plot(X_grid_orig, y_grid_pred - epsilon_val, color='gray', linestyle='--',

```

```

    ↪ lw=1.5, label=rf'Lower Tube ($-\epsilon=\{\epsilon\_val\}$)')
37 plt.fill_between(X_grid_orig, y_grid_pred - epsilon_val, y_grid_pred + epsilon_val,
    ↪ color='gray', alpha=0.15)
38
39 # Support vectors are highlighted - only these points influence the fitted curve
40 plt.scatter(X_sv, y_sv, s=120, facecolors='none', edgecolors='gold',
    ↪ linewidths=2.5, label=f'Support Vectors ({len(X_sv)})')
41
42 plt.title(f"Support Vector Regression on Old Faithful (C=10, eps={epsilon_val})",
    ↪ fontsize=14, fontweight='bold')
43 plt.xlabel("Waiting time before eruption (minutes)", fontsize=12)
44 plt.ylabel("Eruption duration (minutes)", fontsize=12)
45 plt.legend(loc='lower right')
46 plt.tight_layout()
47 plt.show()
48
49 # ---- Quantitative evaluation ----
50 print(f"SVR R2 Score: {r2_score(y_svr, svr_model.predict(X_svr_scaled)):.4f}")
51 print(f"SVR RMSE:      {root_mean_squared_error(y_svr,
    ↪ svr_model.predict(X_svr_scaled)):.4f} minutes")
52 print(f"Support Vectors: {len(X_sv)} out of {len(X_svr)} points
    ↪ ({len(X_sv)/len(X_svr)*100:.1f}%)")

```

Evaluation / Expected Output

Table 17.1: SVM results on the two real datasets.

Experiment	Metric	Value
SVC (RBF, 30 features)	Test accuracy	0.9790
SVC (RBF, 30 features)	Support vectors	96 of 426 (22.5%)
SVR (Old Faithful)	R^2	0.8968
SVR (Old Faithful)	RMSE	0.366 minutes
SVR (Old Faithful)	Support vectors	103 of 272 (37.9%)

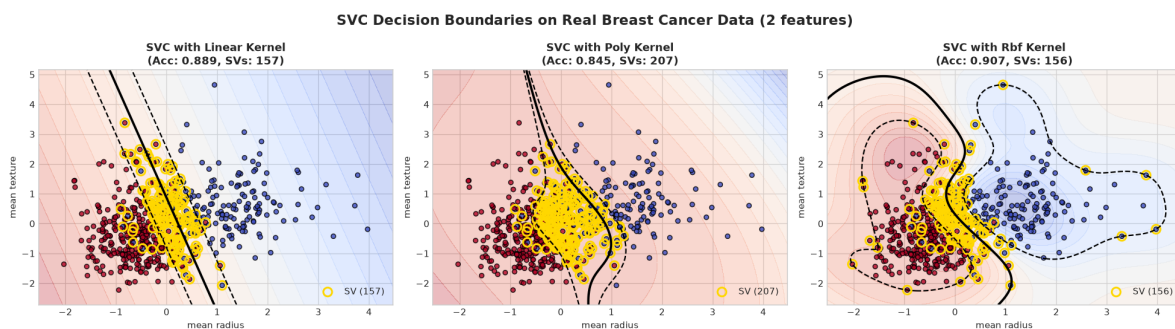


Figure 17.1: SVC decision boundaries, margins and support vectors on two real breast-cancer features for the linear, polynomial and RBF kernels.

The 2D kernel figure shows the linear kernel underfitting the real diagnostic data while the polynomial and RBF kernels produce curved boundaries. The RBF SVC reaches 0.9790 accuracy with only 96 of 426 training points as support vectors — about 77% of the data could be discarded without changing the classifier. The geyser scatter is bimodal (short and long eruptions); the RBF SVR smoothly bridges the

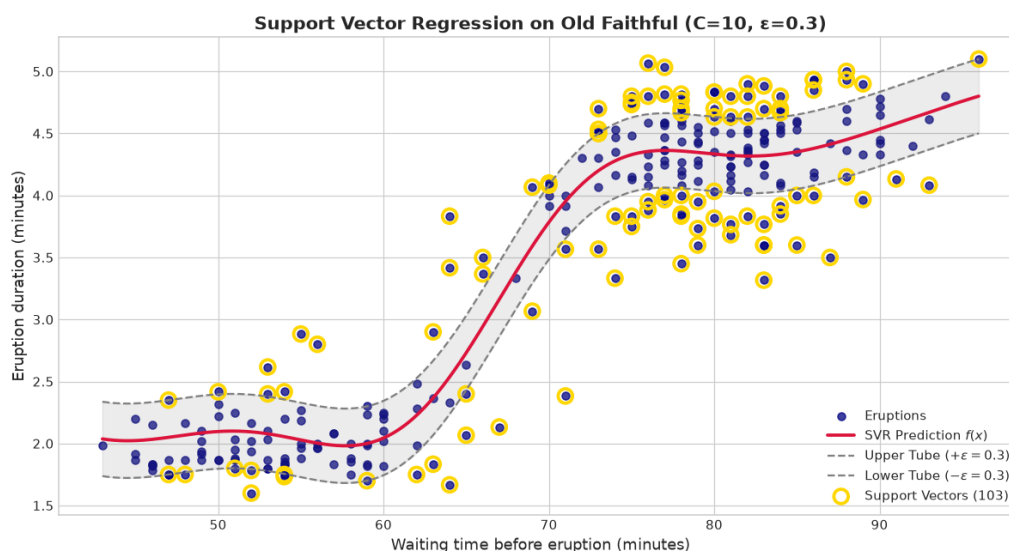


Figure 17.2: Support Vector Regression on the Old Faithful geyser with the $\varepsilon = 0.3$ tube; highlighted points are support vectors.

two modes and the tube leaves most residuals unpenalized.

Viva / Discussion Questions

1. What is a support vector?

A training point on or inside the margin ($\alpha > 0$) that alone defines the decision boundary.

2. Effect of C ?

Large C penalizes violations strongly (narrow margin, possible overfitting); small C allows a wider margin and more violations.

3. Effect of γ ?

Large γ makes each point's influence local (wiggly boundary); small γ makes the RBF smoother, approaching a linear model.

4. What is the kernel trick?

Computing inner products in a high-dimensional feature space without explicitly mapping the points there.

5. What is the ε -tube in SVR?

The zero-loss zone around the fitted function; only points outside it become support vectors. A wider tube (0.3 vs. 0.1) reduced the support-vector count with the same R^2 .

6. Why does SVM require careful scaling?

Margins and kernels use Euclidean distances; unscaled features with larger ranges dominate them and degrade the fit.

7. What is the difference between SVC and SVR?

SVC solves classification by maximizing the margin between classes; SVR solves regression by fitting a tube of width ε around the target and penalizing only points outside it.

Part X

Generative / Foundation Models: Large Language Models

Experiments in this Part

Experiment 18 Large Language Model (LLM) Experiment.	90
--	----

Experiment 18

Large Language Model (LLM) Experiment

Dataset: SMS Spam Collection (data/sms_spam.csv, 5,572 messages; balanced subset 400 train / 200 test) + DistilBERT / DistilGPT2

Key parameters: Fine-tune: fresh 2-class head, 2 epochs, AdamW learning rate 5×10^{-5} , batch 16, max length 64

Headline result: Held-out accuracy 0.9600; spam F1 0.9592; greedy vs. nucleus generation demonstrated

Objective

Demonstrate a reproducible LLM workflow for text classification and generation while understanding tokenization, inference and evaluation.

Suggested Dataset

DistilBERT (SST-2) for tokenization and sentiment, DistilGPT2 for generation, and the real SMS Spam Collection (data/sms_spam.csv) for fine-tuning. A balanced subset (300 ham + 300 spam) is split into 400 training and 200 stratified test messages.

Required Libraries

transformers, torch, pandas.

Brief Theory

Language models operate on **subword tokens** (Byte-Pair Encoding / WordPiece) mapped to integer vocabulary IDs; an **attention mask** marks real tokens versus padding. Self-attention computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{mask}\right)V,$$

so every token attends to every other token. Causal generation predicts one token at a time from $P(t_n | t_1, \dots, t_{n-1}) = \text{softmax}(z_n/T)$: **greedy** decoding takes the argmax each step, **temperature** T scales the logits, and **top- p** (nucleus) sampling restricts sampling to the smallest token set whose cumulative probability exceeds p . **Fine-tuning** reuses pretrained weights, replaces the classification head and trains briefly with a small learning rate.

Procedure

1. Tokenize a sentence and inspect the tokens, IDs and attention mask.
2. Run the sentiment pipeline on fixed test sentences.
3. Generate text with greedy decoding and with temperature/top- p sampling.
4. Load the SMS Spam Collection and build a balanced subset (300 ham + 300 spam) with a 400/200 stratified split.

5. Fine-tune DistilBERT (fresh 2-class head, AdamW learning rate 5×10^{-5} , 2 epochs, batch 16).
6. Evaluate accuracy and F1 on the held-out messages and inspect example predictions.

Reference Implementation

Core numerical and plotting libraries

```

1 # ---- Core numerical and plotting libraries ----
2 import os
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 # ---- PyTorch: used for the fine-tuning demo ----
9 import torch
10 import torch.nn as nn
11 from torch.utils.data import DataLoader, Dataset
12 # ---- Dataset splitting + metrics for the fine-tune evaluation ----
13 from sklearn.model_selection import train_test_split
14 from sklearn.metrics import accuracy_score, f1_score
15 # ---- Hugging Face Transformers: tokenizers, models, pipelines, generation configs
16   ↳ ----
17 from transformers import (
18     AutoTokenizer,
19     AutoModelForSequenceClassification,
20     AutoModelForCausalLM,
21     GenerationConfig,
22     pipeline
23 )
24
25 # Styling setup (consistent look across all notebooks)
26 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
27   ↳ plt.style.available else 'default')
28 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
29 plt.rcParams['figure.figsize'] = (10, 5)
30 plt.rcParams['figure.dpi'] = 110
31
32 torch.manual_seed(42)
33 torch.cuda.manual_seed_all(42) # reproducible GPU training
34 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # GPU if
35   ↳ available
36 print(f"PyTorch on {device}")

```

Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based)

```

1 # ---- Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based) ----
2 model_id = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
3 print(f>Loading Tokenizer: {model_id}...")

```

```

4 tokenizer = AutoTokenizer.from_pretrained(model_id)
5
6 sample_text = "Machine learning and deep neural networks are transforming modern
    ↪ science!"
7 # Tokenize with padding/truncation so the output has a fixed shape (here 20 tokens)
8 encoded = tokenizer(
9     sample_text,
10    padding="max_length",
11    max_length=20,
12    truncation=True,
13    return_tensors="pt"
14 )
15
16 # Convert integer IDs back to readable subword tokens
17 tokens = tokenizer.convert_ids_to_tokens(encoded['input_ids'][0])
18 print(f"Original Text: {sample_text}\n")
19 print(f"Subword Tokens ({len(tokens)}):\n", tokens)
20 print(f"Input IDs:      \n", encoded['input_ids'][0].tolist())
21 print(f"Attention Mask: \n", encoded['attention_mask'][0].tolist())

```

Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2)

```

1 # ---- Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2) ----
2 classifier = pipeline("sentiment-analysis", model=model_id, device=0 if
    ↪ torch.cuda.is_available() else -1)
3
4 test_sentences = [
5     "The new neural network model demonstrated extraordinary accuracy and
    ↪ remarkable speed.",
6     "The dataset was poorly labeled, noisy, and the model completely failed to
    ↪ converge.",
7     "The gradient descent algorithm performed reasonably well, meeting standard
    ↪ expectations.",
8     "A catastrophic failure occurred during training due to exploding gradient
    ↪ issues."
9 ]
10
11 # One forward pass per sentence -> label + calibrated confidence score
12 predictions = classifier(test_sentences)
13
14 # Collect results in a table for display
15 results_table = []
16 for sent, pred in zip(test_sentences, predictions):
17     results_table.append({
18         "Sentence": sent,
19         "Predicted Label": pred['label'],
20         "Confidence Score": f"{pred['score']*100:.2f}%"
21     })
22

```

```

23 df_preds = pd.DataFrame(results_table)
24 display(df_preds)

```

Load a small causal LM (DistilGPT2) for text generation

```

1 # ---- Load a small causal LM (DistilGPT2) for text generation ----
2 gen_model_id = "distilbert/distilgpt2"
3 print(f"Loading Text Generator: {gen_model_id}...")
4 # clean_up_tokenization_spaces=False keeps BPE spacing intact
5 generator = pipeline("text-generation", model=gen_model_id,
6                       device=0 if torch.cuda.is_available() else -1,
7                       ↪ clean_up_tokenization_spaces=False)
8
9
10 # 1. Greedy Search: always pick the most likely next token (deterministic)
11 greedy_output = generator(
12     prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=False)
13 )[0]['generated_text']
14
15 # 2. Temperature + Top-p (Nucleus) Sampling: sample from the most likely tokens up
16     ↪ to cumulative p
17 sampled_output = generator(
18     prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=True,
19     ↪ temperature=0.7, top_p=0.9)
20 )
21
22 print(f"PROMPT: '{prompt}'\n")
23 print(f"--- 1. GREEDY SEARCH OUTPUT ---\n{greedy_output.strip()}\n")
24 print(f"--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT
25     ↪ ---\n{sampld_output.strip()}")

```

Real dataset: SMS Spam Collection (5,572 labeled messages)

```

1 # ---- Real dataset: SMS Spam Collection (5,572 labeled messages) ----
2 sms = pd.read_csv('../data/sms_spam.csv')
3 sms['y'] = (sms['label'] == 'spam').astype(int)
4 print(f"SMS Spam Collection: {len(sms)} messages | ham {(sms.y == 0).sum()}, spam
5     ↪ {(sms.y == 1).sum()}")
6
7 # Balanced subset for a fast fine-tune demo: 300 ham + 300 spam
8 ham = sms[sms.y == 0].sample(n=300, random_state=42)
9 spam = sms[sms.y == 1].sample(n=300, random_state=42)
10 subset = pd.concat([ham, spam]).sample(frac=1,
11     ↪ random_state=42).reset_index(drop=True)
12
13 # Stratified split keeps the ham/spam ratio identical in train and test
14 train_df, test_df = train_test_split(subset, test_size=200, random_state=42,
15     ↪ stratify=subset.y)

```

```

13 print(f"Train messages: {len(train_df)} | Test messages: {len(test_df)} | spam
    ↳ share: {subset.y.mean():.1%}")
14
15 class SMSDataset(Dataset):
16     """Tokenize SMS texts and expose them as PyTorch samples."""
17     def __init__(self, texts, labels, tokenizer):
18         self.encodings = tokenizer(list(texts), padding=True, truncation=True,
    ↳ max_length=64, return_tensors="pt")
19         self.labels = list(labels)
20
21     def __getitem__(self, idx):
22         item = {key: val[idx] for key, val in self.encodings.items()}
23         item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
24         return item
25
26     def __len__(self):
27         return len(self.labels)
28
29 train_loader = DataLoader(SMSDataset(train_df.text, train_df.y, tokenizer),
    ↳ batch_size=16, shuffle=True)
30 test_loader = DataLoader(SMSDataset(test_df.text, test_df.y, tokenizer),
    ↳ batch_size=32)
31
32 # ---- Load the pre-trained classifier with a fresh 2-class head ----
33 # ignore_mismatched_sizes allows replacing the original SST-2 head with a new one.
34 ft_model = AutoModelForSequenceClassification.from_pretrained(model_id,
    ↳ num_labels=2, ignore_mismatched_sizes=True)
35 ft_model.to(device)
36 optimizer = torch.optim.AdamW(ft_model.parameters(), lr=5e-5) # small LR for
    ↳ fine-tuning
37
38 print("Pre-trained Transformer loaded for fine-tuning!")

```

Fine-tuning loop (2 epochs on 400 real SMS messages)

```

1 # ---- Fine-tuning loop (2 epochs on 400 real SMS messages) ----
2 epochs = 2
3 ft_model.train()
4 epoch_losses = []
5
6 for epoch in range(epochs):
7     running_loss = 0.0
8     for batch in train_loader:
9         optimizer.zero_grad()
10
11         # Move the batch to the selected device
12         input_ids = batch['input_ids'].to(device)
13         attention_mask = batch['attention_mask'].to(device)
14         labels = batch['labels'].to(device)

```

```

15
16     # Forward pass: the model computes the classification loss internally
17     outputs = ft_model(input_ids=input_ids, attention_mask=attention_mask,
    ↪ labels=labels)
18     loss = outputs.loss
19     loss.backward()      # backpropagate through the whole transformer
20     optimizer.step()     # update weights
21
22     running_loss += loss.item() * len(input_ids)    # de-average the batch loss
23
24     avg_loss = running_loss / len(train_df)
25     epoch_losses.append(avg_loss)
26     print(f"Epoch [{epoch+1}/{epochs}] Fine-Tuning Loss: {avg_loss:.4f}")

```

Evaluate the fine-tuned model on held-out real SMS messages

```

1  # ---- Evaluate the fine-tuned model on held-out real SMS messages ----
2  ft_model.eval()
3  preds, targets = [], []
4
5  with torch.no_grad():
6      for batch in test_loader:
7          # Move the batch to the same device as the fine-tuned model (GPU when
    ↪ available)
8          input_ids = batch['input_ids'].to(device)
9          attention_mask = batch['attention_mask'].to(device)
10         logits = ft_model(input_ids=input_ids, attention_mask=attention_mask).logits
11         preds += logits.argmax(-1).tolist()      # predicted class (0 = ham, 1 =
    ↪ spam)
12         targets += batch['labels'].tolist()
13
14     print(f"Held-out test accuracy: {accuracy_score(targets, preds):.4f}")
15     print(f"Held-out test F1 (spam): {f1_score(targets, preds):.4f}")
16
17  # ---- Show a few individual predictions with confidence ----
18  labels_map = {0: "ham", 1: "spam"}
19  with torch.no_grad():
20      for i in range(3):
21          message = test_df.text.iloc[i]
22          enc = tokenizer(message, truncation=True, max_length=64,
    ↪ return_tensors="pt").to(device)
23          probs = torch.softmax(ft_model(**enc).logits, dim=1).cpu().numpy()[0]
24          print(f"\nMessage: {message[:75]}...")
25          print(f"True: {labels_map[test_df.y.iloc[i]]} | Predicted:
    ↪ {labels_map[probs.argmax()]} ({probs.max()*100:.1f}%)")

```

Evaluation / Expected Output

Example predictions (all correct): a prize-notification spam at 99.4% confidence, a short ham message at 99.5%, and another spam at 89.7%. Generation: greedy decoding produced the safe deterministic

Table 18.1: DistilBERT fine-tuned on the SMS Spam Collection (200 held-out messages).

Item	Value
Fine-tune loss (epoch 1 $\rightarrow$ 2)	0.705 $\rightarrow$ 0.079
Held-out accuracy	0.9600
Held-out F1 (spam)	0.9592

continuation “*the way we think about the world*”, while nucleus sampling ($T = 0.7$, $p = 0.9$) produced a different but coherent sentence — the effect of temperature and top- p .

The experiment demonstrates the complete practical pipeline and shows that fine-tuning a small pretrained model on only 400 real messages already gives 96% spam detection. Limitations: the balanced subset ignores the natural 13% spam prior of the full collection, the models are small and can hallucinate, and prompts/data must be fixed for reproducibility.

Viva / Discussion Questions

1. What is tokenization?

Splitting text into subword units that map to integer vocabulary IDs, letting a fixed vocabulary represent any word; special tokens and attention masks handle sequence framing and padding.

2. Pretraining vs fine-tuning vs prompting?

Pretraining learns general language representation from massive corpora; fine-tuning adapts those weights to a task with labeled data; prompting steers a frozen model through instructions or examples.

3. Why is LLM output not automatically ground truth?

The model generates the most plausible continuation, not verified facts; it can hallucinate, and confidence scores are not calibrated correctness probabilities.

4. How can prompt wording affect results?

Meaning, tone, format and label wording all change the conditional distribution the model samples from, so prompts must be fixed and reported.

5. Why must the test messages be separated before fine-tuning?

Fine-tuning on data that later serves as the test set leaks the evaluation targets into training; the held-out split must be created first so the reported accuracy stays honest.

Part XI

Non-Parametric Regression: Generalized Regression Neural Network

Experiments in this Part

Experiment 19 Generalized Regression Neural Network (GRNN)	98
--	----

Experiment 19

Generalized Regression Neural Network (GRNN)

Dataset: Real motorcycle impact data (data/motorcycle.csv, R MASS: :mcycle); 133 observations, time vs head acceleration

Key parameters: GRNN from scratch (NumPy); smoothing spread σ selected by 5-fold cross-validation over 0.02–1.0

Headline result: $\sigma = 0.08$ (CV RMSE 25.00 g); held-out R^2 0.7255, RMSE 22.78 g

Objective

Implement GRNN-style kernel regression from scratch and study the effect of the smoothing parameter σ with cross-validation.

Suggested Dataset

Real motorcycle impact data (data/motorcycle.csv, R MASS: :mcycle): 133 observations of time after impact (ms) versus head acceleration (g). Time is standardized before computing kernel distances.

Required Libraries

numpy, pandas, matplotlib, scikit-learn (base classes, KFold, metrics).

Brief Theory

A GRNN has four layers: input, pattern, summation and output. With N training points, each pattern unit computes a Gaussian activation

$$p_i(x) = \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right),$$

the summation layer accumulates $S(x) = \sum_i y_i p_i(x)$ and $D(x) = \sum_i p_i(x)$, and the output layer returns the Nadaraya–Watson conditional mean

$$\hat{y}(x) = \frac{S(x)}{D(x)} = \frac{\sum_i y_i \exp(-\|x - x_i\|^2/2\sigma^2)}{\sum_i \exp(-\|x - x_i\|^2/2\sigma^2)}.$$

Training is one-pass — the model simply stores the data; prediction costs $O(N)$ per query. The only hyperparameter is the smoothing spread σ : small values memorize noise (high variance), large values over-smooth (high bias).

Procedure

1. Load the motorcycle data and standardize the time feature.
2. Implement the GRNN class (fit stores the data; predict computes the kernel weights).
3. Visualize $\sigma = 0.05, 0.25$ and 1.0 (under- and over-smoothing).

4. Select σ by 5-fold cross-validation over 0.02 ... 1.0.
5. Refit on a 75% split and evaluate the 25% held-out set (R^2 , RMSE).

Reference Implementation

Core libraries

```

1 # ---- Core libraries ----
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.base import BaseEstimator, RegressorMixin
8 from sklearn.model_selection import KFold, train_test_split
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import r2_score, root_mean_squared_error
11
12 # Styling setup
13 plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
14             ↪ plt.style.available else 'default')
15 plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
16 plt.rcParams['figure.figsize'] = (10, 6)
17 plt.rcParams['figure.dpi'] = 110
18 np.random.seed(42)

```

GRNN class (from scratch)

```

1 class GRNN(BaseEstimator, RegressorMixin):
2     """
3     Generalized Regression Neural Network (Specht, 1991) - vectorized
4     ↪ implementation.
5
6     Prediction is the Nadaraya-Watson kernel regression estimate:
7         y_hat(x) = sum_i y_i * exp(-||x - x_i||^2 / (2*sigma^2))
8                     / sum_i      exp(-||x - x_i||^2 / (2*sigma^2))
9
10    Training is one-pass: the model simply stores the training set (no gradient
11    ↪ descent).
12    The only hyperparameter is the smoothing spread sigma.
13    """
14
15    def __init__(self, sigma=1.0):
16        self.sigma = sigma
17
18    def fit(self, X, y):
19        # "Training" = memorizing the data (pattern + summation layers)
20        self.X_train_ = np.asarray(X, dtype=np.float64)
21        self.y_train_ = np.asarray(y, dtype=np.float64).ravel()

```

```

20         return self
21
22     def predict(self, X):
23         X_test = np.asarray(X, dtype=np.float64)
24
25         # ---- Pattern layer: squared Euclidean distance between each test and
        ↪ train point ----
26         # Uses the expansion ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b for efficiency.
27         dist_sq = np.sum(X_test**2, axis=1, keepdims=True) + \
28             np.sum(self.X_train**2, axis=1, keepdims=True).T - \
29             2 * np.dot(X_test, self.X_train.T)
30         dist_sq = np.maximum(dist_sq, 0.0) # guard against tiny negative values
        ↪ from rounding
31
32         # ---- Pattern layer activation: Gaussian kernel of each training point ----
33         kernels = np.exp(-dist_sq / (2.0 * (self.sigma ** 2)))
34
35         # ---- Summation layer: D-sum (denominator) and S-sum (numerator) ----
36         D = np.sum(kernels, axis=1) # unweighted sum of activations
37         S = np.dot(kernels, self.y_train_) # activation-weighted sum of
        ↪ targets
38
39         # ---- Output layer: normalized weighted average (prevents 0/0 for isolated
        ↪ queries) ----
40         D_safe = np.where(D < 1e-12, 1e-12, D)
41         return S / D_safe
42
43
44     print("Custom vectorized GRNN class defined!")

```

Load the real motorcycle impact data (MASS::mcycle)

```

1 # ---- Load the real motorcycle impact data (MASS::mcycle) ----
2 moto = pd.read_csv('../data/motorcycle.csv')
3 X = StandardScaler().fit_transform(moto['times'].values.reshape(-1, 1)) # time
        ↪ after impact (scaled)
4 y = moto['accel'].values # head
        ↪ acceleration (g)
5 print(f"Motorcycle data: {len(moto)} observations | time
        ↪ {moto.times.min()}-{moto.times.max()} ms | accel {moto.accel.min():.0f} to
        ↪ {moto.accel.max():.0f} g")
6
7 X_eval = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)
8
9 # Under-smoothing, near-optimal, over-smoothing
10 sigmas = [0.05, 0.25, 1.0]
11 colors = ['purple', 'crimson', 'teal']
12
13 fig, axes = plt.subplots(1, 3, figsize=(18, 5))

```

```

14
15 for ax, sig, col in zip(axes, sigmas, colors):
16     # Fit a fresh GRNN for each sigma (training is instant)
17     grnn_demo = GRNN(sigma=sig).fit(X, y)
18     y_pred_demo = grnn_demo.predict(X_eval)
19
20     ax.scatter(X, y, color='navy', s=25, alpha=0.6, label='Observations')
21     ax.plot(X_eval, y_pred_demo, color=col, lw=2.5, label=rf'GRNN ($\sigma={sig}$)')
22
23     # Training R2 shows the bias-variance behavior for each sigma
24     r2 = r2_score(y, grnn_demo.predict(X))
25     ax.set_title(rf"Smoothing $\sigma = {sig}$ (Train $R^2 = {r2:.3f}$)",
    ↪ fontweight='bold')
26     ax.set_xlabel("Time after impact (standardized)")
27     ax.set_ylabel("Head acceleration (g)")
28     ax.legend(loc='lower right')
29
30 plt.suptitle("GRNN Behavior on Real Motorcycle Impact Data (sigma sweep)",
    ↪ fontsize=15, fontweight='bold')
31 plt.tight_layout()
32 plt.show()

```

5-fold CV grid search for the optimal smoothing sigma

```

1 # ---- 5-fold CV grid search for the optimal smoothing sigma ----
2 sigma_candidates = np.linspace(0.02, 1.0, 50)
3 cv_scores = []
4 kf = KFold(n_splits=5, shuffle=True, random_state=42) # fixed folds for a fair
    ↪ comparison
5
6 for sig in sigma_candidates:
7     fold_rmsees = []
8     for train_idx, val_idx in kf.split(X):
9         # Split the data into this fold's train/validation parts
10         X_tr, y_tr = X[train_idx], y[train_idx]
11         X_va, y_va = X[val_idx], y[val_idx]
12
13         # Fit GRNN on the training fold and score on the validation fold
14         model = GRNN(sigma=sig).fit(X_tr, y_tr)
15         preds = model.predict(X_va)
16         fold_rmsees.append(root_mean_squared_error(y_va, preds))
17
18     cv_scores.append(np.mean(fold_rmsees)) # average CV error for this sigma
19
20 best_idx = np.argmin(cv_scores)
21 best_sigma = sigma_candidates[best_idx]
22 print(f"Optimal Smoothing Parameter sigma: {best_sigma:.3f} (CV RMSE:
    ↪ {cv_scores[best_idx]:.4f} g)")
23

```

```

24 # ---- Plot the CV error curve and mark the optimum ----
25 plt.figure(figsize=(9, 4.5))
26 plt.plot(sigma_candidates, cv_scores, 'b-', lw=2.2)
27 plt.axvline(x=best_sigma, color='crimson', linestyle='--', label=f'Optimal sigma =
    ↳ {best_sigma:.3f}')
28 plt.title("5-Fold Cross-Validation Error vs Smoothing Parameter (sigma)",
    ↳ fontsize=13, fontweight='bold')
29 plt.xlabel("Smoothing Parameter (sigma)", fontsize=11)
30 plt.ylabel("Cross-Validation RMSE (g)", fontsize=11)
31 plt.legend()
32 plt.tight_layout()
33 plt.show()

```

Final model: refit with the CV-selected sigma and evaluate on a held-out split

```

1 # ---- Final model: refit with the CV-selected sigma and evaluate on a held-out
    ↳ split ----
2 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42)
3 final_model = GRNN(sigma=best_sigma).fit(X_tr, y_tr)
4 y_hat = final_model.predict(X_te)
5
6 print(f"Final GRNN with CV-selected sigma = {best_sigma:.3f}")
7 print(f" - Train R2: {r2_score(y_tr, final_model.predict(X_tr)):.4f}")
8 print(f" - Test R2: {r2_score(y_te, y_hat):.4f}")
9 print(f" - Test RMSE: {root_mean_squared_error(y_te, y_hat):.4f} g")

```

Evaluation / Expected Output

Table 19.1: GRNN on the real motorcycle impact data (133 observations).

Item	Value
CV-selected σ	0.080 (CV RMSE 25.00 g)
Train R^2	0.8073
Test R^2	0.7255
Test RMSE	22.78 g (acceleration range $-134 \dots +75$ g)

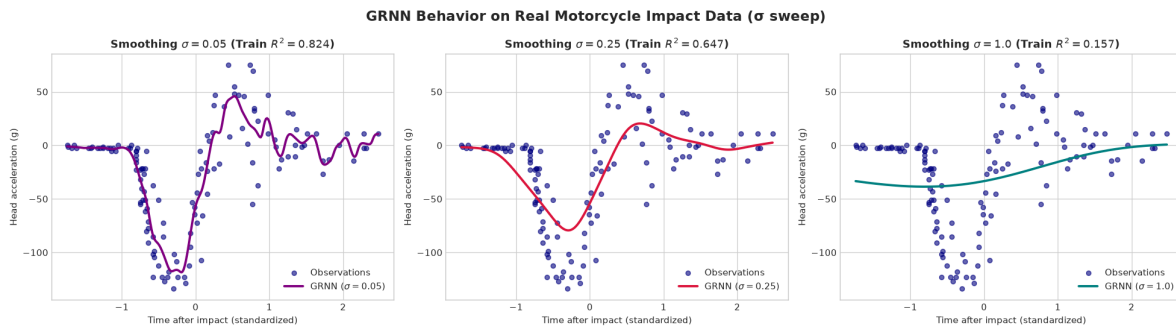


Figure 19.1: GRNN under different smoothing spreads on the motorcycle data: small σ follows noise, large σ flattens the impact peak.

$\sigma = 0.05$ shows local wiggles (variance), while $\sigma = 0.25\text{--}1.0$ underfits the sharp deceleration peak. On the smooth synthetic benchmark a GRNN reaches $R^2 \approx 0.95$; on this real dataset the held-out R^2 is

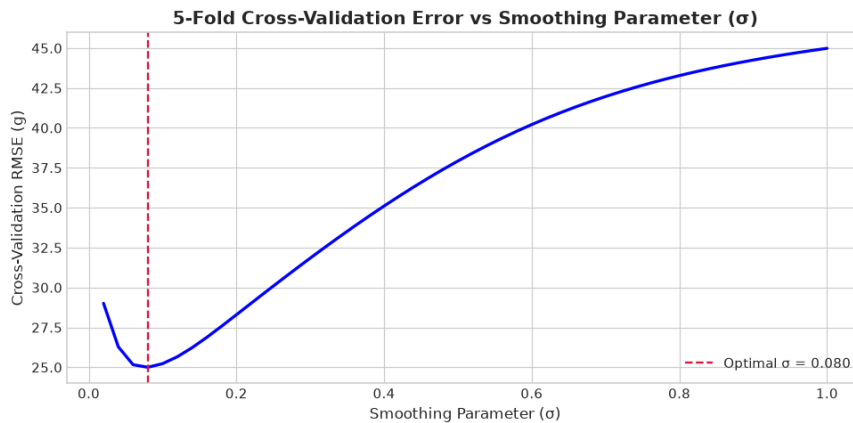


Figure 19.2: Five-fold cross-validation error versus σ ; the clear minimum selects $\sigma = 0.08$.

0.73 because a single global σ cannot be simultaneously small at the impact discontinuity and large elsewhere. This is the classic bias–variance limitation of global-bandwidth kernel regression; adaptive bandwidths or local polynomial regression would improve it.

Viva / Discussion Questions

1. Why is GRNN called non-parametric?

It does not learn a fixed set of parameters; the training data itself forms the model (memory-based kernel regression).

2. What does σ control?

The smoothing bandwidth: small σ fits noise (high variance), large σ over-smooths (high bias).

3. How is GRNN related to kernel regression?

It is exactly the Nadaraya–Watson Gaussian kernel regression estimator of $\mathbb{E}[y \mid x]$, with one kernel per training observation.

4. Why is the test R^2 lower than on synthetic data?

The motorcycle data contains a near-discontinuity and an uneven measurement design, which a single global bandwidth cannot capture.

5. How would you improve the fit?

Use an adaptive or per-location bandwidth, local polynomial (LOESS) regression, or feature scaling plus anisotropic kernels.

6. How does GRNN scale with the size of the training set?

It stores all training points and evaluates a kernel for each prediction, so both memory and prediction time grow linearly with the training set.

Part XII

Integrated Mini Project and Appendices

Experiments in this Part

Integrated Mini Project: Comparative Machine Learning Study	105
Appendix A: Quick Reference Formulas	107
Appendix B: Reproducibility Checklist	107
Appendix C: Suggested Assessment Rubric	108
Appendix D: Student Experiment Record	108

Integrated Mini Project: Comparative Machine Learning Study

Scope: Compare algorithms from different families on identical data, splits and seeds

Protocol: Fixed random seeds; untouched test sets; two or more metrics per task

Headline result: Gradient boosting leads tabular regression; ensembles tie on small classification data; density clustering finds real seismic zones without labels

Objective

For the final laboratory assessment, compare at least three algorithms from different families on one real-world dataset, with a controlled, explainable experiment rather than a search for the largest score.

Suggested Dataset

The manual's own experiments provide the comparison material: Heart Disease (classification), California Housing (regression), Mall Customers (clustering) and the USGS earthquake catalogue (density-based clustering).

Required Libraries

scikit-learn, xgboost, catboost, numpy, pandas, matplotlib.

Workflow

1. Define a specific prediction or clustering problem.
2. Identify the target variable (supervised) or justify the absence of labels.
3. Inspect missing values, duplicates, class balance and feature types.
4. Create a train/test split before fitting preprocessing steps that learn from data.
5. Build a simple baseline.
6. Train at least three candidate algorithms from different families.
7. Use cross-validation on the training data for model selection where appropriate.
8. Evaluate once on the untouched test set.
9. Analyze errors, not only aggregate metrics.
10. Document limitations, reproducibility settings and possible future improvements.

Comparative Results

Classification — Heart Disease (same stratified 75/25 split, seed 42)

Regression — California Housing (5,000 sample, same 80/20 split)

Clustering — Mall Customers ($K = 5$, standardized)

Density-Based — USGS Earthquakes 2023 (7,638 events)

Table 19.2: Ensemble and margin-based classifiers on the same split (76 test samples).

Model	Accuracy	F1	ROC-AUC
Random Forest (RFC)	0.7763	0.8046	0.8704
XGBoost	0.7763	0.8132	0.8509
AdaBoost	0.7895	0.8182	0.8714
CatBoost	0.7632	0.7955	0.8551

Table 19.3: Ensemble regressors on the same split; lower RMSE/MAE is better.

Model	R^2	RMSE	MAE
Random Forest (OOB R^2 0.7513)	0.7415	0.5908	0.3999
XGBoost	0.8025	0.5165	0.3467
AdaBoost	0.5888	0.7452	0.5842
CatBoost	0.8029	0.5160	0.3535

Table 19.4: Clustering algorithms on the same two features.

Algorithm	Silhouette	Davies–Bouldin	Calinski–Harabasz
K-Means	0.5547	0.5722	248.65
Modified K-Means (97th pct. outlier flag)	0.5547	0.5722	248.65 (6 flagged)
Hierarchical (Ward)	0.5538	0.5779	244.41
Fuzzy C-Means (FPC 0.6711)	0.5547	0.5722	248.65

Representative results across the remaining families

Discussion and Conclusions

1. On small tabular classification, simple boosting (AdaBoost) or bagging (RFC) match the heavier models; differences are within noise for 76 test samples.
2. On larger regression data, gradient boosting clearly wins ($R^2 \approx 0.80$ vs. 0.74 for random forests and 0.59 for AdaBoost).
3. Clustering algorithms agree on well-separated data (silhouette ≈ 0.55); density methods are the only option for arbitrary shapes and noise, and their clusters match real seismic zones without labels.
4. Sequence models confirm the expected ranking: LSTM > vanilla RNN, with the largest gap on the task that needs long-range memory.
5. Fair comparison requires identical splits, fixed seeds, untouched test sets and at least two metrics per task; no test-set model selection was performed.

Viva / Discussion Questions

1. Which model performed best overall?

It is task-dependent: AdaBoost/RFC on small classification, XGBoost/CatBoost on tabular regression, LSTM for sequences, RBF SVC for small high-dimensional classification and GRNN for smooth low-dimensional regression.

Table 19.5: Density-based algorithms on the same catalogue (haversine metric).

Algorithm	Clusters	Noise	Silhouette
DBSCAN ($\epsilon = 0.03$ rad)	55	13.5%	0.326
HDBSCAN	40	21.5%	0.631

Table 19.6: Headline result of each remaining laboratory experiment.

Experiment	Headline result
Self-training (7)	0.9298 $\rightarrow$ 0.9415 accuracy using 15% labels (ceiling 0.9766)
MLP (13)	scikit-learn 0.9630; PyTorch 0.9815 test accuracy
RNN vs LSTM (14)	forecasting RMSE 69.93 vs 44.12; classification 0.9300 vs 0.9450
SOM (15)	quantization error 0.1965; topographic error 0.0393
HMM (16)	EM converged; log-likelihood 6019.79 on 1,859 DAX days
SVM (17)	SVC 0.9790 accuracy; SVR R^2 0.8968
LLM (18)	SMS spam fine-tune 0.9600 accuracy / 0.9592 F1
GRNN (19)	$\sigma = 0.08$ by 5-fold CV; held-out R^2 0.7255

2. Why not select the model by its test score?

That leaks the test set into model selection; candidates should be compared on validation data and evaluated once on the test set.

3. Why report more than one metric?

Each metric captures a different failure mode: accuracy hides class imbalance, RMSE weights outliers more than MAE, and silhouette favours compact convex clusters.

4. How is reproducibility ensured?

Fixed seeds everywhere, committed datasets, documented preprocessing and splits, and complete code listings for every experiment.

5. How would you detect data leakage in a comparison like this?

Check that every preprocessing step that learns from data (scaling, encoding, feature selection) is fitted on the training split only, and that model selection never touches the test set.

Appendix A: Quick Reference Formulas

Concept	Formula / Definition
Euclidean distance	$d(x, y) = \sqrt{\sum_j (x_j - y_j)^2}$
K-means objective	$J = \sum_i \ x_i - \mu_{c_i}\ ^2$
Regression MAE	$\text{MAE} = \frac{1}{n} \sum_i y_i - \hat{y}_i $
Regression RMSE	$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$
R^2	$R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$
Precision	$\text{TP} / (\text{TP} + \text{FP})$
Recall	$\text{TP} / (\text{TP} + \text{FN})$
F1	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$
SVM margin idea	Maximize margin while penalizing violations via C
GRNN kernel	$w_i(x) = \exp(-\ x - x_i\ ^2 / (2\sigma^2))$

Appendix B: Reproducibility Checklist

- Python version recorded.

- Package versions recorded.
- Random seed recorded where supported (all experiments use seed 42).
- Dataset source and version recorded (public datasets committed under data/).
- Preprocessing steps recorded (scaling fitted on training data only; haversine radians for geospatial data).
- Train/validation/test protocol recorded (stratified, chronological, early-stopping split, 5-fold CV).
- Hyperparameters recorded.
- Evaluation metrics defined before final testing.
- Plots and tables saved with meaningful names.
- Code runs from a clean environment or requirements file.

Appendix C: Suggested Assessment Rubric

Component	Suggested Weight
Pre-lab preparation / algorithm understanding	10%
Correct implementation	25%
Experimental design and preprocessing	15%
Evaluation and visualization	15%
Analysis and interpretation	20%
Report quality / reproducibility	10%
Viva / discussion	5%

Appendix D: Student Experiment Record

Field	Student Entry
Name	
ID	
Section	
Experiment No.	
Date	
Dataset	
Algorithm / Version	
Key Hyperparameters	
Random Seed	
Main Result	
Observation	
Instructor Signature	